Vivante Programming:

VGLite®

Vector Graphics API

Document Revision 2.19
30 April 2025

This document, for use with the VGLite Graphics API release versions from document date and is compatible with VGLite Driver V4 and
Vivante Vector Graphics IP including:
GCNanoUltraV, GCNanoLiteV (V1311p), GCNanoV, GC355 and GC555.

Legal Notices

COPYRIGHT INFORMATION

This document contains proprietary information of Vivante Corporation and VeriSilicon Holdings Co., Ltd. They reserve the right to make changes to any products herein at any time without notice and do not assume any responsibility or liability arising out of the application or use of any product described herein, except as expressly agreed to in writing by Vivante and/or VeriSilicon; nor does the purchase or use of a product from Vivante or VeriSilicon convey a license under any patent rights, copyrights, trademark rights, or any other of the intellectual property rights of Vivante, VeriSilicon or third parties.

DISCLOSURE/RE-DISTRIBUTION LIMITATIONS

The information contained in this Vivante proprietary copyright document may be re-distributed by Vivante and VeriSilicon customer SoC vendors who have licensed the related core IP, with the understanding that the material be re-packaged/re-branded as a licensee (SoC vendor) written/labeled document. In the adapted document please identify the technology and features described in this document using the Vivante trademark and include a reference to the copyright owner, for example: "This section describes the Vivante® GPU and contains copyright material disclosed with permission of VeriSilicon®."
(VeriSilicon Distribution **LEVEL A: PROPRIETARY INFORMATION**)

TRADEMARK ACKNOWLEDGMENT

VeriSilicon® and the VeriSilicon logo design are the trademarks or the registered trademarks of VeriSilicon Holdings Co., Ltd. Vivante® is a registered trademark of Vivante Corporation. All other brand and product names may be trademarks of their respective companies.

For our current distributors, sales offices, design resource centers, and product information, visit our web page located at http://www.verisilicon.com.

Table of Contents

List of Figures

List of Tables

1 Introduction

Vivante's platform independent VGLite Graphics API (Application Programming Interface) is designed to support 2D vector and 2D raster-based operations for rendering interactive user interface that may include menus, fonts, curves, and images. Its goal is to provide the maximum 2D vector/raster rendering performance, while keeping the memory footprint to the minimum. The Vivante VGLite Graphics API allows users to implement customized applications for mobile or IoT devices that include Vivante Vector Graphics IP.

1.1 Vivante VGLite Graphics API

The Vivante VGLite Graphics API is used to control the Vivante vector graphics hardware units, which provide accelerated vector and raster operations.

The Vivante VGLite API is developed for use with Vivante GCNanoLiteV, GCNanoUltraV, GCNanoV, GC355 and GC555 hardware. GC355 and GC555 support the Khronos OpenVG 1.1 feature set, while GCNanoLiteV, GCNanoUltraV and GCNanoV have a feature set smaller than that required to pass Khronos OpenVG CTS.

The VGLite API driver V4 is a completely new design and implementation of the driver (from 2023Q1) to support the new generation 2.5D GPU (GC555), as well as the previous 2.5D GPU releases (GC255, GC265, GC355). The new V4 driver supports the new and improved VGLite API (version 3.0), and can generate the most CPU efficient, customized driver build for a specific 2.5D GPU release based on the hardware feature set.

VGLite API supported features include Porter-Duff Blending, Gradient Controls, Fast Clear, Arbitrary Rotations, Path Filling rules, Path painting, and Pattern Path Filling.

By default, VGLite API driver V4 supports one implicit global application context in single thread. VGLite V4 driver does not support multi-threaded applications which is not suitable for embedded IoT devices.

1.2 API Function Group

The Vivante VGLite Graphics API has been designed to have independent function groups. It is permissible for users to use only one of the function groups in the VGLite application.

- **Initialization** – Used for initializing hardware and software structures.
- **Blit API** – Used for the raster part of rendering.
- **Draw API** – Used for 2D vector-based draw operations.

1.3 API Files

For customers with access to source:

The Vivante VGLite Graphics API functions are defined in the header file **`VGLite/inc/vg_lite.h`**.

All VGLite application-used enumerations and data types are also defined in **`VGLite/inc/vg_lite.h`**.

1.4 Hardware Versions

The Vivante VGLite API is compatible with a range of Vivante Vector Graphics IPs including: GCNanoLiteV, GCNanoUltraV, GCNanoV, GC355 and GC555.

Note that a specific hardware version has customized feature set which may limit hardware support for some VGLite API options. The VGLite application can use vg_lite_query_feature API to query specific VGLite feature availability.

Users can also check the **VGLite/VGLite/vg_lite_options.h** file which includes CHIPID, REVISION, CID to identify specific HW releases, and gcFEATURE_VG_* macros to define the feature set for the HW release.

The gcFEATURE_VG_* macro values (except for a few SW features) should NOT be changed. Otherwise, the VGLite driver will not function correctly on the specific HW release. Users can change the “SW Features” macro values to disable some software features, unnecessary error checks, or enable VGLite API trace for debug purposes.

2 Common Parameters and Error Values

2.1 Common Parameters

Vivante VGLite Graphics API uses a naming convention scheme wherein definitions are preceded by '**vg_lite**'.

Below is the list of most proprietary defined types and structures in the drivers. Not all may be used in the API functions.

Table 1. Common Parameter Types

Name	Typedef	Value
vg_lite_bool_t	int	A signed 32-bit integer 0: FALSE; 1: TRUE.
vg_lite_int8_t	char	A signed 8-bit integer
vg_lite_uint8_t	unsigned char	An unsigned 8-bit integer
vg_lite_int16_t	short	A signed 16-bit integer
vg_lite_uint16_t	unsigned short	An unsigned 16-bit integer
vg_lite_int32_t	int	A signed 32-bit integer
vg_lite_uint32_t	unsigned int	An unsigned 32-bit integer
vg_lite_uint64_t	unsigned long long	An unsigned 64-bit integer
vg_lite_float_t	float	A 32-bit single precision floating point number
vg_lite_double_t	double	A 64-bit double precision floating point number
vg_lite_char_t	char	A signed 8-bit integer
vg_lite_string	char*	A pointer to a character string
vg_lite_pointer	void*	A generic address pointer (void *). On 32-bit OS, it is a 32-bit address pointer. On 64-bit OS, it is a 64-bit address pointer.
vg_lite_void	void	The void type
vg_lite_color_t	vg_lite_uint32_t	A 32-bit color value The color value specifies the color used in various functions. The color is formed using 8-bit RGBA channels. The red channel is in the lower 8-bit of the color value, followed by the green and blue channels. The alpha channel is in the upper 8-bit of the color value. **vg_lite_color_t**: 31:24 A; 23:16 B; 15:8 G; 7:0 R For L8 target formats, the RGB color is converted to L8 by using the default ITU-R BT.709 conversion rules.
VG_LITE_S8	enum vg_lite_format_t	A signed 8-bit integer coordinate
VG_LITE_S16	enum vg_lite_format_t	A signed 16 bit integer coordinate
VG_LITE_S32	enum vg_lite_format_t	A signed 32-bit integer coordinate
VG_LITE_FP32	enum vg_lite_format_t	A 32-bit floating point coordinate

2.2 Enumeration used for Error Reporting

2.2.1 vg_lite_error_t Enumeration

Most functions in the API include an error status via the **vg_lite_error_t** enumeration. API functions return the status of the command and will report **VG_LITE_SUCCESS** if successful with no errors. Possible error values include the values in the table below.

Used in many functions, including initialization, flush, blit, draw, gradient and pattern functions.

vg_lite_error_t String Values	Description
VG_LITE_GENERIC_IO	Cannot communicate with the kernel driver
VG_LITE_INVALID_ARGUMENT	Invalid argument specified
VG_LITE_MULTI_THREAD_FAIL	Multi-thread/tasks fail *(available from June 2020)*
VG_LITE_NO_CONTEXT	No context specified
VG_LITE_NOT_SUPPORT	Function call is not supported. Hardware support is not available.
VG_LITE_OUT_OF_MEMORY	Out of memory (driver heap)
VG_LITE_OUT_OF_RESOURCES	Out of resources (OS heap)
VG_LITE_SUCCESS	Successful with no errors
VG_LITE_TIMEOUT	Timeout
VG_LITE_ALREADY_EXISTS	Object already exists *(available from August 2021)*
VG_LITE_NOT_ALIGNED	Data alignment error *(available from August 2021)*

3 Hardware Product and Feature Information

These query functions can be used to identify the product and its key features, and to get VGLite driver information.

3.1 Enumerations for Product and Feature Queries

3.1.1 vg_lite_feature_t Enumeration

The following feature values may be queried for availability in compatible hardware. *(expanded March 2023 to support additional hardware for driver V4)*

Used in information functions: vg_lite_query_feature.

vg_lite_feature_t String Values	Description
gcFEATURE_BIT_VG_16PIXELS_ALIGN	Require 16 pixels aligned for input pixel buffer
gcFEATURE_BIT_VG_24BIT	RGB888 or RGBA5658 formats support
gcFEATURE_BIT_VG_24BIT_PLANAR	24-bit planar formats support
gcFEATURE_BIT_VG_AYUV_INPUT	AYUV input format support
gcFEATURE_BIT_VG_BORDER_CULLING	Border culling support
gcFEATURE_BIT_VG_COLOR_KEY	Color key support.
gcFEATURE_BIT_VG_COLOR_TRANSFORMATION	Color transform support.
gcFEATURE_BIT_VG_DEC_COMPRESS	DEC compression format output support
gcFEATURE_BIT_VG_DITHER	Dither support
gcFEATURE_BIT_VG_DOUBLE_IMAGE	Support two image source inputs
gcFEATURE_BIT_VG_FLEXA	FLEXA interface support
gcFEATURE_BIT_VG_GAMMA	Gamma support
gcFEATURE_BIT_VG_GAUSSIAN_BLUR	Gaussian blur sampling support
gcFEATURE_BIT_VG_GLOBAL_ALPHA	Global alpha support
gcFEATURE_BIT_VG_HW_PREMULTIPLY	HW supports alpha premultiply for image
gcFEATURE_BIT_VG_IM_DEC_INPUT	DEC compressed format input support
gcFEATURE_BIT_VG_IM_FASTCLEAR	Fast Clear support
gcFEATURE_BIT_VG_IM_INDEX_FORMAT	Index format support for image
gcFEATURE_BIT_VG_IM_INPUT	Blit and draw API support
gcFEATURE_BIT_VG_IM_REPEAT_REFLECT	Image repeat reflect mode support
gcFEATURE_BIT_VG_INDEX_ENDIAN	Index format endian support
gcFEATURE_BIT_VG_LINEAR_GRADIENT_EXT	Support for extended linear gradient capabilities
gcFEATURE_BIT_VG_LVGL_SUPPORT	LVGL blend mode support
gcFEATURE_BIT_VG_MASK	Mask support
gcFEATURE_BIT_VG_MIRROR	Mirror support

vg_lite_feature_t String Values	Description
gcFEATURE_BIT_VG_NEW_BLEND_MODE	New blend mode DARKEN/LIGHTEN support
gcFEATURE_BIT_VG_NEW_IMAGE_INDEX	New CLUT image index support
gcFEATURE_BIT_VG_PARALLEL_PATHS	New parallel path HW support
gcFEATURE_BIT_VG_PE_CLEAR	Pixel engine clear support
gcFEATURE_BIT_VG_PIXEL_MATRIX	Pixel matrix support
gcFEATURE_BIT_VG_QUALITY_8X	8x anti-aliasing path support
gcFEATURE_BIT_VG_RADIAL_GRADIENT	Radial gradient support
gcFEATURE_BIT_VG_RECTANGLE_TILED_OUT	Rectangle tiled output support
gcFEATURE_BIT_VG_RGBA2_FORMAT	RGBA2222 format support
gcFEATURE_BIT_VG_RGBA8_ETC2_EAC	ETC2/EAC compressed image format support
gcFEATURE_BIT_VG_SCISSOR	Scissor support
gcFEATURE_BIT_VG_SRC_PREMULTIPLIED	Source image alpha pre-multiplied
gcFEATURE_BIT_VG_STENCIL	Stencil image mode support
gcFEATURE_BIT_VG_STRIPE_MODE	Stripe mode support
gcFEATURE_BIT_VG_TESSELLATION_TILED_OUT	Tessellation tiled output support
gcFEATURE_BIT_VG_USE_DST	Read destination pixel support
gcFEATURE_BIT_VG_YUV_INPUT	YUV input format support
gcFEATURE_BIT_VG_YUV_OUTPUT	YUV format output support
gcFEATURE_BIT_VG_YUV_TILED_INPUT	YUV tiled input format support
gcFEATURE_BIT_VG_YUY2_INPUT	YUY2 input format support

3.2 Structures for Product and Feature Queries

3.2.1 vg_lite_info_t Structure

This structure is used to query VGLite driver information.

Used in function: vg_lite_get_info_t.

vg_lite_info_t Members	Type	Description
api_version	vg_lite_uint32_t	VGLite API version
header_version	vg_lite_uint32_t	VGLite header version
release_version	vg_lite_uint32_t	VGLite driver release version
reserved	vg_lite_uint32_t	Reserved for future use

3.3 Functions for Product and Feature Queries

vg_lite_get_product_info

Description:
This function is used to identify the VGLite compatible product.

Syntax:

```
vg_lite_uint32_t vg_lite_get_product_info (
    vg_lite_char              *name
    vg_lite_uint32_t          *chip_id
    vg_lite_uint32_t          *chip_rev
);
```

Parameters:

***name**	Character array to store the name of the chip.
***chip_id**	Stores an ID number for the chip.
***chip_rev**	Stores a revision number for the chip.

vg_lite_get_info

Description:
This function is used to query the VGLite driver information.

Syntax:

```
vg_lite_error_t vg_lite_get_info (
    vg_lite_info_t            *info
);
```

Parameters:

***info**	Points to the VGLite driver information structure which includes the API version, header version, and release version.

vg_lite_get_register

Description:
This function can be used to read a Vivante Vector Graphics register value given the AHB Byte address of a register. Refer to Vivante Vector Graphics Accessible Register specification documents compatible with your IP for register descriptions. The value range of AHB/APB accessible addresses for VGLite cores is usually 0x0 to 0x1FF and 0xA00 to 0xA7F.

Syntax:

```
vg_lite_error_t vg_lite_get_register (
    vg_lite_uint32_t            address
    vg_lite_uint32_t            *result
);
```

Parameters:

address	Byte Address of the register whose value you want.
***result**	The registers value.

vg_lite_query_feature

Description:
This function is used to query if a specific feature is available.

Syntax:

```
vg_lite_uint32_t vg_lite_query_feature (
    vg_lite_feature_t        feature
);
```

Parameters:

feature	Feature being queried, as detailed in enum vg_lite_feature_t.

Returns:
Either the feature is not supported (0) or supported (1).

vg_lite_get_mem_size

Description:

This function queries whether or not there is any remaining allocated contiguous video memory. *(available from June 2020)*

Syntax:

```
vg_lite_error_t vg_lite_get_mem_size(
    vg_lite_uint32_t            *size
);
```

Parameters:

size	Pointer to the remaining allocated contiguous video memory.

Returns:

Returns VG_LITE_SUCCESS if the query is successful and memory is available. Returns VG_LITE_NO_CONTEXT if the driver is not initialized, or there is no available memory.

4 API Control

Before calling any VGLite API function, the application must initialize the VGLite implicit (global) context by calling *vg_lite_init()*, which will fill in a features table, reset the fast-clear buffer, reset the compositing target buffer, as well as allocate the command and tessellation buffers.

The VGLite driver only supports one current context and one thread to issue commands to the Vivante Vector Graphics hardware. The VGLite driver does not support multiple concurrent contexts running simultaneously in multiple threads/processes, as the VGLite kernel driver does not support context switching. A VGLite application can only use a single context at any time to issue commands to the Vivante Vector Graphics hardware. If a VGLite application needs to switch contexts, **vg_lite_close()** should be called to close the current context in the current thread, then **vg_lite_init()** can be called to initialize a new context either in the current thread or from another thread/process.

4.1 Context Initialization and Control Functions

vg_lite_init

Description:
This function initializes the memory and data structures needed for VGLite draw/blit functions, by allocating memory for the command buffer and a tessellation buffer of the specified size.

GC555 and GCNanoUltraV have a newly designed hardware tessellation module which requires significantly less memory for the tessellation buffer than GC355 and GCNanoLite. Specifically, the GC555 and GCNanoUltraV required tessellation buffer size is "buffer_height * 128 byte". **vg_lite_init** API can simply be called with render buffer “width” and “height” as the input parameters for GC555 and GCNanoUltraV. This will result in the best path tessellation performance.

GC355 and GCNanoLiteV hardware tessellation module requires a tessellation buffer with size "buffer_height * buffer_width * 8 byte". If system memory is limited, the application can define a smaller tessellation window based on the amount of memory available. GPU hardware can process the entire render buffer path tessellation in multiple passes with the tessellation window sliding across the render buffer. The multi-pass path tessellation with the smaller tessellation window has a certain performance overhead.

The minimum tessellation window that can be used is 16x16. If tess_height or tess_width is less than 0 in **vg_lite_init** API, then no path tessellation buffer is created and path drawing APIs do not work, only blit APIs can be used after **vg_lite_init**.

If this would be the first context that accesses the hardware, the hardware will be turned on and initialized. If a new context needs to be initialized**, vg_lite_close** must be called to close the current context. Otherwise, **vg_lite_init** will return an error.

Syntax:

```
vg_lite_error_t vg_lite_init (
    vg_lite_int32_t             tess_width,
    vg_lite_int32_t             tess_height
);
```

Parameters:

tess_width	Width of tessellation window. Maximum cannot be greater than render buffer width. If less than or equal to 0, then no tessellation buffer is created, in which case only blit APIs can be used afterwards.
tess_height	Height of tessellation window. Maximum cannot be greater than render buffer height. If less than or equal to 0, then no tessellation buffer is created, in which case blit APIs can be used afterwards.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_close

Description:
This function will deallocate all the resources and free up all the memory that was initialized earlier by the **vg_lite_init** function. It will also turn OFF the hardware automatically if this was the only active context.

Syntax:

```
vg_lite_error_t vg_lite_close (
    void
);
```

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_flush

Description:
This function explicitly submits the command buffer to the GPU without waiting for it to complete. *(From Dec 2019, return type is vg_lite_error_t, previously was void.)*

Syntax:

```
vg_lite_error_t vg_lite_flush (
    void
);
```

Returns:
Returns VG_LITE_SUCCESS if the flush is successful. See vg_lite_error_t enum for other return codes.

vg_lite_finish

Description:
This function explicitly submits the command buffer to the GPU and waits for it to complete.

Syntax:

```
vg_lite_error_t vg_lite_finish (
    void
);
```

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_frame_delimiter

Description:
This function sets a flag for GPU to signal the completion of the current frame. A **vg_lite_finish** is called by default within this API. The enum **VG_LITE_FRAME_END_FLAG** is the only value that can be set by flag parameter.

Syntax:

```
vg_lite_error_t vg_lite_frame_delimiter (
    vg_lite_frame_flag_t flag
);
```

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_set_command_buffer_size

Description:
This function is optional. If used, call it before **vg_lite_init** if you want to change the command buffer size. *(available from March 2020)*

This function is useful for devices where memory is limited and less than the default. The VGLite Command buffer is set to 32K by default, so that VGLite applications can render more complex paths with better performance. This function can be used to adjust the command buffer size to fit specific application and system/device requirements.

Syntax:

```
vg_lite_error_t vg_lite_set_command_buffer_size (
    vg_lite_uint32_t            size
);
```

Parameters:

size	Size of the VGLite Command buffer. Default is 64K.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_set_command_buffer

Description:
This function sets a user-defined external memory buffer (physical, 64-byte aligned) as the VGLite command buffer. By default, the VGLite driver allocates a static command buffer internally. Thus, it is not necessary for an application to allocate and set the command buffer. This function is only used for devices where an application needs to allocate the command buffer dynamically. *(from December 2021)*

Syntax:

```
vg_lite_error_t vg_lite_set_command_buffer (
    vg_lite_uint32_t            physical,
    vg_lite_uint32_t            size
);
```

Parameters:

physical	The physical address of a memory buffer. The address must be 64-byte aligned.
size	The size of memory buffer. The size must be 128-byte aligned.

Returns:
Returns VG_LITE_SUCCESS if the command buffer set is successful. See vg_lite_error_t enum for other return codes.

vg_lite_set_tess_buffer

Description:
This function specifies a memory buffer from an application as the VGLite driver's tessellation buffer. By default, the VGLite driver allocates a static tessellation buffer internally. Thus, it is not necessary for an application to allocate and set the tessellation buffer. This function is only used for devices where the application needs to allocate the tessellation buffer dynamically. *(from December 2021)*

Syntax:

```
vg_lite_error_t vg_lite_set_tess_buffer (
    vg_lite_uint32_t            physical,
    vg_lite_uint32_t            size
);
```

Parameters:

physical	The physical address of a tessellation buffer. The address must be 64-byte aligned.
size	The size of tessellation buffer. tessellation buffer size = target buffer's height * 128B.

Returns:
Returns VG_LITE_SUCCESS if the tessellation buffer set is successful. See vg_lite_error_t enum for other return codes.

vg_lite_set_memory_pool

Description:
This function sets the specific memory pool from which certain type of buffers, VG_LITE_COMMAND_BUFFER, VG_LITE_TESSELLATION_BUFFER, or VG_LITE_RENDER_BUFFER, should be allocated. By default, all types of buffers are allocated from VG_LITE_MEMORY_POOL_1. This API must be called before vg_lite_init() for setting VG_LITE_COMMAND_BUFFER or VG_LITE_TESSELLATION_BUFFER memory pools. This API can be called anytime for VG_LITE_RENDER_BUFFER to affect the following vg_lite_allocate() calls. *(from December 2023)*

Syntax:

```
vg_lite_error_t vg_lite_set_memory_pool (
    vg_lite_buffer_type_t       type,
    vg_lite_memory_pool_t       pool
);
```

Parameters:

type	The buffer type (VG_LITE_COMMAND_BUFFER, VG_LITE_TESSELLATION_BUFFER, or VG_LITE_RENDER_BUFFER) to be allocated from memory pool.
pool	The memory pool (VG_LITE_MEMORY_POOL_1, VG_LITE_MEMORY_POOL_2) from which the buffer type should be allocated.

Returns:

Returns VG_LITE_SUCCESS if the memory pool set is successful. See vg_lite_error_t enum for other return codes.

5 Pixel Buffers

5.1 Pixel Buffer Alignment

The VGLite hardware requires the pixel buffer start address and stride to be properly byte aligned to work correctly. The start address and stride alignment requirement for a pixel buffer depends on the specific pixel format, and gcFEATURE_VG_16PIXELS_ALIGNED value (0/1) in vg_lite_options.h file.

Refer to the Alignment Notes Table 2: Image Source Start Address and Stride Alignment Summary later in this document.

5.2 Pixel Cache

The Vivante Imaging Engine (IM) includes two fully associative caches. Each cache has 8 lines, each line has 64 bytes. In this case, one cache line can hold either a 4x4-pixel Tile or a 16x1-pixel row.

5.3 Internal Representation

For non 32-bit color formats, each pixel will be extended to 32-bits as such:

If the source and destination formats have the same color format, but differ in the number of bits per color channel, the source channel is multiplied by $(2^d - 1)/(2^s - 1)$ and rounded to the nearest integer, where:

d is the number of bits in the destination channel
s is the number of bits in the source channel

Example: a b11111 5-bit source channel gets converted to an 8-bit destination b11111000.
The YUV formats are internally converted to RGB. Pixel selection is unified for all formats by using the LSB of the coordinate.

5.4 Pixel Buffer Enumerations

5.4.1 vg_lite_buffer_format_t Enumeration

This enumeration specifies the color format to use for a buffer. This applies to both Image and Render Target. Formats include supported swizzles for RGB. For YUV swizzles, use the related values and parameters in vg_lite_swizzle_t.

Application can use vg_lite_query_feature API to determine support for some hardware dependent formats. For example, related vg_lite_feature_t enum values include gcFEATURE_BIT_VG_RGBA2_FORMAT and gcFEATURE_BIT_VG_IM_INDEX_FORMAT.

NOTE: See Alignment Notes Table 2 following the value descriptions for alignment requirements summary for the image formats. *(alignment columns refined March and Sept 2023)*

Used in structure: vg_lite_buffer_t.
See also vg_lite_blit, vg_lite_clear, vg_lite_draw.

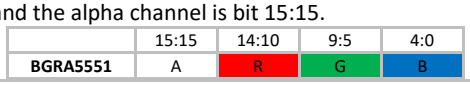
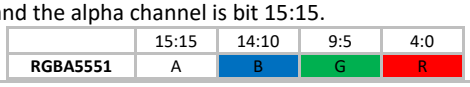
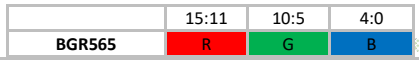
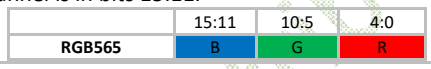
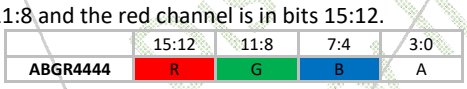
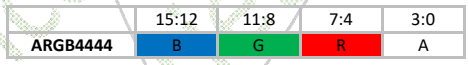
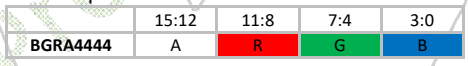
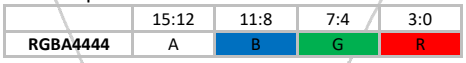
OpenVG VGImageFormat Note: The bits for each color channel are stored within a machine word from MSB to LSB in the order indicated by the pixel format name. This is the opposite of the original VG_LITE_* formats which are ordered from LSB to MSB. Formats with the same organization are listed in the same row as their VG_Lite counterparts.
Original VGLite API Image Format Note: The bits for each color channel are stored within a machine word from LSB to MSB in the order indicated by the pixel format name. This is the opposite of the OPENVG VG_* formats which are ordered from MSB to LSB.

The following codes, as also used in OpenVG 1.1 Specification Table 11, are used for format description:

- A Alpha channel
- B Blue color channel
- G Green color channel
- R Red color channel
- X Uninterpreted padding byte or bit
- L Grayscale
- BW 1-bit black and white
- l Linear color space
- s Non-linear (sRGB) color space
- PRE Alpha values are premultiplied

vg_lite_buffer_format_t String Value	Description	Supported as Source	Supported as Dest	Start Address / Stride Alignment: Bytes
VG_LITE_ABGR8888 VG_sRGBA_8888 VG_sRGBA_8888_PRE VG_lRGBA_8888 VG_lRGBA_8888_PRE	32-bit ABGR format with 8 bits per color channel. Alpha is in bits 7:0, blue in bits 15:8, green in bits 23:16, and the red channel is in bits 31:24. ABGR8888: 31:24 R, 23:16 G, 15:8 B, 7:0 A	Yes	Yes	Start 4B / Stride 64B

vg_lite_buffer_format_t String Value	Description	Supported as Source	Supported as Dest	Start Address / Stride Alignment: Bytes
VG_LITE_ARGB8888 VG_sBGRA_8888 VG_sBGRA_8888_PRE VG_lBGRA_8888 VG_lBGRA_8888_PRE	32-bit ARGB format with 8 bits per color channel. Alpha is in bits 7:0, red in bits 15:8, green in bits 23:16, and the blue channel is in bits 31:24. 31:24 23:16 15:8 7:0 ARGB8888 B G R A	Yes	Yes	Start 4B / Stride 64B
VG_LITE_BGRA8888 VG_sARGB_8888 VG_sARGB_8888_PRE VG_lARGB_8888 VG_lARGB_8888_PRE	32-bit BGRA format with 8 bits per color channel. Blue in bits 7:0, green in bits 15:8, red is in bits 23:16, and the alpha channel is in bits 31:24. 31:24 23:16 15:8 7:0 BGRA8888 A R G B	Yes	Yes	Start 4B / Stride 64B
VG_LITE_RGBA8888 VG_sABGR_8888 VG_sABGR_8888_PRE VG_lABGR_8888 VG_lABGR_8888_PRE	32-bit RGBA format with 8 bits per color channel. Red is in bits 7:0, green in bits 15:8, blue in bits 23:16, and the alpha channel is in bits 31:24. 31:24 23:16 15:8 7:0 RGBA8888 A B G R	Yes	Yes	Start 4B / Stride 64B
VG_LITE_BGRX8888 VG_sXRGB_8888 VG_lXRGB_8888	32-bit BGRX format with 8 bits per color channel. Blue in bits 7:0, green in bits 15:8, red is in bits 23:16, and the X channel is in bits 31:24. 31:24 23:16 15:8 7:0 BGRX8888 X R G B	Yes	Yes	Start 4B / Stride 64B
VG_LITE_RGBX8888 VG_sXBGR_8888 VG_lXBGR_8888	32-bit RGBX format with 8 bits per color channel. Red is in bits 7:0, green in bits 15:8, blue in bits 23:16, and the X channel is in bits 31:24. 31:24 23:16 15:8 7:0 RGBX8888 X B G R	Yes	Yes	Start 4B / Stride 64B
VG_LITE_XBGR8888 RGBX VG_sRGBX_8888 VG_lRGBX_8888	32-bit XBGR format with 8 bits per color channel. X channel is in bits 7:0, blue in bits 15:8, green in bits 23:16, and the red channel is in bits 31:24. 31:24 23:16 15:8 7:0 XBGR8888 R G B X	Yes	Yes	Start 4B / Stride 64B
VG_LITE_XRGB8888 VG_sBGRX_8888 VG_lBGRX_8888	32-bit XRGB format with 8 bits per color channel. X channel is in bits 7:0, red in bits 15:8, green in bits 23:16, and the blue channel is in bits 31:24. 31:24 23:16 15:8 7:0 XRGB8888 B G R X	Yes	Yes	Start 4B / Stride 64B
VG_LITE_ABGR1555 VG_sRGBA_5551	16-bit ABGR format with 5 bits per color channel and one bit alpha. Alpha channel is in bit 0:0, blue in bits 5:1, green in bits 10:6 and the red channel is in bits 15:11. 15:11 10:6 5:1 0:0 ABGR5551 R G B A	Yes	Yes	Start 4B / Stride 32B
VG_LITE_ARGB1555 VG_sBGRA_5551	16-bit ARGB format with 5 bits per color channel and one bit alpha. The alpha channel is bit 0:0, red in bits 5:1, green in bits 10:6 and the blue channel is in bits 15:11. 15:11 10:6 5:1 0:0 ARGB5551 B G R A	Yes	Yes	Start 4B / Stride 32B

vg_lite_buffer_format_t String Value	Description	Supported as Source	Supported as Dest	Start Address / Stride Alignment: Bytes
VG_LITE_BGRA5551 VG_sARGB_1555	16-bit BGRA format with 5 bits per color channel and one bit alpha. Blue is in bit 4:0, green in bits 9:5,red in bits 14:0 and the alpha channel is bit 15:15. 	Yes	Yes	Start 4B / Stride 32B
VG_LITE_RGBA5551 VG_sABGR_1555	16-bit RGBA format with 5 bits per color channel and one bit alpha. Red is in bit 4:0, green in bits 9:5, blue in bits 14:0 and the alpha channel is bit 15:15. 	Yes	Yes	Start 4B / Stride 32B
VG_LITE_BGR565 VG_sRGB_565	16-bit BGR format with 5 and 6 bits per color channel. Blue is in bits 4:0, green in bits 10:5 and the red channel is in bits 15:11. 	Yes	Yes	Start 4B / Stride 32B
VG_LITE_RGB565 VG_sBGR_565	16-bit RGB format with 5 or 6 bits per color channel. Red is in bits 4:0, green in bits 10:5 and the blue channel is in bits 15:11. 	Yes	Yes	Start 4B / Stride 32B
VG_LITE_ABGR4444 VG_sRGBA_4444	16-bit ABGR format with 4 bits per color channel. Alpha is in bits 3:0, blue in bits 7:4, green in bits 11:8 and the red channel is in bits 15:12. 	Yes	Yes	Start 4B / Stride 32B
VG_LITE_ARGB4444 VG_sBGRA_4444	16-bit ARGB format with 4 bits per color channel. Alpha is in bits 3:0, red in bits 7:4, green in bits 11:8 and the blue channel is in bits 15:12. 	Yes	Yes	Start 4B / Stride 32B
VG_LITE_BGRA4444 VG_sARGB_4444	16-bit BGRA format with 4 bits per color channel. Red is in bits 11:8, green in bits 7:4, blue in bits 3:0 and the alpha channel is in bits 15:12. 	Yes	Yes	Start 4B / Stride 32B
VG_LITE_RGBA4444 VG_sABGR_4444	16-bit RGBA format with 4 bits per color channel. Red is in bits 3:0, green in bits 7:4, blue in bits 11:8 and the alpha channel is in bits 15:12. 	Yes	Yes	Start 4B / Stride 32B

vg_lite_buffer_format_t String Value	Description	Supported as Source	Supported as Dest	Start Address / Stride Alignment: Bytes
VG_LITE_YUY2 VG_LITE_YUYV	Packed YUV format, 32-bit for 2 pixels. Y0 is in bits 7:0 and V0 is in bits 31:23. (available for Source IMAGE only) YUY2: 31:24 V0 / V2; 23:16 Y1 / Y3; 15:8 U0 / U2; 7:0 Y0 / Y2	Yes	No	Start 4B / Stride 32B
VG_LITE_A4 VG_A_4	4-bit alpha format. There are no RGB values. A4: 3:0 A	Yes	No	Start 4B / Stride 8B
VG_LITE_A8 VG_A_8	8-bit alpha format. There are no RGB values. A8: 7:0 A	Yes	Yes	Start 4B / Stride 16B
VG_LITE_L8 VG_sL_8 VG_lL_8	8-bit grayscale format. There are no RGB values. L8: 7:0 A	Yes	Yes	Start 4B / Stride 16B

Hardware dependent formats for vg_lite_buffer_format_t	Description	Supported as Source	Supported as Dest	Start Address / Stride Alignment: Bytes
VG_LITE_ABGR2222	8-bit BGRA format with 2 bits per color channel. Alpha is in bits 1:0, blue in bits 3:2, green in bits 5:4 and the red channel is in bits 7:6. ABGR2222: 7:6 R; 5:4 G; 3:2 B; 1:0 A	Yes	Yes	Start 4B / Stride 16B
VG_LITE_ARGB2222	8-bit BGRA format with 2 bits per color channel. Alpha is in bits 1:0, red in bits 3:2,green in bits 5:4 and the blue channel is in bits 7:6. ARGB2222: 7:6 B; 5:4 G; 3:2 R; 1:0 A	Yes	Yes	Start 4B / Stride 16B
VG_LITE_BGRA2222	8-bit BGRA format with 2 bits per color channel. Blue is in bits 1:0, green in bits 3:2, red in bits 5:4 and the alpha channel is in bits 7:6. BGRA2222: 7:6 A; 5:4 R; 3:2 G; 1:0 B	Yes	Yes	Start 4B / Stride 16B
VG_LITE_RGBA2222	8-bit RGBA format with 2 bits per color channel. Red is in bits 1:0, green in bits 3:2, blue in bits 5:4 and the alpha channel is in bits 7:6. RGBA2222: 7:6 A; 5:4 B; 3:2 G; 1:0 R	Yes	Yes	Start 4B / Stride 16B
VG_LITE_INDEX_1	1-bit index format.	Yes	No	8B

Hardware dependent formats for vg_lite_buffer_format_t	Description	Supported as Source	Supported as Dest	Start Address / Stride Alignment: Bytes
VG_LITE_INDEX_2	2-bit index format.	Yes	No	both 8B
VG_LITE_INDEX_4	4-bit index format.	Yes	No	both 8B
VG_LITE_INDEX_8	8-bit index format.	Yes	No	both 16B
VG_LITE_NV12_TILED	Supertiled (8x8 pixels), planar YUV format, 96-bit for 4 pixels. Y plane is 32 bits for 4 pixels and is organized in 64 pixel super tiles (8x8 Y); UV plane is 64 bits for 4 pixels. Pixels are organized in super tiles are (4x4 UV pairs). (available for Source IMAGE only on supporting hardware) **Y Buffer** **Y Buffer** **UV Buffer**	Yes	No	Y: both 16 Bytes UV: both 8 Bytes
VG_LITE_ANV12_TILED	Pixel organization as NV12_TILED but with an Alpha Buffer, also supertiled. (available for Source IMAGE only on supporting hardware) **Alpha Buffer** **Y Buffer** **Y Buffer** **UV Buffer**	Yes	No	A, Y: both 16 Bytes UV: both 8 Bytes
VG_LITE_AYUY2_TILED	Supertiled (8x8) and packed YUV format with separate tiled Alpha Buffer. Y0 is in bits 7:0 and V is in bits 31:23. (available for Source IMAGE only on supporting hardware) **Alpha Buffer** **YUY2 Buffer**	Yes	No	both 32B
VG_LITE_RGB888	24-bit RGB format with 8 bits per color channel. Red is in bits 7:0, green in bits 15:8, blue in bits 23:16. **RGB888**	Yes	Yes	Start 4B / Stride 32B

Hardware dependent formats for vg_lite_buffer_format_t	Description	Supported as Source	Supported as Dest	Start Address / Stride Alignment: Bytes
VG_LITE_BGR888	24-bit RGB format with 8 bits per color channel. Blue is in bits 7:0, green in bits 15:8, red in bits 23:16. BGR888: 23:16 R, 15:8 G, 7:0 B	Yes	Yes	Start 4B / Stride 32B
VG_LITE_ARGB8565	24-bit RGBA format with 4 and 5 bits per color channel. Alpha channel is in bit 7:0, red in bits 12:8, green in bits 18:13 and the blue in bits 23:19. ARGB8565: 23:19 B, 18:13 G, 12:8 R, 7:0 A	Yes	Yes	Start 4B / Stride 32B
VG_LITE_BGRA5658	24-bit RGBA format with 4 and 5 bits per color channel. Blue is in bits 4:0, green in bits 10:5, red in bits 15:11,alpha channel is in bit 23:16. BGRA5658: 23:16 A, 15:11 R, 10:5 G, 4:0 B	Yes	Yes	Start 4B / Stride 32B
VG_LITE_ABGR8565	24-bit RGBA format with 4 and 5 bits per color channel. Alpha channel is in bit 7:0, blue in bits 12:8, green in bits 18:13 and the red in bits 23:19. ABGR8565: 23:19 R, 18:13 G, 12:8 B, 7:0 A	Yes	Yes	Start 4B / Stride 32B
VG_LITE_RGBA5658	24-bit RGBA format with 4 and 5 bits per color channel. Red is in bits 4:0, green in bits 10:5, blue in bits 15:11 and the alpha channel is in bits 23:16 RGBA5658: 23:16 A, 15:11 B, 10:5 G, 4:0 R	Yes	Yes	Start 4B / Stride 32B

5.4.2 Image Buffer Alignment Requirement

The image (or source) buffer alignment requirement depends on the specific pixel format, and some gcFEATURE_*_ALIGNED defines in the vg_lite_options.h file.

Table 2. Image Buffer Alignment Summary

Image Format	Bits per pixel	Source Tile Mode	Start Address Alignment Requirement in Bytes	Stride Alignment Requirement in Bytes	Buffer Height Alignment Requirement	Supported for Destination
VG_LITE_INDEX1	1	linear	8B	2B	1	
		tile	8B	1B	4	
VG_LITE_INDEX2	2	linear	8B	4B	1	
		tile	8B	1B	4	
VG_LITE_INDEX4	4	linear	8B	8B	1	
		tile	8B	2B	4	
VG_LITE_INDEX8	8	linear	16B	16B	1	
		tile	16B	4B	4	
VG_LITE_A4	4	linear	8B	8B	1	
		tile	8B	2B	4	
VG_LITE_A8	8	linear	16B	16B	1	Yes
		tile	16B	4B	4	Yes
VG_LITE_L8	8	linear	16B	16B	1	Yes
		tile	16B	4B	4	Yes
VG_LITE_ARGB2222	8	linear	16B	16B	1	Yes
		tile	16B	4B	4	Yes
VG_LITE_RGB565	16	linear	32B	32B	1	Yes
		tile	32B	8B	4	Yes
VG_LITE_ARGB1555	16	linear	32B	32B	1	Yes
		tile	32B	8B	4	Yes
VG_LITE_ARGB4444	16	linear	32B	32B	1	Yes
		tile	32B	8B	4	Yes
VG_LITE_ARGB8888	32	linear	64B	64B	1	Yes
		tile	64B	16B	4	Yes
VG_LITE_XRGB8888	32	linear	64B	64B	1	Yes
		tile	64B	16B	4	Yes
VG_LITE_ARGB8565	24	linear	64B	48B*	1	Yes
		tile	64B	12B*	4	Yes
VG_LITE_RGB888	24	linear	64B	48B*	1	Yes
		tile	64B	12B*	4	Yes
VG_LITE_YUY2/UYVY	16	linear	32B	32B	1	
		tile	32B	8B	4	
VG_LITE_NV12	12	linear	Y: 32B UV: 32B	Y: 32B UV: 32B	1	
VG_LITE_YV12	12	linear	Y: 32B U: 16B V: 16B	Y: 32B U: 16B V: 16B	1	
VG_LITE_NV16	16	linear	Y: 32B UV: 32B	Y: 32B UV: 32B	1	
VG_LITE_YV16	16	linear	Y: 32B U: 16B V: 16B	Y: 32B U: 16B V: 16B	1	

Image Format	Bits per pixel	Source Tile Mode	Start Address Alignment Requirement in Bytes	Stride Alignment Requirement in Bytes	Buffer Height Alignment Requirement	Supported for Destination
VG_LITE_YV24	24	linear	Y: 32B U: 32B V: 32B	Y: 32B U: 32B V: 32B	1	
VG_LITE_ETC2	8	tile	16B	4B	4	

*Note:

1. The values in the table reflect the alignment requirements of the data in memory. The stride of ARGB8888 / ARGB8565 is seen as 4Byte per pixel when configuring hardware.
2. For tile mode, stride is still the byte size of a row of pixels in the buffer instead of 4 rows.
3. When DECNano function enable for the buffer, the total buffer size need align to 64Byte*compression rate for ARGB8888 or XRGB8888 format, align to 48Byte*compress rate for RGB888 format.

Additional Alignment Requirement

1. Buffer starting address must be 16 pixel-byte-size aligned. I.e. 8 bit-per-pixel format buffer must be 16 bytes aligned; 16 bit-per-pixel format buffer must be 32 bytes aligned; 24 and 32 bit-per-pixel format buffer must be 64 bytes aligned.
2. For linear mode buffer, buffer stride must be 16 pixel-byte-size aligned.
3. For tile mode buffer, buffer width and height must be 4 pixel aligned so buffer width and height end at tile boundary.

5.4.3 Destination Buffer Alignment Requirement

The destination (or render target) buffer alignment requirement depends on the specific pixel format, and some gcFEATURE_*_ALIGNED defines in the vg_lite_options.h file.

Table 3. Destination Buffer Alignment Summary

Target Format	Bits per pixel	Target Tile Mode	VG Tile Mode	Start Address Alignment Requirement in Bytes	Stride Alignment Requirement in Bytes	Buffer Height Alignment Requirement
VG_LITE_A8	8	linear	linear	4B	1B	1
			tile	64B	64B	4
		tile	linear	64B	64B	4
			tile	64B	16B	4
VG_LITE_L8	8	linear	linear	4B	1B	1
			tile	64B	64B	4
		tile	linear	64B	64B	4
			tile	64B	16B	4
VG_LITE_ARGB2222	8	linear	linear	4B	1B	1
			tile	64B	64B	4
		tile	linear	64B	64B	4
			tile	64B	16B	4
VG_LITE_RGB565	16	linear	linear	4B	2B	1
			tile	64B	64B	4
		tile	linear	64B	64B	4
			tile	64B	16B	4
VG_LITE_ARGB1555	16	linear	linear	4B	2B	1
			tile	64B	64B	4
		tile	linear	64B	64B	4
			tile	64B	16B	4
VG_LITE_ARGB4444	16	linear	linear	4B	2B	1
			tile	64B	64B	4
		tile	linear	64B	64B	4
			tile	64B	16B	4
VG_LITE_ARGB8888	32	linear	linear	4B	4B	1
			tile	64B	64B	4
		tile	linear	64B	64B	4
			tile	64B	16B	4
VG_LITE_XRGB8888	32	linear	linear	4B	4B	1
			tile	64B	64B	4
		tile	linear	64B	64B	4
			tile	64B	16B	4
VG_LITE_ARGB8565	24	linear	linear	64B	3B*	1
			tile	64B	48B*	4
		tile	linear	64B	48B*	4
			tile	64B	12B*	4
VG_LITE_RGB888	24	linear	linear	64B	3B*	1
			tile	64B	48B*	4
		tile	linear	64B	48B*	4
			tile	64B	12B*	4

*Note:

1. The values in the table reflect the alignment requirements of pixel data in memory. The stride of ARGB8888/ARGB8565 is seen as 4 Bytes per pixel when configuring hardware.
2. For tile mode, buffer stride is still the byte size of a row of pixels instead of 4 rows of pixels.
3. For PE clear function, the clear size must align to 48 Bytes for RGB888 or ARGB8565 format.
4. For PE clear function with DECNano enabled, the clear size must align to 48 Bytes for RGB888, align to 64 Bytes for ARGB8888 or XRGB8888.
5. If DECNano function is enabled for the buffer, the target buffer start address needs to align to 64 Bytes.
6. If DECNano function is enabled for the buffer, the total buffer size needs to align to a 64 Byte*compression rate for ARGB8888 or XRGB8888 format and align to a 48 Byte*compression rate for RGB888 format.

Additional Alignment Requirement

1. Buffer starting address must be at least 4 Byte aligned. Buffer stride must be at least one pixel size aligned.
2. Buffer starting address must be 64Byte aligned for 24 bit-per-pixel format, or tile mode, or DECNano enabled.
3. Buffer height must be 4-pixel aligned for tile mode buffer.
4. For tile mode buffer, buffer stride must be 16 Byte aligned for non-24bit-per-pixel formats. So, 8 bits-per-pixel format buffer width must be 16-pixel aligned; 16 bits-per-pixel format buffer width must be 8-pixel aligned; 32 bit-per-pixel format buffer width must be 4 pixel aligned.
5. For tile mode buffer, buffer stride must be 12 Byte aligned for 24 bits-per-pixel formats, i.e., the buffer width must be 4-pixel aligned.
6. For PE clear function, the clear size must align to 48 Bytes for 24-bits-per-pixel formats.
7. For PE clear function with DECNano enabled, the clear size must align to 48 Bytes for 24 bits-per-pixel formats and align to 64 Bytes for 32 bits-per-pixel formats.
8. If source buffer tile mode is different from destination buffer tile mode, buffer starting address must be 64 Byte aligned, buffer stride must be 64 Byte aligned for non-24 bits-per-pixel formats, buffer stride must be 48 Byte aligned for 24 bits-per-pixel formats.

VGLite hardware requires the raster image width to be a multiple of 16 pixels for linear gradient and radial gradient operations. This requirement applies to all image formats. Therefore, the user needs to pad an arbitrary image width to a multiple of 16 pixels for VGLite linear gradient and radial gradient APIs.

5.4.4 vg_lite_buffer_layout_t Enumeration

Specifies the buffer data layout in memory.

Used in structure: vg_lite_buffer.

vg_lite_buffer_layout_t String Value	Description
VG_LITE_LINEAR	Linear (scanline) layout.
VG_LITE_TILED	Data is organized in 4x4 pixel tiles. Note: for this layout, the buffer start address and stride need to be 64 byte aligned.

5.4.5 vg_lite_compress_mode_t Enumeration

Specifies the DECNano comprssion mode. *(from March 2023)*

Used in structure: vg_lite_buffer_t.

vg_lite_compress_mode_t String Value	Description
VG_LITE_DEC_DISABLE	Disable compression.
VG_LITE_DEC_NON_SAMPLE	Compression ratio is 1.6 for argb8888, 2.0 for xrgb8888
VG_LITE_DEC_HSAMPLE	Compression ratio is 2.0 for argb8888, 2.6 for xrgb8888
VG_LITE_DEC_HV_SAMPLE	Compression ratio is 2.6 for argb8888, 4.0 for xrgb8888

5.4.6 vg_lite_gamma_conversion_t Enumeration

Specifies the gamma conversion mode. *(from Sept 2022)*

Used in function: vg_lite_set_gamma.

vg_lite_gamma_conversion_t String Value	Description
VG_LITE_GAMMA_NO_CONVERSION	Leave color as is.
VG_LITE_GAMMA_LINEAR	Convert from sRGB to linear.
VG_LITE_GAMMA_NON_LINEAR	Convert from linear to sRGB space.

5.4.7 vg_lite_index_endian_t Enumeration

Specifies the endian order parsing mode for index formats. *(from March 2023)*

Used in structure: vg_lite_buffer_t.

vg_lite_index_endian_t String Value	Description
VG_LITE_INDEX_ENDIAN_LITTLE_ENDIAN	Parse the index pixel from low to high, when using index1, the parsing order is bit0~bit7. when using index2, the parsing order is bit0:1,bit2:3,bit4:5.bit6:7. when using index4, the parsing order is bit0:3,bit4:7.
VG_LITE_INDEX_ENDIAN_BIG_ENDIAN	Parse the index pixel from low to high, when using index1, the parsing order is bit7~bit0. when using index2, the parsing order is bit7:6,bit5:4,bit3:2.bit1:0. when using index4, the parsing order is bit4:7,bit0:3.

5.4.8 vg_lite_image_mode_t Enumeration

Specifies how an image is rendered onto a buffer. *(prior to Sept 2022 name was vg_lite_buffer_image_mode_t)*

Used in structure: vg_lite_buffer_t.

vg_lite_image_mode_t String Value	Description
VG_LITE_ZERO	Image input is ignored
VG_LITE_NORMAL_IMAGE_MODE	Image drawn with blending mode, **VG_DRAW_IMAGE_NORMAL**
VG_LITE_MULTIPLY_IMAGE_MODE	Image is multiplied with paint color, **VG_DRAW_IMAGE_MULTIPLY**
VG_LITE_STENCIL_MODE	Image is used as a stencil for paint color, **VG_DRAW_IMAGE_STENCIL**
VG_LITE_NONE_IMAGE_MODE	Image input is ignored
VG_LITE_RECOLOR_MODE	LVGL recolor image mode, image pixels mixed with paint color.

5.4.9 vg_lite_map_flag_t Enumeration

Specifies whether mapping is for user memory or the DMA buffer. *(from March 2023)*

Used in function: vg_lite_map.

vg_lite_map_flag_t String Value	Description
VG_LITE_MAP_USER_MEMORY	Mapping is for user memory.
VG_LITE_MAP_DMABUF	Mapping is for the DMA buffer.

5.4.10 vg_lite_paint_type_t Enumeration

Specifies paint type. *(from May 2023)*

Used in structure: vg_lite_buffer_t.

vg_lite_paint_type_t String Value	Description
VG_LITE_PAINT_ZERO	
VG_LITE_PAINT_COLOR	Color
VG_LITE_PAINT_LINEAR_GRADIENT	Linear Gradient
VG_LITE_PAINT_RADIAL_GRADIENT	Radial Gradient
VG_LITE_PAINT_PATTERN	Pattern

5.4.11 vg_lite_transparency_t Enumeration

Specifies the transparency mode for a buffer. *(prior to Sept 2022 name was vg_lite_buffer_transparency_mode_t)*

Used in structure: vg_lite_buffer.

vg_lite_transparency_t String Value	Description
VG_LITE_IMAGE_OPAQUE	Opaque image: all image pixels are copied to the VG PE for rasterization
VG_LITE_IMAGE_TRANSPARENT	Transparent image: only the non-transparent image pixels are copied to the VG PE. Note: this mode is only valid when IMAGE_MODE (vg_lite_image_mode_t) is either VG_LITE_NORMAL_IMAGE_MODE or VG_LITE_MULTIPLY_IMAGE_MODE.

5.4.12 vg_lite_swizzle_t Enumeration

This enumeration specifies the swizzle for the UV components of YUV data.

Used in structure: vg_lite_yuvinfo.

vg_lite_swizzle_t String Value	Description
VG_LITE_SWIZZLE_UV	U in lower bits, V in upper bits
VG_LITE_SWIZZLE_VU	V in lower bits, U in upper bits

5.4.13 vg_lite_yuv2rgb_t Enumeration

This enumeration specifies the standard for conversion of YUV data to RGB data.

Used in structure: vg_lite_yuvinfo.

vg_lite_yuv2rgb_t String Value	Description
VG_LITE_YUV601	YUV Converting with ITC.BT-601 standard
VG_LITE_YUV709	YUV Converting with ITC.BT-709 standard

5.5 Pixel Buffer Structures

5.5.1 vg_lite_buffer_t Structure

This structure defines the buffer layout for a VGLite image or memory data.

Used in structures: vg_lite_linear_gradient_t, vg_lite_radial_gradient_t.
Used in init functions: vg_lite_allocate, vg_lite_free, vg_lite_upload_buffer, vg_lite_map, vg_lite_unmap.
Used in blit functions: vg_lite_blit, vg_lite_blit_rect, vg_lite_clear, vg_lite_create_masklayer, vg_lite_fill_masklayer, vg_lite_blend_masklayer, vg_lite_set_masklayer, vg_lite_render_masklayer, vg_lite_destroy_masklayer,
Used in draw functions: vg_lite_draw, vg_lite_draw_pattern, vg_lite_draw_grad, vg_lite_draw_radial_grad.

vg_lite_buffer_t Members	Type	Description
width	vg_lite_int32_t	Width of buffer in pixels
height	vg_lite_int32_t	Height of buffer in pixels
stride	vg_lite_int32_t	Stride in bytes
tiled	vg_lite_buffer_layout_t	Linear or tiled format for buffer enum
format	vg_lite_buffer_format_t	color format enum
handle	vg_lite_pointer	memory handle
memory	vg_lite_pointer	pointer to the start address of the memory
address	vg_lite_uint32_t	GPU address
yuv	vg_lite_yuvinfo_t	YUV format info struct
image_mode	vg_lite_image_mode_t	Blit image mode enum
transparency_mode	vg_lite_transparency_t	Image transparency mode enum
fc_buffer[3]	vg_lite_fc_buffer_t	Three (3) fast clear buffers, reserved YUV format *(from March 2023)*
compress_mode	vg_lite_compress_mode	Compression mode *(from March 2023)*
index_endian	vg_lite_index_endian_t	Big/Little Endian setting for index formats *(from March 2023)*
paintType	vg_lite_paint_type_t	Paint type enum *(from May 2023)*
fc_enable	vg_lite_int8_t	Enable Image fast clear *(moved from Aug 2023)*
scissor_layer	vg_lite_int8_t	Get paintcolor from different paint type *(from Aug 2023)*
premulitplied	vg_lite_int8_t	The RGB pixel values are alpha-premultipled *(from Aug 2023)*

5.5.2 vg_lite_fc_buffer_t Structure

This structure defines the organization of a fast clear buffer. *(from March 2023)*

Used in structure: vg_lite_buffer_t.

vg_lite_fc_buffer_t Members	Type	Description
width	vg_lite_int32_t	Width of buffer in pixels
height	vg_lite_int32_t	Height of buffer in pixels
stride	vg_lite_int32_t	Stride in bytes
handle	vg_lite_pointer	memory handle as allocated by the VGLite kernel
memory	vg_lite_pointer	logical pointer to the start address of the memory for the CPU
address	vg_lite_uint32_t	address to the buffer's memory for the GPU hardware
color	vg_lite_uint32_t	The fast clear color value

5.5.3 vg_lite_yuvinfo_t Structure

This structure defines the organization of VGLite YUV data.

Used in structure: vg_lite_buffer_t.

vg_lite_yuvinfo_t Members	Type	Description
swizzle	vg_lite_swizzle_t	UV swizzle enum
yuv2rgb	vg_lite_yuv2rgb_t	YUV conversion standard enum
uv_planar	vg_lite_uint32_t	UV (U) planar address for GPU, generated by driver
v_planar	vg_lite_uint32_t	V planar address for GPU, generated by driver
alpha_planar	vg_lite_uint32_t	Alpha planar address for GPU, generated by driver
uv_stride	vg_lite_uint32_t	UV (U) stride in bytes
v_stride	vg_lite_uint32_t	V planar stride in bytes
alpha_stride	vg_lite_uint32_t	Alpha stride in bytes
uv_height	vg_lite_uint32_t	UV (U) height in pixels
v_height	vg_lite_uint32_t	V stride in bytes
uv_memory	vg_lite_pointer	Logical pointer to the UV (U) planar memory
v_memory	vg_lite_pointer	Logical pointer to the V planar memory
uv_handle	vg_lite_pointer	Memory handle of the UV (U) planar, generated by driver
v_handle	vg_lite_pointer	Memory handle of the V planar, generated by driver

5.6 Pixel Buffer Functions

vg_lite_allocate

Description:
This function is used to allocate a buffer before using it in either blit or draw functions.

In order for the hardware to access some memory, like a source image or a target buffer, it needs to be allocated first. The supplied vg_lite_buffer_t structure needs to be initialized with the size (width and height) and format of the requested buffer. If the stride is set to zero, this function will fill it in. The only input parameter to this function is the pointer to the buffer structure. If the structure has all the information needed, appropriate memory will be allocated for the buffer.

This function will call the kernel to actually allocate the memory. The memory handle, logical address, and hardware addresses in the vg_lite_buffer_t structure will be filled in by the kernel.

Alignment Note: Though Vivante Vector Graphics hardware has an alignment requirement of 64 bytes, the VGLite Driver sets alignment to 128 bytes for render target buffer to conform to the alignment requirement of Vivante Display Controller. For source image buffer alignment requirement, see Alignment Notes Table 3 following the vg_lite_buffer_format_t value descriptions.

Note: The DEC compressed image buffer allocation requires the buffer stride to set as the uncompressed image buffer would be in terms of size **(stride = width * bpp).** But the allocated buffer size for DEC compressed image **is in fact lower (total_size = width * height * ratio, where ratio is based on the vg_lite_compress_mode_t description)**.

Syntax:

```
vg_lite_error_t vg_lite_allocate (
    vg_lite_buffer_t        *buffer
);
```

Parameters:

***buffer**	Pointer to the buffer that holds the size and format of the buffer being allocated. Either the memory or address field needs to be set to a non-zero value to map either a logical or physical address into hardware accessible memory.

Returns:
VG_LITE_SUCCESS if the allocating contiguous buffer is allocated successfully.
VG_LITE_OUT_OF_RESOURCES if there is insufficient memory in the host OS heap for the buffer
VG_LITE_OUT_OF_MEMORY if allocation of a contiguous buffer failed.

vg_lite_free

Description:
This function is used to deallocate the buffer that was previously allocated. This will free up the memory for that buffer.

Syntax:

```
vg_lite_error_t vg_lite_free (
    vg_lite_buffer_t            *buffer
);
```

Parameters:

***buffer**	Pointer to a buffer structure that was filled in by vg_lite_allocate.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_upload_buffer

Description:
The function uploads the pixel data to a GPU memory buffer object. Note that the format of the data (pixel) to be uploaded must be the same as described in the buffer object. The input data memory buffer should contain enough data to be uploaded to the GPU buffer pointed by the input parameter "buffer". *(replaces deprecated vg_lite_buffer_upload from Sept 2022)*

Note: Vivante Vector Graphics IP only uses data[0] and stride[0] as it does not support planar YUV formats.

Syntax:

```
vg_lite_error_t vg_lite_upload_buffer (
    vg_lite_buffer_t            *buffer,
    vg_lite_uint8_t             *data[3],
    vg_lite_uint32_t             stride[3]
);
```

Parameters:

***buffer**	Pointer to a buffer structure that was filled in by vg_lite_allocate.
***data[3]**	Pointer to pixel data. For YUV format, there may be up to 3 pointers.
stride[3]	Stride for the pixel data.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_map

Description:
This function is used to map the logical and physical memory appropriately for a particular buffer. For some operating systems, it can be used to get proper translation to physical or logical address of the buffer needed by the GPU.

If you want the use a frame buffer directly as a target buffer, you need to wrap a vg_lite_buffer_t structure around it and call the kernel to map the supplied logical or physical address into hardware accessible memory. For example, if you know the logical address of the frame buffer, set the "memory" field of the vg_lite_buffer_t structure with that address. If you know the physical address, set the "memory" field to NULL and set the "address" field with the physical address. The **vg_lite_map** API will fill the unknown logical or physical addresses in the vg_lite_buffer_t structure with valid addresses.

vg_lite_buffer_t structure can describe single plane pixel data, as well as multi-plane pixel data, such as YUV formats (2-pixel planes or 3-pixel planes). When a multi-plane YUV buffer is mapped, "memory" field is the logical address of the y-plane data buffer, "yuv.uv_memory" field is the logical address of UV-plane data buffer, "yuv.v_memory" is the logical address of V-plane data buffer.

Syntax:

```
vg_lite_error_t vg_lite_map (
    vg_lite_buffer_t          *buffer,
    vg_lite_map_flag_t        flag,
    int32_t                   fd
);
```

Parameters:

***buffer**	Pointer to the buffer structure to be mapped.
flag	Enum vg_lite_map_flag_t value which specifies whether mapping is for user memory or DMA buffer. *(from March 2023)*
fd	File descriptor for dma_buf if flag is VG_LITE_MAP_DMABUF. Otherwise, this parameter is ignored. *(from March 2023)*

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_unmap

Description:
This function unmaps the buffer and frees any memory resources allocated by a previous call to vg_lite_map.

Syntax:

```
vg_lite_error_t vg_lite_unmap (
    vg_lite_buffer_t            *buffer
);
```

Parameters:

***buffer**	Pointer to a buffer structure that was filled in by vg_lite_map.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_flush_mapped_buffer

Description:
This function flushes the CPU cache for the mapped buffer to make sure the buffer contents are written to GPU memory.

Syntax:

```
vg_lite_error_t vg_lite_flush_mapped_buffer (
    vg_lite_buffer_t           *buffer
);
```

Parameters:

***buffer**	Pointer to a buffer structure that was filled in by vg_lite_map.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_set_CLUT

Description:
This function sets the Color Lookup Table (CLUT) in context state for index color image. Once the CLUT is set (Not NULL), the image pixel color for index format image rendering is obtained from the Color Lookup Table (CLUT) according to the pixel's color index value.

Note: Available only for IP with Indexed color support.

Syntax:

```
vg_lite_error_t vg_lite_set_CLUT (
    vg_lite_uint32_t            count,
    vg_lite_uint32_t            *colors
);
```

Parameters:

count	This is the count of the colors in the color look up table. For INDEX_1, there can be up to 2 colors in the table; For INDEX_2, there can be up to 4 colors in the table; For INDEX_4, there can be up to 16 colors in the table; For INDEX_8, there can be up to 256 colors in the table.
***colors**	The Color Lookup Table (CLUT) pointed by "colors" will be stored in the context and programmed to the command buffer when needed. The CLUT will not take effect until the command buffer is submitted to HW. The color is in ARGB format with A located in the upper bits. Note: The VGLite driver does not validate the CLUT contents from application.

Returns:
VG_LITE_SUCCESS since no checking is done.

vg_lite_enable_dither

Description:
This function is used to enable the dither function. Dither is turned off by default.
Applications can use VGLite API vg_lite_query_feature(gcFEATURE_BIT_VG_DITHER) to determine HW support for dither.

Syntax:

```
vg_lite_error_t vg_lite_enable_dither (
);
```

Parameters: None

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_disable_dither

Description:
This function is used to disable the dither function. Dither is turned off by default.

Syntax:

```
vg_lite_error_t vg_lite_disable_dither (
);
```

Parameters: None

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_set_gamma

Description:
This function sets a gamma value.
Application can use VGLite API vg_lite_query_feature(gcFEATURE_BIT_VG_GAMMA) to determine HW support for gamma.

Syntax:

```
vg_lite_error_t vg_lite_set_gamma (
    vg_lite_gamma_conversion_t    gamma_value
);
```

Parameters:

gamma_value	Sets a gamma value. See enum vg_lite_gamma_conversion_t.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

6 Matrices

This part of the API provides matrix controls.

Note: All the transformations in the driver/API are actually the final plane/surface coordinate system. There is no transformation of different coordinate systems with VGLite.

6.1 Matrix Control Float Parameter Type

Name	Typedef	Value
vg_lite_float_t	float	A single precision floating point number

vg_lite_pixel_matrix_t[20]	vg_lite_float_t	Pixel transform matrix m[20] which transforms each pixel as follows:
		`|a'|  |m0  m1  m2  m3  m4 | |a|`
		`|r'|  |m5  m6  m7  m8  m9 | |r|`
		`|g'| = |m10 m11 m12 m13 m14|.|g|`
		`|b'|  |m15 m16 m17 m18 m19| |b|`
		`|1 |  |0   0   0   0   1  | |1|`

6.2 Matrix Control Structures

6.2.1 vg_lite_matrix_t Structure

This structure defines a 3x3 float matrix.

Used in structures: vg_lite_linear_gradient_t, vg_lite_radial_gradient_t.
Used in blit functions: vg_lite_blit, vg_lite_blit_rect.
Used in function: vg_lite_render_masklayer.
Used in draw functions: vg_lite_draw, vg_lite_draw_grad, vg_lite_draw_radial_grad, vg_lite_draw_pattern, vg_lite_identity, vg_lite_scale, vg_lite_translate.

vg_lite_matrix_t Members	Type	Description
m[3][3]	vg_lite_float_t	3x3 matrix, in [row] [column] order

6.2.2 vg_lite_pixel_channel_enable_t Structure

This structure provides enable disable flags for hardware pixel channels A,R,G,B.

Used in function: vg_lite_set_pixel_matrix_t.

vg_lite_pixel_channel_enable_t Members	Type	Description
enable_a	vg_lite_uint8_t	Enable A channel
enable_b	vg_lite_uint8_t	Enable B channel
enable_g	vg_lite_uint8_t	Enable G channel
enable_r	vg_lite_uint8_t	Enable R channel

6.3 Matrix Control Functions

vg_lite_identity

Description:
This function loads an identity matrix into a matrix variable.

Syntax:

```
vg_lite_error_t vg_lite_identity (
    vg_lite_matrix_t            *matrix,
);
```

Parameters:

***matrix**	Pointer to the vg_lite_matrix_t structure that will be loaded with an identity matrix

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_set_pixel_matrix

Description:
This function sets up a pixel transform matrix m[20] which transforms each pixel as follows:

```
|a'|   |m0  m1  m2  m3  m4 | |a|
|r'|   |m5  m6  m7  m8  m9 | |r|
|g'| = |m10 m11 m12 m13 m14|.|g|
|b'|   |m15 m16 m17 m18 m19| |b|
|1 |   |0   0   0   0   1  | |1|
```

The pixel transform for the A, R, G, B channels can be enabled/disabled individually with the channel parameter.

Applications can use VGLite API vg_lite_query_feature (gcFEATURE_BIT_VG_PIXEL_MATRIX) to determine HW support for gaussian blur.

Syntax:

```
vg_lite_error_t vg_lite_set_pixel_matrix (
    vg_lite_pixel_matrix_t          matrix,
    vg_lite_pixel_channel_enable_t  *channel
);
```

Parameters:

matrix	Specifies the vg_lite_pixel_matrix_t pixel transform matrix that will be loaded
***channel**	Pointer to the vg_lite_pixel_channel_enable_t structure used to enable/disable individual channels.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_rotate

Description:
This function rotates a matrix a specified number of degrees.

Syntax:

```
vg_lite_error_t vg_lite_rotate (
    vg_lite_float_t             degrees,
    vg_lite_matrix_t            *matrix
);
```

Parameters:

degrees	Number of degrees to rotate the matrix. Positive numbers rotate clockwise. The coordinates for the transformation are given in the surface coordinate system (top-to-bottom orientation). Rotations with positive angles are in the clockwise direction
***matrix**	Pointer to the vg_lite_matrix_t structure that will be rotated.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_scale

Description:
This function scales a matrix in both horizontal and vertical directions.

Syntax:

```
vg_lite_error_t vg_lite_scale (
    vg_lite_float_t             scale_x,
    vg_lite_float_t             scale_y,
    vg_lite_matrix_t            *matrix
);
```

Parameters:

scale_x	Horizontal scale.
scale_y	Vertical scale.
***matrix**	Pointer to the vg_lite_matrix_t structure that will be scaled.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_translate

Description:
This function translates a matrix to a new location.

Syntax:

```
vg_lite_error_t vg_lite_translate (
    vg_lite_float_t             x,
    vg_lite_float_t             y,
    vg_lite_matrix_t            *matrix
);
```

Parameters:

x	X location of the transformation.
y	Y location of the transformation.
***matrix**	Pointer to the vg_lite_matrix_t structure that will be translated.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

7 Blits for Compositing and Blending

This part of the API performs the hardware accelerated blit operations.

Compositing rules describe how two areas are combined to form a single area. Blending rules describes how combining the colors of the overlapping areas are combined. VGLite supports two blending operations and a subset of the Porter-Duff operations [PD84]. The Porter-Duff operators assume the pixels have the alpha associated (pre-multiplied), i.e., pixels are premultiplied prior to the blending operation. Note that GC555, GC355 and some GCNanoUltraV hardware supports alpha premultiply for RGB image, but GCNanoLiteV does not.

The source image is copied to the destination window with a specified matrix that can include translation, rotation, scaling, and perspective correction.

- The blit function can be used with or without the blend mode.
- The blit function can be used with or without specifying any color value.
- The blit function can be used for color conversion with an identity matrix and appropriate formats specified for the source and the destination buffers. In this case do not specify blend mode and color value.

7.1 BLIT Enumerations

7.1.1 vg_lite_blend_t Enumeration

This enumeration defines the blending modes supported by some VGLite API functions. S and D represent source and destination non-pre-multiplied RGB color channels. Sa and Da represent the source and destination alpha channels. SP and DP represent source and destination alpha-pre-multiplied RGB color channels (SP = S*Sa, DP = D*Da).

Note: VG_LITE_BLEND_*_LVGL modes are supported on all VG cores. On VG cores that do not support gcFEATURE_BIT_VG_LVGL_SUPPORT, the LVGL blend modes are supported by a combination of software and hardware operations. OPENVG_BLEND_* modes can only be supported on GC355 and GC555 cores.

Used in blit functions: vg_lite_blit, vg_lite_blit2, vg_lite_blit_rect.
Used in draw functions: vg_lite_draw, vg_lite_draw_grad, vg_lite_draw_radial_grad, vg_lite_draw_pattern.

vg_lite_blend_t String Values	Description
VG_LITE_BLEND_NONE	**S, no blending** Non-premultiplied
VG_LITE_BLEND_SRC_OVER	**S + D * (1 - Sa)** Non-premultiplied
VG_LITE_BLEND_DST_OVER	**S * (1 – Da) + D** Non-premultiplied
VG_LITE_BLEND_SRC_IN	**S * Da** Non-premultiplied
VG_LITE_BLEND_DST_IN	**D * Sa** Non-premultiplied
VG_LITE_BLEND_MULTIPLY	**S * (1 - Da) + D * (1 - Sa) + S * D** Non-premultiplied
VG_LITE_BLEND_SCREEN	**S + D - S * D** Non-premultiplied

vg_lite_blend_t String Values	Description
VG_LITE_BLEND_DARKEN	**min(SRC_OVER, DST_OVER)** Non-premultiplied
VG_LITE_BLEND_LIGHTEN	**max(SRC_OVER, DST_OVER)** Non-premultiplied
VG_LITE_BLEND_ADDITIVE	**S + D** Non-premultiplied
VG_LITE_BLEND_SUBTRACT	**D * (1 - Sa)** Non-premultiplied
VG_LITE_BLEND_NORMAL_LVGL	**S * Sa + D * (1 - Sa)** Non-premultiplied *(from March 2023)*
VG_LITE_BLEND_ADDITIVE_LVGL	**(S + D) * Sa + D * (1 - Sa)** Non-premultiplied *(from March 2023)*
VG_LITE_BLEND_SUBTRACT_LVGL	**(S - D) * Sa + D * (1 - Sa)** Non-premultiplied *(from March 2023)*
VG_LITE_BLEND_MULTIPLY_LVGL	**(S * D) * Sa + D * (1 - Sa)** Non-premultiplied *(from March 2023)*
OpenVG Porter-Duff Blend String Values	*(from Aug 2023)*
OPENVG_BLEND_NONE	**SP, no blending** Premultiplied
OPENVG_BLEND_SRC_OVER	**(SP + DP * (1 - Sa)) / (Sa + Da * (1 - Sa))** Premultiplied
OPENVG_BLEND_DST_OVER	**(SP * (1 - Da) + DP) / (Sa * (1 - Da) + Da)** Premultiplied
OPENVG_BLEND_SRC_IN	**(SP * Da) / (Sa * Da)** Premultiplied
OPENVG_BLEND_DST_IN	**(DP * Sa) / (Sa * Da)** Premultiplied
OPENVG_BLEND_MULTIPLY	**(SP*DP + SP*(1 - Da) + DP*(1 - Sa)) / (Sa + Da*(1 - Sa))** Premultiplied
OPENVG_BLEND_SCREEN	**(SP + DP - (SP*DP)) / (Sa + Da*(1 - Sa))** Premultiplied
OPENVG_BLEND_DARKEN	**min(SRC_OVER, DST_OVER)** Premultiplied
OPENVG_BLEND_LIGHTEN	**max(SRC_OVER, DST_OVER)** Premultiplied
OPENVG_BLEND_ADDITIVE	**(SP + DP) / (Sa + Da)** Premultiplied

7.1.2 vg_lite_color_t Parameter

The common parameter vg_lite_color_t is described in Section 1.4. LINK to Common Parameter Types.

7.1.3 vg_lite_color_transform_t Structure

Specifies the pixel color_transform values for scale and bias.

Used in functions: vg_lite_set_color_transform.

vg_lite_color_transform_t Members	Type	Description
a_scale	vg_lite_float_t	Scale value for alpha.
a_bias	vg_lite_float_t	Bias value for alpha.
r_scale	vg_lite_float_t	Scale value for red.
r_bias	vg_lite_float_t	Bias value for red.
g_scale	vg_lite_float_t	Scale value for green.
g_bias	vg_lite_float_t	Bias value for green.
b_scale	vg_lite_float_t	Scale value for blue.
b_bias	vg_lite_float_t	Bias value for blue.

7.1.4 vg_lite_filter_t Enumeration

Specifies the sample filtering mode in VGLite blit and draw APIs.

Used in blit functions: vg_lite_blit, vg_lite_blit_rect,
Used in draw functions: vg_lite_draw_radial_grad, vg_lite_draw_pattern.

vg_lite_filter_t String Values	Description
VG_LITE_FILTER_POINT	Fetch only the nearest image pixel.
VG_LITE_FILTER_LINEAR	Use linear interpolation along horizontal line.
VG_LITE_FILTER_BI_LINEAR	Use a 2x2 box around the image pixel and perform an interpolation.
VG_LITE_FILTER_GAUSSIAN	Perform 3x3 gaussian blur with the convolution for image pixel. *(from March 2023)*

7.1.5 vg_lite_global_alpha_t Enumeration

Specifies the global alpha mode in VGLite blit APIs.

Used in blit function: vg_lite_dest_global_alpha.

vg_lite_global_alpha_t String Values	Description
VG_LITE_NORMAL	= 0: Use original src/dst alpha value.
VG_LITE_GLOBAL	Use global src/dst alpha value to replace original src/dst alpha value.
VG_LITE_SCALED	Multiply global src/dst alpha value and original src/dst alpha value.

7.1.6 vg_lite_mask_operation_t Enumeration

Specifies the mask operation mode in VGLite blit APIs.

Used in functions: vg_lite_blend_masklayer, vg_lite_render_masklayer.

vg_lite_mask_operation_t String Values	Description
VG_LITE_CLEAR_MASK	This operation sets all mask values in the region of interest to 0, ignoring the new mask layer.
VG_LITE_FILL_MASK	This operation sets all mask values in the region of interest to 1, ignoring the new mask layer.
VG_LITE_SET_MASK	This operation copies values in the region of interest from the new mask layer, overwriting the previous mask values.
VG_LITE_UNION_MASK	This operation replaces the previous mask in the region of interest by its union with the new mask layer. The resulting values are always greater than or equal to their previous value.
VG_LITE_INTERSECT_MASK	This operation replaces the previous mask in the region of interest by its intersection with the new mask layer. The resulting mask values are always less than or equal to their previous value.
VG_LITE_SUBTRACT_MASK	This operation subtracts the new mask from the previous mask and replaces the previous mask in the region of interest by the resulting mask. The resulting values are always less than or equal to their previous value.

7.1.7 vg_lite_orientation_t Enumeration

Specifies the mirror orientation in VGLite blit APIs.

Used in functions: vg_lite_set_mirror.

vg_lite_orientation_t String Values	Description
VG_LITE_ORIENTATION_TOP_BOTTOM	Target output orientation is from top to bottom (default).
VG_LITE_ORIENTATION_BOTTOM_TOP	Target output orientation is from bottom to top.

7.1.8 vg_lite_param_type_t Enumeration

Specifies the parameter type in **`vg_lite_get_parameter`** API.

vg_lite_param_type_t String Values	Description
VG_LITE_GPU_IDLE_STATE	GPU idle state (TRUE or FALSE). Count must be 1.
VG_LITE_SCISSOR_RECT	Current scissor rectangle settings. Count must be 4n.
VG_LITE_HARDWARE_RUNNING_TIME	GPU hardware running time.
VG_LITE_SRC_BUF_ALIGNED_CHECK	Source image buffer alignment status.
VG_LITE_DST_BUF_ALIGNED_CHECK	Destination image buffer alignment status.

7.2 BLIT Structures

7.2.1 vg_lite_buffer_t Structure

Defined under Pixel Buffer Structures. LINK to vg_lite_buffer_t structure.

7.2.2 vg_lite_color_key_t Structure

A “color key” have two sections, where each section contains R, G, B channels which are noted as high_rgb and low_rgb respectively. *(from April 2022)*

When the enable value is true, the color key specified is effective and the alpha value is used to replace the alpha channel of the destination pixel when its RGB channels are in range [low_rgb, high_rgb]. After the color key is used in the current frame, if the color key is not needed for the next frame, it should be disabled before the next frame.

Used in structure: vg_lite_color_key4_t.

vg_lite_color_key_t Members	Type	Description
enable	vg_lite_uint8_t	When set (true), this color key is enabled.
low_r	vg_lite_uint8_t	The R channel of low_rgb.
low_g	vg_lite_uint8_t	The G channel of low_rgb.
low_b	vg_lite_uint8_t	The B channel of low_rgb.
alpha	vg_lite_uint8_t	The alpha channel to replace the destination pixel alpha channel.
high_r	vg_lite_uint8_t	The R channel of high_rgb.
high_g	vg_lite_uint8_t	The G channel of high_rgb.
high_b	vg_lite_uint8_t	The B channel of high_rgb.

7.2.3 vg_lite_color_key4_t Structure

The priority order is: color_key_0 > color_key_1 > color_key_2 > color_key_3. *(from April 2022)*

Used in blit function: vg_lite_set_color_key

vg_lite_color_key4_t Members	Type	Description
color_key_0	vg_lite_color_key_t	high_rgb_0, low_rgb_0, alpha_0, enable_0
color_key_1	vg_lite_color_key_t	high_rgb_1, low_rgb_1, alpha_1, enable_1
color_key_2	vg_lite_color_key_t	high_rgb_2, low_rgb_2, alpha_2, enable_2
color_key_3	vg_lite_color_key_t	high_rgb_3, low_rgb_3, alpha_3, enable_3

7.2.4 vg_lite_matrix_t Structure

Defined under Matrix Control Structures LINK to vg_lite_matrix_t structure.

7.2.5 vg_lite_path_t Structure

Defined under Vector Path Structures. LINK to vg_lite_path_t structure.

7.2.6 vg_lite_rectangle_t Structure

This structure defines the organization of a rectangle of VGLite data.

Used in blit function: vg_lite_clear.

vg_lite_rectangle_t Members	Type	Description
x	vg_lite_int32_t	X Origin of rectangle, left coordinate in pixels
y	vg_lite_int32_t	Y Origin of rectangle, top coordinate in pixels
width	vg_lite_int32_t	X Width of rectangle in pixels
height	vg_lite_int32_t	Y Height of rectangle in pixels

7.2.7 vg_lite_point_t Structure

This structure defines a 2D Point. *(from March 2021)*

Used in structure: vg_lite_point4_t.

vg_lite_point_t Members	Type	Description
x	vg_lite_int32_t	X value of coordinate
y	vg_lite_int32_t	Y value of coordinate

7.2.8 vg_lite_point4_t Structure

This structure defines four 2D points that form a polygon. The points are defined by structure vg_lite_point_t. *(from March 2021)*

vg_lite_point4_t Members	Type	Description
vg_lite_point[4]	vg_lite_int32_t each	a set of four points

7.2.9 vg_lite_float_point_t Structure

This structure defines a 2D float point. *(from March 2024)*

Used in structure: vg_lite_float_point4_t.

vg_lite_float_point_t Members	Type	Description
x	vg_lite_float_t	X value of coordinate
y	vg_lite_float_t	Y value of coordinate

7.2.10 vg_lite_float_point4_t Structure

This structure defines four 2D float points that form a polygon. The points are defined by structure vg_lite_float_point_t. *(from March 2024)*

Used in blit function: vg_lite_get_transform_matrix.

vg_lite_float_point4_t Members	Type	Description
vg_lite_float_point[4]	vg_lite_float_t each	a set of four points

7.3 BLIT Functions

vg_lite_blit

Description:
This is the blit function. The blit operation is performed using a source and a destination buffer. The source and destination buffer structures are defined using the vg_lite_buffer_t structure. Blit copies a source image to the destination window with a specified matrix that can include translation, rotation, scaling, and perspective correction. Note that vg_lite_buffer_t does not support coverage sample anti-aliasing so the destination buffer edge may not be smooth especially with a rotation matrix. VGLite path rendering can be used to achieve high quality coverage sample anti-aliasing (16X, 8X, 4X) rendering effect.

Notes:

- The blit function can be used with or without the blend function (vg_lite_blend_t).
- The blit function can be used with or without specifying any color value (vg_lite_color_t).
- The blit function can be used for color conversion with an identity matrix and appropriate formats specified for the source and the destination buffers. In this case do not specify blend mode and color value.

Syntax:

```
vg_lite_error_t vg_lite_blit (
    vg_lite_buffer_t            *target,
    vg_lite_buffer_t            *source,
    vg_lite_matrix_t            *matrix,
    vg_lite_blend_t             blend,
    vg_lite_color_t             color,
    vg_lite_filter_t            filter
);
```

Parameters:

***target**	Points to the vg_lite_buffer_t structure which defines the destination buffer. See Image Source Alignment Requirement for valid destination color formats for the blit functions.
***source**	Points to the vg_lite_buffer_t structure for the source buffer. All color formats available in the vg_lite_buffer_format_t enum are valid source formats for the blit function.
***matrix**	Points to a vg_lite_matrix_t structure that defines the 3x3 transformation matrix of source pixels into the target. If matrix is NULL, an identity matrix is assumed, meaning the source will be copied directly on the target at (0,0) location.

blend Specifies one of the enum vg_lite_blend_t values for hardware supported blend modes to be applied to each image pixel. If no blending is required, set this value to VG_LITE_BLEND_NONE (0).
Note: If the “matrix” parameter is specified with rotation or perspective, and the “blend" parameter is specified as VG_LITE_BLEND_NONE, VG_LITE_BLEND_SRC_IN, or VG_LITE_BLEND_DST_IN, the VGLite driver will overwrite the application’s setting for the BLIT operation as follows:

- If gcFEATURE_BIT_VG_BORDER_CULLING (vg_lite_feature_t) is supported, the transparency mode will always be set to TRANSPARENT.
- If gcFEATURE_BIT_VG_BORDER_CULLING (vg_lite_feature_t) is not supported, the blend mode will always be set to VG_LITE_BLEND_SRC_OVER.

This is due to some limitations in the VGLite hardware.

color If non-zero, this color value is used as a mix color. The mix color gets multiplied with each source pixel before blending happens. If you don't need a mix color, set the color parameter to 0.
NOTE: this parameter has no effect if the source vg_lite_buffer_t structure member image_mode is set to VG_LITE_ZERO or VG_LITE_NORMAL_IMAGE_MODE.

filter Specifies the filter type. All formats available in the vg_lite_filter_t enum are valid formats for this function. A value of zero (0) indicates VG_LITE_FILTER_POINT.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_blit2

Description:

This is the blit function for use with two sources. The blit2 operation is performed using two source buffers and one destination buffer. The source and destination buffer structures are defined using the vg_lite_buffer_t structure. Source0 and Source1 are first blended according to the blend mode with a specific transformation matrix for each image. Source1 is used as the source while Source0 is used as dest and is directly output to the render target buffer.

The specified matrices can include translation, rotation, scaling, and perspective correction. Note that vg_lite_buffer_t does not support coverage sample anti-aliasing so the destination buffer edge may not be smooth especially with a rotation matrix. VGLite path rendering can be used to achieve high quality coverage sample anti-aliasing (16X, 8X, 4X) rendering effect.

Application can use VGLite API vg_lite_query_feature(gcFEATURE_BIT_VG_DOUBLE_IMAGE) to determine HW support for double image.

Notes:

- The vg_lite_blit function can be used for color conversion for Source0 or Source1 before merging sources with vg_lite_blit2.

Syntax:

```
vg_lite_error_t vg_lite_blit2 (
    vg_lite_buffer_t              *target,
    vg_lite_buffer_t              *source0,
    vg_lite_buffer_t              *source1,
    vg_lite_matrix_t              *matrix0,
    vg_lite_matrix_t              *matrix1,
    vg_lite_blend_t               blend,
    vg_lite_filter_t              filter
);
```

Parameters:

***target**	Points to the vg_lite_buffer_t structure which defines the destination buffer. See Image Source Alignment Requirement for valid destination color formats for the blit functions.
***source0, *source1**	Points to the vg_lite_buffer_t structure for the source0 and source1 buffers. All color formats available in the vg_lite_buffer_format_t enum are valid source formats for the blit functions.
***matrix0, *matrix1**	Points to a vg_lite_matrix_t structure that defines the 3x3 transformation matrix0 for the source0 pixels and matrix1 for the source1 pixels. If matrix0 and matrix1 are both NULL, the identity matrix is assumed, meaning the blending result of Source0 and Source1 is copied directly on the target at location(0,0).

blend	Specifies one of the enum vg_lite_blend_t values for hardware supported blend modes to be applied to each image pixel. If no blending is required, set this value to VG_LITE_BLEND_NONE (0). Note: If the “matrix” parameter is specified with rotation or perspective, and the “blend" parameter is specified as VG_LITE_BLEND_NONE, VG_LITE_BLEND_SRC_IN, or VG_LITE_BLEND_DST_IN, the VGLite driver will overwrite the application’s setting for the BLIT operation as follows: • If gcFEATURE_BIT_VG_BORDER_CULLING (vg_lite_feature_t) is supported, the transparency mode will always be set to TRANSPARENT. • If gcFEATURE_BIT_VG_BORDER_CULLING (vg_lite_feature_t) is not supported, the blend mode will always be set to VG_LITE_BLEND_SRC_OVER. This is due to some limitations in the VGLite hardware.
filter	Specifies the filter type. All formats available in the vg_lite_filter_t enum are valid formats for this function. A value of zero (0) indicates VG_LITE_FILTER_POINT.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_blit_rect

Description:
This is the blit rectangle function. The blit operation is performed using a source and a destination buffer. The source and destination buffer structures are defined using the vg_lite_buffer_t structure. Blit copies a source image to the destination window with a specified matrix that can include translation, rotation, scaling, and perspective correction. Note that vg_lite_buffer_t does not support coverage sample anti-aliasing so the destination buffer edge may not be smooth especially with a rotation matrix. VGLite path rendering can be used to achieve high quality coverage sample anti-aliasing (16X, 8X, 4X) rendering effect.

Notes:

- The blit_rect function can be used with or without the blend function (vg_lite_blend_t).
- The blit_rect function can be used with or without specifying any color value (vg_lite_color_t) .
- The blit_rect function can be used for color conversion with an identity matrix and appropriate formats specified for the source and destination buffers. In this case do not specify blend mode and color value.
- *The vg_lite_blit_rect rectangle start origin point is always (0,0) for hardware versions prior to GCNanoLiteV 1311p which do not support a non-zero rectangle origin.*

Syntax:

```
vg_lite_error_t vg_lite_blit_rect (
    vg_lite_buffer_t            *target,
    vg_lite_buffer_t            *source,
    vg_lite_rectangle_t         *rect,
    vg_lite_matrix_t            *matrix,
    vg_lite_blend_t             blend,
    vg_lite_color_t             color,
    vg_lite_filter_t            filter
);
```

Parameters:

***target**	Points to the vg_lite_buffer_t structure which defines the destination buffer. See Source Image Alignment Requirement for valid destination color formats for the blit_rect functions.
***source**	Points to the vg_lite_buffer_t structure for the source buffer. All color formats available in the vg_lite_buffer_format_t enum are valid source formats for the blit_rect function.
***rect**	Specifies the rectangle area (x, y, width, height) of the source image to blit. Note: Non-zero source origins are supported.
***matrix**	Points to a vg_lite_matrix_t structure that defines the 3x3 transformation matrix of source pixels into the target. If matrix is NULL, an identity matrix is assumed, meaning the source will be copied directly on the target at 0,0 location.

blend Specifies one of the enum vg_lite_blend_t values for hardware supported blend modes to be applied to each image pixel. If no blending is required, set this value to VG_LITE_BLEND_NONE (0).
Note: If the “matrix” parameter is specified with rotation or perspective, and the “blend" parameter is specified as VG_LITE_BLEND_NONE, VG_LITE_BLEND_SRC_IN, or VG_LITE_BLEND_DST_IN, the VGLite driver will overwrite the application’s setting for the BLIT operation as follows:

- If gcFEATURE_BIT_VG_BORDER_CULLING (vg_lite_feature_t) is supported, the transparency mode will always be set to TRANSPARENT.
- If gcFEATURE_BIT_VG_BORDER_CULLING (vg_lite_feature_t) is not supported, the blend mode will always be set to VG_LITE_BLEND_SRC_OVER.

This is due to some limitations in the VGLite hardware.

color If non-zero, this color value is used as a mix color. The mix color gets multiplied with each source pixel before blending happens. If you don't need a mix color, set the color parameter to 0.
NOTE: this parameter has no effect if the source vg_lite_buffer_t structure member image_mode is set to VG_LITE_ZERO or VG_LITE_NORMAL_IMAGE_MODE.

filter Specifies the filter type. All formats available in the vg_lite_filter_t enum are valid formats for this function. A value of zero (0) indicates VG_LITE_FILTER_POINT.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_copy_image

Description:
This API copied a pixel rectangle with dimension (width, height) from source buffer to destination buffer. The source image pixel (sx + *i*, sy + *j*) is copied to the destination image pixel (dx + *i*, dy + *j*), for $0 \le i <$ width and $0 \le j <$ height. Pixels whose source or destination lie outside of the bounds of the respective image are ignored. Pixel format conversion is applied as needed.

No pre-multiply, transformation, blending, filtering operations are applied to the pixel copy.

Syntax:

```
vg_lite_error_t vg_lite_copy_image (
    vg_lite_buffer_t              *target,
    vg_lite_buffer_t              *source,
    vg_lite_int32_t               sx,
    vg_lite_int32_t               sy,
    vg_lite_int32_t               dx,
    vg_lite_int32_t               dy,
    vg_lite_int32_t               width,
    vg_lite_int32_t               height
);
```

Parameters:

***target**	Points to the vg_lite_buffer_t structure which defines the destination buffer. See Image Source Alignment Requirement for valid destination color formats for the blit functions.
***source**	Points to the vg_lite_buffer_t structure for the source buffer. All color formats available in the vg_lite_buffer_format_t enum are valid source formats for the blit function.
sx, sy	Pixel coordinates of the lower-left corner of pixel rectangle within the source buffer.
dx, dy	Pixel coordinates of the lower-left corner of pixel rectangle within the target buffer.
width	Width of the copied pixel rectangle.
height	Height of the copied pixel rectangle.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_get_transform_matrix

Description:

This function generates a 3x3 homogeneous transformation matrix base on 4 coordinates of a source polygon and 4 coordinates of a target polygon. The generated matrix can perform non-affined transformation from the source polygon to the target polygon for perspective projection, or other non-linear transformation. However, this function may fail to generate a valid transformation matrix and return error if it is mathematically impossible to transform the source polygon to the target polygon. *(from March 2021)*

Syntax:

```
vg_lite_error_t vg_lite_get_transform_matrix (
    vg_lite_float_point4_t       src,
    vg_lite_float_point4_t       dst,
    vg_lite_matrix_t             *mat
);
```

Parameters:

src	Pointer to a set of four 2D float points that form a source polygon.
dst	Pointer to a set of four 2D float points that form a destination polygon.
mat	Output parameter, pointer to a 3x3 homogenous matrix that transforms the source polygon to a destination polygon.

Returns:

Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_clear

Description:
This function performs the clear operation, clearing/filling the specified buffer (entire buffer or partial rectangle in a buffer) with an explicit color.

Syntax:

```
vg_lite_error_t vg_lite_clear (
    vg_lite_buffer_t              *target,
    vg_lite_rectangle_t           *rect,
    vg_lite_color_t               color
);
```

Parameters:

***target**	Pointer to the vg_lite_buffer_t structure for the destination buffer. All color formats available in the vg_lite_buffer_format_t enum are valid destination formats for the clear function.
***rect**	Pointer to a vg_lite_rectangle_t structure that specifies the area to be filled. If the rectangle is NULL, the entire target buffer will be filled with the specified color.
color	Clear color, as specified in the vg_lite_color_t enum which is the color value to use for filling the buffer. If the buffer is in L8 format, the RGBA color will be converted into a luminance value.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_set_color_key

Description:

This function sets a color key. Color key can be used for blit or for draw pattern operations. *(from April 2022)*

A “color key” has two sections, where each section contains R, G, and B channels which are noted as high_rgb and low_rgb respectively.

When the vg_lite_color_key_t structure value enable is true, the color key specified is effective and the alpha value is used to replace the alpha channel of the destination pixel when its RGB channels are within range [low_rgb, high_rgb]. After the color key is used in the current frame, if the color key is not needed for the next frame, it should be disabled before the next frame.

Hardware support for color key is not available for GCNanoLiteV. Application can use VGLite API vg_lite_query_feature(gcFEATURE_BIT_VG_COLOR_KEY) to determine HW support for color key.

Syntax:

```
vg_lite_error_t vg_lite_set_color_key (
    vg_lite_color_key4_t          colorkey
);
```

Parameters:

colorkey	A color key as defined by vg_lite_color_key4_t. There are 4 groups of color key states: - color_key_0: high_rgb_0, low_rgb_0, alpha_0, enable_0. - color_key_1: high_rgb_1, low_rgb_1, alpha_1, enable_1. - color_key_2: high_rgb_2, low_rgb_2, alpha_2, enable_2. - color_key_3: high_rgb_3, low_rgb_3, alpha_3, enable_3. The priority order of these states is: color_key_0 > color_key_1 > color_key_2 > color_key_3.

Return:

VG_LITE_SUCCESS if successful. Otherwise, VG_LITE_NOT_SUPPORT if color key is not supported in hardware.

vg_lite_gaussian_filter

Description:
This function sets 3x3 gaussian blur weighted values to filter an image pixel. *(from March 2023)*

The parameters w0, w1, w2 define a 3x3 gaussian blur weight matrix as:

```
| w2   w1   w2 |
| w1   w0   w1 |
| w2   w1   w2 |
```

The sum of the 9 kernel weights must be 1.0 to avoid convolution overflow (w0 + 4*w1 + 4*w2 = 1.0).
The 3x3 weight matrix applies to a 3x3 pixel block:

```
| pixel[i-1][j-1]    pixel[i][j-1]      pixel[i+1][j-1]|
| pixel[i-1][j]      pixel[i][j]        pixel[i+1][j]  |
| pixel[i-1][j+1]    pixel[i][j+1]      pixel[i+1][j+1]|
```

With the following dot product equation:

```
color[i][j] = w2*pixel[i-1][j-1] + w1*pixel[i][j-1] + w2*pixel[i+1][j-1]
            + w1*pixel[i-1][j]   + w0*pixel[i][j]   + w1*pixel[i+1][j]
            + w2*pixel[i-1][j+1] + w1*pixel[i][j+1] + w2*pixel[i+1][j+1];
```

Applications can use VGLite API vg_lite_query_feature (gcFEATURE_BIT_VG_GAUSSIAN_BLUR) to determine HW support for gaussian blur.

Syntax:

```
vg_lite_error_t vg_lite_gaussian_filter (
    vg_lite_float_t                w0
    vg_lite_float_t                w1
    vg_lite_float_t                w2
);
```

Parameters:

w0, w1, w2 w0, w1, w2 define a 3x3 gaussian blur weighted matrix as:

```
| w2   w1   w2 |
| w1   w0   w1 |
| w2   w1   w2 |
```

Return:
VG_LITE_SUCCESS if successful. Otherwise, VG_LITE_NOT_SUPPORT if gaussian blur is not supported in hardware.

7.4 Blit/Draw Extended Functions

The following BLIT or DRAW related functions typically require GC355 or GC555 hardware and are not available for all Vivante Vector Graphics hardware configurations.

Applications can use VGLite API vg_lite_query_feature to determine HW support for the related functionality.

vg_lite_get_parameter

Description:
This function returns the selected VGLite / GPU states to the application.
(from Aug 2023)

Syntax:

```
vg_lite_error_t vg_lite_get_parameter (
    vg_lite_param_type_t          type,
    vg_lite_int32_t               count,
    vg_lite_pointer               params
);
```

Parameters:

type	The parameter type to be queried.
count	The number of returned parameters.
params	The pointer to the array of returned parameters.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_enable_scissor

Description:
This function enables scissor rectangle operation for the rectangle regions defined by the **vg_lite_scissor_rects** API. *(from March 2020, modified August 2020)*

Applications can use VGLite API vg_lite_query_feature (gcFEATURE_BIT_VG_SCISSOR) to determine HW support for the following scissoring APIs: **vg_lite_enable_scissor,** **vg_lite_disable_scissor,** and **vg_lite_scissor_rects**. Support is available with GC355, GC555 and selected GC265.

Syntax:

```
vg_lite_error_t vg_lite_enable_scissor (
      void
  );
```

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_disable_scissor

Description:
This function disables scissor operation for the rectangle regions defined by **vg_lite_scissor_rects** API. *(from March 2020, modified August 2020)*

Syntax:

```
vg_lite_error_t vg_lite_disable_scissor (
      void
  );
```

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_scissor_rects

Description:
This function defines scissor rectangle regions on the hardware mask layer. However, the scissor function is enable/disabled by **vg_lite_enable_scissor** and **vg_lite_disable_scissor** APIs.
(from August 2022)

Syntax:

```
vg_lite_error_t vg_lite_scissor_rects (
    vg_lite_buffer_t              *target,
    vg_lite_uint32_t              nums,
    vg_lite_rectangle_t           rect[]
);
```

Parameters:

target	Target render buffer that has the scissor mask layer.
nums	Number of scissor rectangles.
rect[]	The scissor rectangle array.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_set_scissor

Description:
This is a legacy scissor API function that can be used to set a single scissor rectangle for the render target. This scissor API is supported by a different hardware mechanism other than the mask layer, and it has better performance than the mask layer scissor function.

This API is not enabled/disabled by **vg_lite_enable_scissor** and **vg_lite_disable_scissor** APIs. Instead, the **vg_lite_set_scissor** API calls with a valid scissor rectangle input (x, y, right, bottom) enables the scissor function by default. The **vg_lite_set_scissor** API call with input parameter (-1, -1, -1, -1) disables the scissor function.

Syntax:

```
vg_lite_error_t vg_lite_set_scissor (
    vg_lite_int32_t              x,
    vg_lite_int32_t              y,
    vg_lite_int32_t              right,
    vg_lite_int32_t              bottom
);
```

Parameters:

x	X Origin of rectangle, left coordinate in pixels
y	Y Origin of rectangle, top coordinate in pixels
right	X rightmost pixel of rectangle
bottom	Y bottom pixel of rectangle

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_disable_color_transform

Description:
This function is used to disable color transformation. By default, color transform is turned off. *(from Sept 2022, only for GC355 and GC555 hardware)*

Applications can use VGLite API vg_lite_query_feature (gcFEATURE_BIT_VG_COLOR_TRANSFORMATION) to determine HW support for color transformation. Support is available with GC355 and GC555.

Syntax:

```
vg_lite_error_t vg_lite_disable_color_transform (
);
```

Parameters: None

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_enable_color_transform

Description:
This function is used to enable color transformation. By default, color transform is turned off. *(from Sept 2022, only for GC355 and GC555 hardware)*

Syntax:

```
vg_lite_error_t vg_lite_enable_color_transform (
);
```

Parameters: None

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_set_color_transform

Description:
This function is used to set pixel scale and bias values for color transformation for each pixel channel. *(from August 2022, only for GC355 and GC555 hardware)*

Syntax:

```
vg_lite_error_t vg_lite_set_color_transform (
    vg_lite_color_transform_t     *values
);
```

Parameters:

***values**	Pointer to the color transformation values to set. See enum vg_lite_color_transform_t.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_enable_masklayer

Description:
This function controls the availability of mask functionality. Mask is turned off by default. *(from August -Sept 2022, requires GC555 hardware)*

Applications can use VGLite API vg_lite_query_feature (gcFEATURE_BIT_VG_MASK) to determine HW support for mask. The blit and draw mask functions below require GC555 hardware support. These functions were introduced in August 2022 and the function's syntax or name were further refined in September 2022.

Syntax:

```
vg_lite_error_t vg_lite_enable_masklayer (
    void
  );
```

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_disable_masklayer

Description:
This function controls the availability of mask functionality. Mask is turned off by default. *(from August -Sept 2022, requires GC555 hardware, prior to Sept 2022 name was vg_lite_disable_mask_layer)*

Syntax:
```
vg_lite_error_t vg_lite_disable_masklayer (
    void
);
```

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_create_masklayer

Description:
This function creates a mask layer with the specified width and height. The mask format defaults to A8 and the default mask value is 255. *(from August 2022-Sept, requires GC555 hardware)*

Syntax:
```
vg_lite_error_t vg_lite_create_masklayer (
    vg_lite_buffer_t              *masklayer,
    vg_lite_uint32_t              width,
    vg_lite_uint32_t              height
);
```

Parameters:

***masklayer**	Points to the address of the buffer of the mask layer to be created.
width	Mask layer width (in pixels).
height	Mask layer height (in pixels).

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_fill_masklayer

Description:
This function sets the values of a given mask layer within a given rectangular region to a given value. The value must be between 0 and 255. *(from August-Sept 2022, requires GC555 hardware)*

Syntax:

```
vg_lite_error_t vg_lite_fill_masklayer (
    vg_lite_buffer_t            *masklayer,
    vg_lite_rectangle_t         *rect,
    vg_lite_uint8_t             value
);
```

Parameters:

***masklayer**	Points to the address of the buffer of the mask layer to be filled.
***rect**	The rectangle area (x, y, width, height) to be filled with value.
value	The value of the fill area. The value must be between 0 and 255.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_blend_masklayer

Description:
This function blends the specified area of the source mask layer with the destination mask layer according to a vg_lite_mask_operation_t enumeration value, to create a blended destination mask layer. *(from August-Sept 2022, requires GC555 hardware)*

Syntax:

```
vg_lite_error_t vg_lite_blend_masklayer (
    vg_lite_buffer_t            *dst_masklayer,
    vg_lite_buffer_t            *src_masklayer,
    vg_lite_mask_operation      operation,
    vg_lite_rectangle_t         *rect,
);
```

Parameters:

***dst_masklayer**	Points to the address of the buffer of the destination mask layer.
***src_masklayer**	Points to the address of the buffer of the source mask layer.
operation	Blending mode to be applied to each image pixel, as defined by enum vg_lite_mask_operation_t.
***rect**	The rectangle area (x, y, width, height) of the blend operation.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_set_masklayer

Description:
This function sets the given mask layer to the hardware. *(from August-Sept 2022, requires GC555 hardware)*

Syntax:

```
vg_lite_error_t vg_lite_set_masklayer (
    vg_lite_buffer_t             *masklayer
  );
```

Parameters:

***masklayer**	Points to the address of the buffer of the mask layer to be set.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_render_masklayer

Description:
This function draws the mask layer according to the specified path, color and matrix information. *(from August-Sept 2022, requires GC555 hardware)*

Syntax:

```
vg_lite_error_t vg_lite_render_masklayer (
    vg_lite_buffer_t              *masklayer,
    vg_lite_mask_operation        operation,
    vg_lite_path_t                *path,
    vg_lite_fill_t                fill_rule,
    vg_lite_color_t               color,
    vg_lite_matrix_t              *matrix
);
```

Parameters:

***masklayer**	Points to the address of the buffer of the destination mask layer.
operation	Blending mode to be applied to each image pixel, as defined by enum vg_lite_mask_operation_t.
***path**	Pointer to the vg_lite_path_t structure containing path data which describes the path to draw. Refer to the section on Vector Path Data Opcodes in this document for opcode detail.
fill_rule	Specifies the vg_lite_fill_t enum value for the fill rule for the path.
color	Specifies the color vg_lite_color_t RGBA value to be applied to each pixel drawn by the path.
***matrix**	Points to a vg_lite_matrix_t structure that defines the 3x3 transformation matrix of the path. If matrix is NULL, an identity matrix is assumed, meaning the source will be copied directly on the target at 0,0 location. which is usually a bad idea since the path can be anything.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_destroy_masklayer

Description:
This function is used to free a mask layer. *(from August-Sept 2022, requires GC555 hardware)*

Syntax:

```
vg_lite_error_t vg_lite_destroy_masklayer (
    vg_lite_buffer_t              masklayer
);
```

Parameters:

***masklayer**	Points to the address of the buffer of the mask layer to be destroyed.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_set_mirror

Description:
This function is used to control mirror functionality. By default, mirror is turned off and the default output orientation is from top to bottom. *(from August 2022, only for GC555 hardware)*

Application can use VGLite API vg_lite_query_feature (gcFEATURE_BIT_VG_MIRROR) to determine HW support for mirror. Mirror functions require GC555 hardware.

Syntax:

```
vg_lite_error_t vg_lite_set_mirror (
    vg_lite_orientation_t         orientation
);
```

Parameters:

orientation	The orientation mode as defined by enum vg_lite_orientation_t.

Returns:
VG_LITE_SUCCESS or VG_LITE_NOT_SUPPORT if not supported.

vg_lite_source_global_alpha

Description:
This function will set image/source global alpha and return a status error code. *(from June 2021, requires GCNanoUltraV or GC555 hardware)*

Application can use VGLite API vg_lite_query_feature (gcFEATURE_BIT_VG_GLOBAL_ALPHA) to determine HW support for global alpha. The global alpha BLIT related functions require GCNanoUltraV or GC555 hardware.

Syntax:

```
vg_lite_error_t vg_lite_source_global_alpha (
    vg_lite_global_alpha_t        alpha_mode,
    vg_lite_uint8_t               alpha_value
);
```

Parameters:

alpha_mode	Global alpha mode value. See enum vg_lite_global_alpha_t.
alpha_value	The image/source global alpha value to set.

Returns:
VG_LITE_SUCCESS or VG_LITE_NOT_SUPPORT if global alpha is not supported.

vg_lite_dest_global_alpha

Description:
This function will set destination global alpha and return a status error code. *(from June 2021, requires GCNanoUltraV or GC555 hardware)*

Syntax:

```
vg_lite_error_t vg_lite_dest_global_alpha (
    vg_lite_global_alpha_t        alpha_mode,
    vg_lite_uint8_t               alpha_value
);
```

Parameters:

alpha_mode	Global alpha mode value. See enum vg_lite_global_alpha_t.
alpha_value	The destination global alpha value to set.

Returns:
VG_LITE_SUCCESS or VG_LITE_NOT_SUPPORT if global alpha is not supported.

8 Vector Path Control

8.1 Vector Path Enumerations

8.1.1 vg_lite_format_t Enumeration

Values for vg_lite_format_t are defined in the table Common Parameter Types. LINK to Common Parameters table.

If vg_lite_format_t	Path data alignment in array should be:
VG_LITE_S8	8 bit
VG_LITE_S16	2 bytes
VG_LITE_S32	4 bytes

8.1.2 vg_lite_quality_t Enumeration

Specifies the level of hardware assisted anti-aliasing.

Used in structure: vg_lite_path_t.
Used in functions: vg_lite_init_path, vg_lite_init_arc_path.

vg_lite_quality_t String Values	Description
VG_LITE_HIGH	High quality: 16x coverage sample anti-aliasing
VG_LITE_UPPER	Upper quality: 8x coverage sample anti-aliasing. Use vg_lite_query_feature to determine availability of 8x CSAA (feature enum value gcFEATURE_BIT_VG_QUALITY_8X. *(deprecated from June 2020, available with supported hardware from August 2022)*
VG_LITE_MEDIUM	Medium quality: 4x coverage sample anti-aliasing
VG_LITE_LOW	Low quality: no anti-aliasing

8.2 Vector Path Structures

8.2.1 vg_lite_hw_memory Structure

This structure simply records the memory allocation info by kernel.

Used in structure: vg_lite_path_t.

vg_lite_hw_memory_t Members	Type	Description
handle	vg_lite_pointer	GPU memory object handle
memory	vg_lite_pointer	Logical memory address
address	vg_lite_uint32_t	GPU memory address
bytes	vg_lite_uint32_t	Size of memory
property	vg_lite_uint32_t	Bit 0 is used for path upload: 0: Disable path data uploading (always embedded into command buffer). 1: Enable auto path data uploading.

8.2.2 vg_lite_path_t Structure

This structure describes VGLite path data.
Path data is composed of op codes and coordinates. The format for op codes is always VG_LITE_S8. Refer to the section on Vector Path Data Opcodes in this document for opcode detail.

Used in init functions: vg_lite_init_path, vg_lite_init_arc_path, vg_lite_upload_path, vg_lite_clear_path, vg_lite_append_path.
Used in function: vg_lite_render_masklayer.
Used in draw functions: vg_lite_draw, vg_lite_draw_grad, vg_lite_draw_radial_grad, vg_lite_draw_pattern.

vg_lite_path_t Members	Type	Description
bounding_box[4]	vg_lite_float_t	bounding box for path [0] left [1] top [2] right [3] bottom
quality	vg_lite_quality_t	enum for quality hint for the path, anti-aliasing level
format	vg_lite_format_t	enum for coordinate format. The coordinates may have these formats: If vg_lite_format_t / Path data alignment in array should be: VG_LITE_S8 — 8 bit VG_LITE_S16 — 2 bytes VG_LITE_S32 — 4 bytes
uploaded	vg_lite_hw_memory_t	struct with path data that has been uploaded into GPU addressable memory
path_length	vg_lite_uint32_t	number of bytes in the path data
path	vg_lite_pointer	pointer to the physical description of the path
path_changed	vg_lite_int8_t	0: not changed; 1: changed.
pdata_internal	vg_lite_int8_t	0: path data memory is allocated by the application; 1: path data memory is allocated by the driver.
path_type	vg_lite_path_type_t	The draw path type as specified in enum vg_lite_path_type_t. *(added for stroke control, from March 2022)*
*stroke	vg_lite_stroke_t	As defined by structure vg_lite_stroke_t *(added for stroke control, from March 2022)*
stroke_path	vg_lite_pointer	Pointer to the physical description of the stroke path. *(added for stroke control, from March 2022)*
stroke_size	vg_lite_uint32_t	Number of bytes in the stroke path data. *(added for stroke control, from March 2022)*
stroke_color	vg_lite_color_t	The stroke path fill color. *(from Sept 2022)*
add_end	vg_lite_int8_t	Flag that add end_path in driver *(from March 2023)*

Special Notes for Path Objects:

- Endianness has no impact, as it is aligned against the boundaries.

- Multiple contiguous op codes should be packed by the size of the specified data format. E.g., by 2 bytes for VG_LITE_S16 or by 4 bytes for VG_LITE_S32.
    - For example, since opcodes are 8-bits (1 byte), for 16-bit (2 byte) or 32-bit (4 byte) data types:

```
…
<opcode1_that_needs_data>
<align_to_data_size>
<data_for_opcode1>
<opcode2_that_doesnt_need_data>
<opcode3_that_needs_data>
<align_to_data_size>
<data_for_opcode3>
…
```

- Path data in the array should always be 1-, 2, or 4-byte aligned, depending on the format:
    - For example, for 32-bit (4 byte) data types:

```
…
<opcode1_that_needs_data>
<pad to 4 bytes>
<4 byte data_for_opcode1>
<opcode2_that_doesnt_need_data>
<opcode3_that_needs_data>
<pad to 4 bytes>
<4 byte data_for_opcode3>
…
```

8.3 Vector Path Functions

When using a small tessellation window and depending on a path's size, a path might be uploaded to the hardware multiple times because the hardware scanline convert path with the provided tessellation window size, so VGLite path rendering performance might go down. So, it is better to set the tessellation buffer size to the most common path size, for example if you only render 24-pt fonts, you can set the tessellation buffer to be 24x24.

All the RGBA color formats available in the vg_lite_buffer_format_t are supported as the destination buffer for the draw function.

vg_lite_get_path_length

Description:
This function calculates the path command buffer length (in bytes).

The application is responsible for allocating a buffer according to the buffer length calculated with this function. Then the buffer is used by the path as a command buffer. The VGLite driver does not allocate the path command buffer.

Syntax:

```
vg_lite_uint32_t vg_lite_get_path_length (
    vg_lite_uint8_t             *opcode,
    vg_lite_uint32_t            count,
    vg_lite_format_t            format
);
```

Parameters:

***opcode**	Pointer to the opcode array to use to construct the path. *(*opcode from March 2023)*
count	The opcode count.
format	The coordinate data format. All formats available for vg_lite_format_t are valid formats for this function.

Returns:
Returns the command buffer length in bytes.

vg_lite_append_path

Description:
This function assembles the command buffer for the path. The command buffer is allocated by the application and assigned to the path. This function makes the final GPU command buffer for the path based on the input opcodes (cmd) and coordinates (data). Note that the application is responsible for allocating a buffer large enough for the path. *(from Jan 2022, returns a vg_lite_error_t status code)*

Syntax:

```
vg_lite_error_t vg_lite_append_path (
    vg_lite_path_t              *path
    vg_lite_uint8_t             *opcode,
    vg_lite_pointer             data,
    vg_lite_uint32_t            seg_count
);
```

Parameters:

***path**	Pointer to the vg_lite_path_t structure with the path definition.
***opcode**	Pointer to the opcode array to use to construct the path. *(*opcode from March 2023)*
data	Pointer to the coordinate data array to use to construct the path.
seg_count	The opcode count.

Returns:
Returns VG_LITE_SUCCESS if successful. See vg_lite_error_t enum for other return codes.

vg_lite_init_path

Description:
This function initializes a path definition with specified values. *(From Dec 2019 returns vg_lite_error_t, previous was void.)*

Syntax:

```
vg_lite_error_t vg_lite_init_path (
    vg_lite_path_t              *path,
    vg_lite_format_t            format,
    vg_lite_quality_t           quality,
    vg_lite_uint32_t            length,
    vg_lite_pointer             *data,
    vg_lite_float_t             min_x,
    vg_lite_float_t             min_y,
    vg_lite_float_t             max_x,
    vg_lite_float_t             max_y
);
```

Parameters:

***path**	Pointer to the vg_lite_path_t structure for the path object to be initialized with the member values specified.
format	The coordinate data format. All formats available in the vg_lite_format_t enum are valid formats for this function.
quality	The quality for the path object. All formats available in the vg_lite_quality_t enum are valid formats for this function.
length	The length of the path data (in bytes).
***data**	Pointer to path data. Note: the driver just saves the "data" pointer and references the path data through the "data" pointer for the path rendering. Therefore, the application must ensure the path data is static before the path rendering is completed.
min_x **min_y** **max_x** **max_y**	Minimum and maximum x and y values specifying the bounding box of the path.

Returns:
Returns VG_LITE_SUCCESS if successful. See vg_lite_error_t enum for other return codes.

vg_lite_init_arc_path

Description:
This function initializes an arc path definition with specified values. *(from February 2021)*

Syntax:

```
vg_lite_error_t vg_lite_init_arc_path (
    vg_lite_path_t              *path,
    vg_lite_format_t            format,
    vg_lite_quality_t           quality,
    vg_lite_uint32_t            length,
    vg_lite_pointer             *data,
    vg_lite_float_t             min_x,
    vg_lite_float_t             min_y,
    vg_lite_float_t             max_x,
    vg_lite_float_t             max_y
);
```

Parameters:

***path**	Pointer to the vg_lite_path_t structure for the path object to be initialized with the member values specified.
format	The coordinate data format. The vg_lite_format_t enum value should be FP32.
quality	The quality for the path object. All formats available in the vg_lite_quality_t enum are valid formats for this function.
length	The length of the path data (in bytes).
***data**	Pointer to path data.
min_x **min_y** **max_x** **max_y**	Minimum and maximum x and y values specifying the bounding box of the path.

Returns:
Returns VG_LITE_SUCCESS if successful. See vg_lite_error_t enum for other return codes.

vg_lite_upload_path

Description:
This function is used to upload a path to GPU memory.

In normal cases, the VGLite driver will copy any path data into a command buffer structure during runtime. This does take some time if there are many paths to be rendered. Also, in an embedded system the path data won't change - so it makes sense to upload the path data into GPU memory in such a form that the GPU can directly access it. This function will signal the driver to allocate a buffer that will contain the path data and the required command buffer header and footer data for the GPU to access the data directly. Call vg_lite_clear_path to free this buffer after the path is used.

Syntax:

```
vg_lite_error_t vg_lite_upload_path (
    vg_lite_path_t              *path
);
```

Parameters:

***path**	Pointer to a vg_lite_path_t structure that contains the path to be uploaded.

Returns:
VG_LITE_OUT_OF_MEMORY if not enough GPU memory is available for buffer allocation.

vg_lite_clear_path

Description:
This function will clear and reset path member values. If the path has been uploaded, it frees the GPU memory allocated when uploading the path. *(From Dec 2019 returns vg_lite_error_t, previous was void.)*

Syntax:

```
Vg_lite_error_t vg_lite_clear_path (
    vg_lite_path_t              *path
);
```

Parameters:

***path**	Pointer to the vg_lite_path_t path definition to be cleared.

Returns:
Returns VG_LITE_SUCCESS if successful. See vg_lite_error_t enum for other return codes.

8.4 Vector Path Opcodes for Plotting Paths

The following opcodes are path drawing commands available for vector path data.

A Path operation is submitted to the GPU as [Opcode | Coordinates]. The operation code is stored as a VG_LITE_S8 while the Coordinates are specified via vg_lite_format_t.

Table 4. Vector Path Data Opcodes

Opcode	Arguments	Description
0x00	None	**VLC_OP_END**. Finish tessellation. Close any open path.
0x01	None	**VLC_OP_CLOSE**. For VGLite driver internal use only. Application should not use this OP directly.
0x02	(x,y)	**VLC_OP_MOVE**. Move to the given vertex. Close any open path. $start_x = x$ $start_y = y$
0x03	(Δx,Δy)	**VLC_OP_MOVE_REL**. Move to the given relative point. Close any open path. $start_x = start_x + \Delta x$ $start_y = start_y + \Delta y$
0x04	(x,y)	**VLC_OP_LINE**. Draw a line to the given point. $Line(start_x, start_y, x, y)$ $start_x = x$ $start_y = y$
0x05	(Δx,Δy)	**VLC_OP_LINE_REL**. Draw a line to the given relative point. $x = start_x + \Delta x$ $y = start_y + \Delta y$ $Line(start_x, start_y, x, y)$ $start_x = x$ $start_y = y$
0x06	(cx,cy) (x,y)	**VLC_OP_QUAD**. Draw a quadratic Bezier curve to the given end point using the specified control point. $Quad(start_x, start_y, cx, cy, x, y)$ $start_x = x$ $start_y = y$
0x07	(Δcx,Δcy) (Δx,Δy)	**VLC_OP_QUAD_REL**. Draw a quadratic Bezier curve to the given relative end point using the specified relative control point. $cx = start_x + \Delta cx$ $cy = start_y + \Delta cy$ $x = start_x + \Delta x$ $y = start_y + \Delta y$ $Quad(start_x, start_y, cx, cy, x, y)$ $start_x = x$ $start_y = y$

Opcode	Arguments	Description
0x08	(cx_1,cy_1) (cx_2,cy_2) (x,y)	**VLC_OP_CUBIC**. Draw a cubic Bezier curve to the given end point using the specified control points. $Cubic(start_x, start_y, cx_1, cy_1, cx_2, cy_2, x, y)$ $start_x = x$ $start_y = y$
0x09	(Δcx_1,Δcy_1) (Δcx_2,Δcy_2) (Δx,Δy)	**VLC_OP_CUBIC_REL**. Draw a cubic Bezier curve to the given relative end point using the specified relative control points. $cx_1 = start_x + \Delta cx_1$ $cy_1 = start_y + \Delta cy_1$ $cx_2 = start_x + \Delta cx_2$ $cy_2 = start_y + \Delta cy_2$ $x = start_x + \Delta x$ $y = start_y + \Delta y$ $Cubic(start_x, start_y, cx_1, cy_1, cx_2, cy_2, x, y)$ $start_x = x$ $start_y = y$
0x0A	None	**VLC_OP_BREAK**. Indicates 64-bit path data (including the opcode) is a no-op.
0x0B	(x)	**VLC_OP_HLINE**. Draw a horizontal line to the given point. $Line(start_x, start_y, x, start_y)$ $start_x = x$
0x0C	(Δx)	**VLC_OP_HLINE_REL**. Draw a horizontal line to the given relative point. $x = start_x + \Delta x$ $Line(start_x, start_y, x, start_y)$ $start_x = x$
0x0D	(y)	**VLC_OP_VLINE**. Draw a vertical line to the given point. $Line(start_x, start_y, start_x, y)$ $start_y = y$
0x0E	(Δy)	**VLC_OP_VLINE_REL**. Draw a vertical line to the given relative point. $y = start_y + \Delta y$ $Line(start_x, start_y, start_x, y)$ $start_y = y$
0x0F	(x,y)	**VLC_OP_SQUAD**. Draw a smooth quadratic Bezier curve to the given end point. The curve starts at the current vertex and the control point is computed as twice the current vertex minus the current control point. $cx = 2 * start_x - cx$ $cy = 2 * start_y - cy$ $Quad(start_x, start_y, cx, cy, x, y)$ $start_x = x$ $start_y = y$

Opcode	Arguments	Description
0x10	(Δx,Δy)	**VLC_OP_SQUAD_REL**. Draw a smooth quadratic Bezier curve to the given relative end point. The curve starts at the current vertex and the control point is computed as twice the current vertex minus the current control point. $cx = 2 * start_x - cx$ $cy = 2 * start_y - cy$ $x = start_x + \Delta x$ $y = start_y + \Delta y$ $Quad(start_x, start_y, cx, cy, x, y)$ $start_x = x$ $start_y = y$
0x11	(cx_2,cy_2) (x,y)	**VLC_OP_SCUBIC**. Draw a smooth cubic Bezier curve to the given end point. The curve starts at the current vertex and the first control point (cx_1, cy_1) is computed as twice the current vertex minus the current control point. The second control point (cx_2, cy_2) is specified as arguments. $cx_1 = 2 * start_x - cx_2$ $cy_1 = 2 * start_y - cy_2$ $Cubic(start_x, start_y, cx_1, cy_1, cx_2, cy_2, x, y)$ $start_x = x$ $start_y = y$
0x12	(Δcx_2,Δcy_2) (Δx,Δy)	**VLC_OP_SCUBIC_REL**. Draw a smooth cubic Bezier curve to the given relative end point. The curve starts at the current vertex and the first control point (cx_1, cy_1) is computed as twice the current vertex minus the current control point. The second control point (cx_2, cy_2) is specified as arguments. $cx_2 = start_x + \Delta cx_2$ $cy_2 = start_y + \Delta cy_2$ $cx_1 = 2 * start_x - cx_2$ $cy_1 = 2 * start_y - cy_2$ $x = start_x + \Delta x$ $y = start_y + \Delta y$ $Cubic(start_x, start_y, cx_1, cy_1, cx_2, cy_2, x, y)$ $start_x = x$ $start_y = y$
0x13	(rh,rv,rot,x,y)	**VLC_OP_SCCWARC**. Draw a small CCW Arc to the given end point using the specified radius and rotation angle. $SCCWARC(rh, rv, rot, x, y)$ $start_x = x$ $start_y = y$
0x14	(rh,rv,rot,x,y)	**VLC_OP_SCCWARC_REL**. Draw a small CCW Arc to the given relative end point using the specified radius and rotation angle. $x = start_x + \Delta x$ $y = start_y + \Delta y$ $SCCWARC(rh, rv, rot, x, y)$ $start_x = x$ $start_y = y$
0x15	(rh,rv,rot,x,y)	**VLC_OP_SCWARC**. Draw a small CW Arc to the given end point using the specified radius and rotation angle. $SCWARC(rh, rv, rot, x, y)$ $start_x = x$ $start_y = y$

Opcode	Arguments	Description
0x16	(rh,rv,rot,x,y)	**VLC_OP_SCWARC_REL**. Draw a small CW Arc to the given relative end point using the specified radius and rotation angle. $x = start_x + \Delta x$ $y = start_y + \Delta y$ $SCWARC(rh, rv, rot, x, y)$ $start_x = x$ $start_y = y$
0x17	(rh,rv,rot,x,y)	**VLC_OP_LCCWARC**. Draw a large CCW Arc to the given end point using the specified radius and rotation angle. $LCCWARC(rh, rv, rot, x, y)$ $start_x = x$ $start_y = y$
0x18	(rh,rv,rot,x,y)	**VLC_OP_LCCWARC_REL**. Draw a large CCW Arc to the given relative end point using the specified radius and rotation angle. $x = start_x + \Delta x$ $y = start_y + \Delta y$ $LCCWARC(rh, rv, rot, x, y)$ $start_x = x$ $start_y = y$
0x19	(rh,rv,rot,x,y)	**VLC_OP_LCWARC**. Draw a large CW Arc to the given end point using the specified radius and rotation angle. $LCWARC(rh, rv, rot, x, y)$ $start_x = x$ $start_y = y$
0x1A	(rh,rv,rot,x,y)	**VLC_OP_LCWARC_REL**. Draw a large CW Arc to the given relative end point using the specified radius and rotation angle. $x = start_x + \Delta x$ $y = start_y + \Delta y$ $LCWARC(rh, rv, rot, x, y)$ $start_x = x$ $start_y = y$

9 Vector Based Draw Operations

This part of the API performs the hardware accelerated draw operations.

9.1 Draw and Gradient Enumerations

9.1.1 vg_lite_blend_t Enumeration

This enumeration is detailed under the Blit section. LINK to vg_lite_blend_t enumeration.

9.1.2 vg_lite_color_t Parameter

The common parameter vg_lite_color_t is described in Section 1.4 Common Parameter Types.
LINK to vg_lite_color_t color parameter description.

9.1.3 vg_lite_fill_t Enumeration

This enumeration is used to specify the fill rule to use. For drawing any path, the hardware supports both non-zero and odd-even fill rules.

To determine whether any point is contained inside an object, imagine drawing a line from that point out to infinity in any direction such that the line does not cross any vertex of the path. For each edge that is crossed by the line, add 1 to the counter if the edge is crossed from left to right, as seen by an observer walking across the line towards infinity, and subtract 1 if the edge crossed from right to left. In this way, each region of the plane will receive an integer value.

The non-zero fill rule says that a point is inside the shape if the resulting sum is not equal to zero. The even/odd rule says that a point is inside the shape if the resulting sum is odd, regardless of sign.

Used in function: vg_lite_render_masklayer.
Used in draw functions: vg_lite_draw, vg_lite_draw_grad, vg_lite_draw_radial_grad, vg_lite_draw_pattern.

vg_lite_fill_t String Values	Description
VG_LITE_FILL_NON_ZERO	Non-zero fill rule. A pixel is drawn if it crosses at least one path pixel.
VG_LITE_FILL_EVEN_ODD	Even-odd fill rule. A pixel is drawn if it crosses an odd number of path pixels.

9.1.4 vg_lite_filter_t Enumeration

Defined under Blit. LINK to vg_lite_filter_t enumeration.

9.1.5 vg_lite_gradient_spreadmode_t Enumeration

vg_lite_gradient_spreadmode_t enum is defined to match OpenVG enum VGColorRampSpreadMode *(from March 2023, replaces vg_lite_radial_gradient_spreadmode, requires GC355/GC555 hardware)*

The application may only define stops with offsets between 0 and 1. Spread modes define how the given set of stops are repeated or extended in order to define interpolated color values for arbitrary input values outside the [0,1] range.

Used in structure: vg_lite_radial_gradient_t.

vg_lite_gradient_spreadmode_t String Values	Description
VG_LITE_GRADIENT_SPREAD_FILL	The current fill color is used for all stop values less than 0 or greater than 1, respectively.
VG_LITE_GRADIENT_SPREAD_PAD	Colors defined at 0 and 1 are used for all stop values less than 0 or greater than 1, respectively.
VG_LITE_GRADIENT_SPREAD_REPEAT	Color values defined between 0 and 1 are repeated indefinitely in both directions.
VG_LITE_GRADIENT_SPREAD_REFLECT	Color values defined between 0 and 1 are repeated indefinitely in both directions, but with alternate copies of the range reversed.

9.1.6 vg_lite_pattern_mode_t Enumeration

Defines how the region outside the image pattern is filled for the path.

Used in function: vg_lite_draw_grad, vg_lite_draw_pattern.

vg_lite_pattern_mode_t String Values	Description
VG_LITE_PATTERN_COLOR	Pixels outside the bounds of the source image should be taken as the color
VG_LITE_PATTERN_PAD	Pixels outside the bounds of the source image should be taken as having the same color as the closest edge pixel. The color of the pattern border is expanded to fill the region outside the pattern.
VG_LITE_PATTERN_REPEAT	Pixels outside the bounds of the source image should be repeated indefinitely in all directions. *(from March 2023)*
VG_LITE_PATTERN_REFLECT	Pixels outside the bounds of the source image should be reflected indefinitely in all directions. *(from March 2023)*

9.1.7 vg_lite_radial_gradient_spreadmode_t Enumeration

(Deprecated March 2023) use vg_lite_gradient_spreadmode_t. Defines the radial gradient padding mode. *(from Nov 2020, requires GC355 hardware)*

Used in structure: vg_lite_radial_gradient_t.

vg_lite_radial_gradient_spreadmode_t String Values	Description
VG_LITE_RADIAL_GRADIENT_SPREAD_FILL = 0	The current fill color is used for all stop values less than 0 or greater than 1, respectively.
VG_LITE_RADIAL_GRADIENT_SPREAD_PAD	Colors defined at 0 and 1 are used for all stop values less than 0 or greater than 1, respectively.
VG_LITE_RADIAL_GRADIENT_SPREAD_REPEAT	Color values defined between 0 and 1 are repeated indefinitely in both directions.
VG_LITE_RADIAL_GRADIENT_SPREAD_REFLECT	Color values defined between 0 and 1 are repeated indefinitely in both directions, but with alternate copies of the range reversed.

9.2 Draw and Gradient Structures

9.2.1 vg_lite_buffer_t Structure

Defined under Pixel Buffer Structures. LINK to vg_lite_buffer_t structure.

9.2.2 vg_lite_color_ramp_t Structure

This structure defines the stops for the radial gradient. The five parameters provide the offset and color for the stop. Each stop is defined by a set of floating-point values which specify the offset and the sRGBA color and alpha values. Color channel values are in the form of a non-premultiplied (R, G, B, alpha) quad. All parameters are in the range of [0,1]. The red, green, blue, alpha value of [0, 1] is mapped to an 8-bit pixel value [0, 255]. *(from November 2020, requires GC355 hardware)*

The define for the max number of radial gradient stops is #define MAX_COLOR_RAMP_STOPS 256.

Used in radial gradient structure: vg_lite_radial_gradient_t.

vg_lite_color_ramp_t Members	Type	Description
stop	vg_lite_float_t	Offset value for the color stop
red	vg_lite_float_t	Red color channel value for the color stop
green	vg_lite_float_t	Green color channel value for the color stop
blue	vg_lite_float_t	Blue color channel value for the color stop
alpha	vg_lite_float_t	Alpha color channel value for the color stop

9.2.3 vg_lite_linear_gradient_t Structure

This structure defines the organization of a linear gradient in VGLite data. The linear gradient is applied to filling a path. It will generate a 256x1 image according to the specified settings.

Used in init and draw functions: vg_lite_init_grad, vg_lite_set_grad, vg_lite_update_grad, vg_lite_get_grad_matrix, vg_lite_clear_grad, vg_lite_draw_grad.

vg_lite_linear_gradient_t Constants	**Type**	**Description**
VLC_MAX_GRADIENT_STOPS	vg_lite_int32_t	Constant. Maximum number of gradient colors = 16.
vg_lite_linear_gradient_t Members	**Type**	**Description**
colors[VLC_MAX_GRADIENT_STOPS]	vg_lite_uint32_t	Color array for the gradient
count	vg_lite_uint32_t	Number of colors
stops[VLC_MAX_GRADIENT_STOPS]	vg_lite_uint32_t	Number of color stops, from 0 to 255
matrix	vg_lite_matrix_t	Struct for the matrix to transform the gradient color ramp
image	vg_lite_buffer_t	Image object struct to represent the color ramp

9.2.4 vg_lite_ext_linear_gradient Structure

This structure defines the organization of the extended parameters possible for a linear gradient. *(from April 2022)*

Used in functions: vg_lite_draw_linear_grad.

vg_lite_ext_linear_gradient_t Members	**Type**	**Description**
count	vg_lite_uint32_t	Count of colors, up to 256.
matrix	vg_lite_matrix_t	The matrix to transform the gradient.
image	vg_lite_buffer_t	The image for rendering as gradient pattern.
linear_grad	vg_lite_linear_gradient_parameter_t	Linear gradient parameters. Includes center point, focal point and radius.
ramp_length	vg_lite_uint32_t	Color ramp length for gradient paints provided to the driver
color_ramp[VLC_MAX_COLOR_RAMP_STOPS]	vg_lite_color_ramp_t	Color ramp parameter for gradient paints provided to the driver
converted_length	vg_lite_uint32_t	Converted internal color ramp length.
converted_ramp[VLC_MAX_COLOR_RAMP_STOPS+2]	vg_lite_color_ramp_t	Converted internal color ramp.
pre-multiplied	vg_lite_uint8_t	If this value is set to 1, the color value of color_ramp will be multiplied by the alpha value of color_ramp.
spread_mode	vg_lite_radial_gradient_spreadmode_t	The spread mode that is applied to the pixels out of the image after transformed.

9.2.5 vg_lite_linear_gradient_parameter Structure

This structure defines radial direction for a linear gradient. *(from April 2022)*

Line0 connects point (X0, Y0) to point (X1, Y1) and represents the radial direction of the linear gradient.

Line1 is a line perpendicular to line0 which passes through point (X0, Y0).

Line2 is a line perpendicular to line0 which passes through point (X1, Y1)

The linear gradient paint is applied at the intersection of the path fill area and the plane, starting from line 1 and ending at line 2.

Used in structure: vg_lite_ext_linear_gradient.
Used in functions: vg_lite_set_linear_grad.

vg_lite_linear_gradient_parameter_t Members	Type	Description
X0	vg_lite_float_t	X origin of linear gradient radial direction.
Y0	vg_lite_float_t	Y origin of linear gradient radial direction.
X1	vg_lite_float_t	X end point of linear gradient radial direction.
Y1	vg_lite_float_t	Y end point of linear gradient radial direction.

9.2.6 vg_lite_matrix_t Structure

Defined under Matrix Structures. LINK to vg_lite_matrix_t structure.

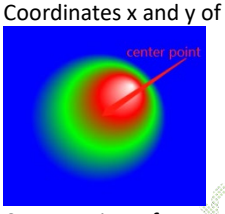
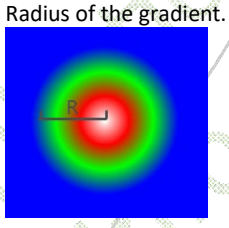
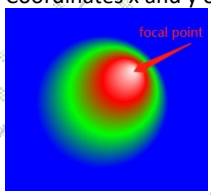
9.2.7 vg_lite_path_t Structure

Defined under Vector Path Structures. LINK to vg_lite_path_t structure.

9.2.8 vg_lite_radial_gradient_parameter_t Structure

This structure defines the gradient radius and the X and Y coordinates for the center and focal points of the gradient.
(from November 2020, requires GC355 or GC555 hardware)

Used in radial gradient structure: vg_lite_radial_gradient_t.

vg_lite_radial_gradient_parameter_t Members	Type	Description *(member order updated from August 2021)*
cx	vg_lite_float_t	Coordinates x and y of the gradient color center point.  Center point refers to the center of the gradient color.
cy	vg_lite_float_t	
r	vg_lite_float_t	Radius of the gradient. 
fx	vg_lite_float_t	Coordinates x and y of the gradient color focal point  Focal point refers to the center of the gradient color.
fy	vg_lite_float_t	

9.2.9 vg_lite_radial_gradient_t Structure

This structure defines the application of the radial gradient to fill a path. *(from November 2020, requires GC355 or GC555 hardware).*

Used in radial gradient functions: vg_lite_draw_grad, vg_lite_set_radial_grad, vg_lite_update_radial_grad, vg_lite_get_radial_grad, vg_lite_clear_radial_grad

vg_lite_radial_gradient_t Members	Type	Description
count	vg_lite_uint32_t	Count of colors, up to 256
matrix	vg_lite_matrix_t	Structure which specifies the transform matrix for the gradient
image	vg_lite_buffer_t	Structure which specifies the image for rendering as a gradient pattern
radial_grad	vg_lite_radial_gradient_parameter_t	Structure which specifies the location of the gradient's center point (cx, cy), focal point (fx, fy) and radius(r)
ramp_length	vg_lite_uint32_t	Color ramp parameters for gradient paints provided to the driver
color_ramp[VLC_MAX_COLOR_RAMP_STOPS]	vg_lite_color_ramp_t	Structure which specifies the color ramp.
converted_length	vg_lite_uint32_t	Converted internal color ramp
converted_ramp[VLC_MAX_COLOR_RAMP_STOPS+2]	vg_lite_color_ramp_t	Structure which specifies the Internal color ramp.
pre_multiplied	vg_lite_uint32_t	If this value is set to 1, the color value of color_ramp will be multiplied by the alpha value of color_ramp.
spread_mode	vg_lite_radial_gradient_spreadmode_t	Enum which specifies the tiling mode that is applied to the pixels out of the image after transformation.

9.3 Draw Functions

vg_lite_draw

Description:
Performs a hardware accelerated 2D vector draw operation.
The size of the tessellation buffer can be specified, and that size will be aligned to the minimum required alignment of the hardware by the kernel. If you make the tessellation buffer smaller, less memory will be allocated, but a path might be sent down to the hardware multiple times because the hardware will walk the target with the provided tessellation window size, so performance might be lower. It is good practice to set the tessellation buffer size to the most common path size. For example, if all you do is render up to 24-pt fonts, you can set the tessellation buffer to be 24x24.

Notes:

- All the color formats available in the vg_lite_buffer_format_t enum are supported as the destination buffer for the draw function.
- Strokes are not supported by the hardware. They need to be converted to paths before being used in the draw API.

Syntax:

```
vg_lite_error_t vg_lite_draw (
    vg_lite_buffer_t            *target,
    vg_lite_path_t              *path,
    vg_lite_fill_t              fill_rule,
    vg_lite_matrix_t            *matrix,
    vg_lite_blend_t             blend,
    vg_lite_color_t             color
);
```

Parameters:

***target**	Pointer to the vg_lite_buffer_t structure for the destination buffer. All color formats available in the vg_lite_buffer_format_t enum are valid destination formats for the draw function.
***path**	Pointer to the vg_lite_path_t structure containing path data which describes the path to draw. Refer to the section on Vector Path Data Opcodes in this document for opcode detail.
fill_rule	Specifies the vg_lite_fill_t enum value for the fill rule for the path.
***matrix**	Pointer to a vg_lite_matrix_t structure that defines the **affine** transformation matrix of the path. If matrix is NULL, an identity matrix is assumed. **Note:** non-affine transformation is not supported for vg_lite_draw, so a perspective transformation matrix has no effect on path.
blend	Select one of the hardware supported blend modes in the vg_lite_blend_t enum to be applied to each drawn pixel. If no blending is required, set this value to VG_LITE_BLEND_NONE (0).
color	The color applied to each pixel drawn by the path.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_draw_grad

Description:
This function is used to fill a path with a gradient according to specified fill rules. The specified path will be transformed according to the selected matrix and filled with the gradient.

Syntax:

```
vg_lite_error_t vg_lite_draw_grad (
    vg_lite_buffer_t            *target,
    vg_lite_path_t              *path,
    vg_lite_fill_t              fill_rule,
    vg_lite_matrix_t            *matrix,
    vg_lite_linear_gradient_t   *grad,
    vg_lite_blend_t             blend
);
```

Parameters:

***target**	Pointer to the vg_lite_buffer_t structure containing data describing the target path.
***path**	Pointer to the vg_lite_path_t structure containing path data which describes the path to draw for the linear gradient. Refer to the section on Vector Path Data Opcodes in this document for opcode detail.
fill_rule	Specifies the vg_lite_fill_t enum value for the fill rule for the path.
***matrix**	Pointer to a vg_lite_matrix_t structure that defines the 3x3 transformation matrix of the path. If matrix is NULL, an identity matrix is assumed which is usually a bad idea since the path can be anything.
***grad**	Pointer to the vg_lite_linear_gradient_t structure which contains the values to be used to fill the path.
blend	Specified the blend mode in the vg_lite_blend_t enum to be applied to each drawn pixel. If no blending is required, set this value to VG_LITE_BLEND_NONE (0).

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_draw_radial_grad

Description:
This function is used to fill a path with a radial gradient according to specified fill rules. The specified path will be transformed according to the selected matrix and filled with the gradient.

Application can use VGLite API vg_lite_query_feature (gcFEATURE_BIT_VG_RADIAL_GRADIENT) to determine HW support for radial gradient.

Syntax:

```
vg_lite_error_t vg_lite_draw_radial_grad (
    vg_lite_buffer_t            *target,
    vg_lite_path_t              *path,
    vg_lite_fill_t              fill_rule,
    vg_lite_matrix_t            *path_matrix,
    vg_lite_radial_gradient_t   *grad,
    vg_lite_color_t             paint_color,
    vg_lite_blend_t             blend,
    vg_lite_filter_t            filter
);
```

Parameters:

***target**	Pointer to the vg_lite_buffer_t structure containing data describing the target path.
***path**	Pointer to the vg_lite_path_t structure containing path data which describes the path to draw for the linear gradient. Refer to the section on Vector Path Data Opcodes in this document for opcode detail.
fill_rule	Specifies the vg_lite_fill_t enum value for the fill rule for the path.
***path_matrix**	Pointer to a vg_lite_matrix_t structure that defines the 3x3 transformation matrix of the path. If matrix is NULL, an identity matrix is assumed which is usually a bad idea since the path can be anything.
***grad**	Pointer to the vg_lite_radial_gradient_t structure which contains the values to be used to fill the path. Note: grad->image.image_mode does not support VG_LITE_MULTIPLY_IMAGE_MODE.
paint_color	Specifies the paint color vg_lite_color_t RGBA value to be applied by VG_LITE_RADIAL_GRADIENT_SPREAD_FILL, which set by function vg_lite_set_radial_grad. When pixels are out of the image after transformation, this paint_color is applied to them. See also enum vg_lite_radial_gradient_spreadmode_t.
blend	Specifies the blend mode in the vg_lite_blend_t enum to be applied to each drawn pixel. If no blending is required, set this value to VG_LITE_BLEND_NONE (0).
filter	Specified the filter mode vg_lite_filter_t enum value to be applied to each drawn pixel. If no filtering is required, set this value to VG_LITE_BLEND_POINT (0).

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_draw_pattern

Description:

This function fills a path with an image pattern. The path will be transformed according to the specified matrix and filled with the transformed image pattern.

Syntax:

```
vg_lite_error_t vg_lite_draw_pattern (
    vg_lite_buffer_t            *target,
    vg_lite_path_t              *path,
    vg_lite_fill_t              fill_rule,
    vg_lite_matrix_t            *path_matrix,
    vg_lite_buffer_t            *pattern_image,
    vg_lite_matrix_t            *pattern_matrix,
    vg_lite_blend_t             blend,
    vg_lite_pattern_mode_t      pattern_mode,
    vg_lite_color_t             pattern_color,
    vg_lite_color_t             color,
    vg_lite_filter_t            filter
);
```

Parameters:

***target**	Pointer to the vg_lite_buffer_t structure that defines the path to draw.
***path**	Pointer to the vg_lite_path_t structure containing path data which describes the path to draw. Refer to the section on Vector Path Data Opcodes in this document for opcode detail.
fill_rule	Specifies the vg_lite_fill_t enum value for the fill rule for the path.
***path_matrix**	Pointer to a vg_lite_matrix_t structure that defines the 3x3 transformation matrix of the path. If matrix is NULL, an identity matrix is assumed, which is usually a bad idea since the path can be anything.
***pattern_image**	Pointer to the vg_lite_buffer_t structure that describes the image pattern.
***pattern_matrix**	Pointer to a vg_lite_matrix_t structure that defines the 3x3 transformation matrix of the source pixels into the target. If matrix is NULL, an identity matrix is assumed, meaning the source will be copied directly onto the target at 0,0 location.
blend	Specifies one of the vg_lite_blend_t enum values for hardware supported blend modes to be applied to each drawn pixel in the image. If no blending is required, set this value to VG_LITE_BLEND_NONE (0).
pattern_mode	Specifies the vg_lite_pattern_mode_t value which defines how the region outside the image pattern is to be filled.
pattern_color	Specifies a 32bpp ARGB color (vg_lite_color_t) to be applied to the fill outside the image pattern area when the pattern_mode value is VG_LITE_PATTERN_COLOR. *(from Dec 2019, type now vg_lite_color_t, previously was uint32_t)*

color	Specifies a 32bpp ARGB color (vg_lite_color_t) to be applied as a mix color. If non-zero, the mix color value gets multiplied with each source pixel before blending happens. If a mix color is not needed, set the color parameter to 0. *(from May 2023)* NOTE: this parameter has no effect if the pattern image vg_lite_buffer_t structure image_mode is set as VG_LITE_ZERO or VG_LITE_NORMAL_IMAGE_MODE.
filter	Specifies the filter type. All formats available in the vg_lite_filter_t enum are valid formats for this function. A value of zero (0) indicates VG_LITE_FILTER_POINT.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

9.4 Linear Gradient Initialization and Control Functions

This part of the API performs linear gradient operations.

A color gradient (color progression, color ramp) is a smooth transition between a set of colors (color stops) that is done along a line (linear, or axial color gradient) or radially, along concentric circles (radial color gradient). The color transition is done by linear interpolation between two consecutive color stops.

Note: VGLite supports linear color gradients for GCNanoLiteV and GCNanoUltraV. Both linear and radial gradients are supported with GC355 and GC555.

vg_lite_init_grad

Description:
This function initializes the internal buffer for the linear gradient object with default settings for rendering.

Syntax:

```
vg_lite_error_t vg_lite_init_grad (
    vg_lite_linear_gradient_t       *grad,
);
```

Parameters:

***grad**	Pointer to the vg_lite_linear_gradient_t structure which defines the gradient to be initialized. Default values are used.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_clear_grad

Description:
This function is used to clear the values of a linear gradient object and free the image buffer's memory.

Syntax:

```
vg_lite_error_t vg_lite_clear_grad (
    vg_lite_linear_gradient_t   *grad,
);
```

Parameters:

***grad**	Pointer to the vg_lite_linear_gradient_t structure which is to be cleared.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_set_grad

Description:
This function is used to set values for the members of the vg_lite_linear_gradient_t structure.

Note: **vg_lite_set_grad** API adopts the following rules to set the default gradient colors if the input parameters are incomplete or invalid.

a. If no valid stops have been specified (e.g., due to an empty input array, out-of-range, or out-of-order stops), a stop at 0 with (R, G, B, α) color (0.0, 0.0, 0.0, 1.0) (opaque black) and a stop at 1 with color (1.0, 1.0, 1.0, 1.0) (opaque white) are implicitly defined.
b. If at least one valid stop has been specified, but none has been defined with an offset of 0, an implicit stop is added with an offset of 0 and the same color as the first user-defined stop.
c. If at least one valid stop has been specified, but none has been defined with an offset of 1, an implicit stop is added with an offset of 1 and the same color as the last user-defined stop.

Syntax:

```
vg_lite_error_t vg_lite_set_grad (
    vg_lite_linear_gradient_t   *grad,
    vg_lite_uint32_t            count,
    vg_lite_uint32_t            *colors,
    vg_lite_uint32_t            *stops
);
```

Parameters:

***grad**	Pointer to the vg_lite_linear_gradient_t structure to be set.
count	This is the count of the colors in the linear gradient. The maximum color stop count is defined by VLC_MAX_GRAD which is 16.
***colors**	Specifies the color array for the gradient stops. The color is in ARGB8888 format with alpha in the upper byte.
***stops**	Pointer to the gradient stop offset.

Returns:
Always returns VG_LITE_SUCCESS.

vg_lite_get_grad_matrix

Description:
This function is used to get a pointer to the gradient object's transformation matrix. This allows an application to manipulate the matrix to facilitate correct rendering of the gradient path.

Syntax:

```
vg_lite_error_t vg_lite_get_grad_matrix (
    vg_lite_linear_gradient_t   *grad,
);
```

Parameters:

***grad**	Pointer to the vg_lite_linear_gradient_t structure which contains the matrix to be retrieved.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_update_grad

Description:
This function is used to update or generate values for an image object that is going to be rendered. The vg_lite_linear_gradient_t object has an image buffer which is used to render the gradient pattern. The image buffer will be created or updated with the corresponding grad parameters.

Syntax:

```
vg_lite_error_t vg_lite_update_grad (
    vg_lite_linear_gradient_t   *grad,
);
```

Parameters:

***grad**	Pointer to the vg_lite_linear_gradient_t structure which contains the update values to be used for the object to be rendered.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

9.5 Linear Gradient Extended Functions

The following functions are available only with IP which includes hardware support for extended linear gradient capabilities, such as GC355 and GC555. These functions are not available with GCNanoLiteV, GCNanoUltraV or GCNanoV.

Applications can use VGLite API vg_lite_query_feature(gcFEATURE_BIT_VG_LINEAR_GRADIENT_EXT) to determine HW support for linear gradient.

vg_lite_set_linear_grad

Description:

This function is used to set the values which define the linear gradient. *(from April 2022)*

Syntax:

```
vg_lite_error_t vg_lite_set_linear_grad (
    vg_lite_ext_linear_gradient_t          *grad,
    vg_lite_uint32_t                       count,
    vg_lite_color_ramp_t                   *color_ramp,
    vg_lite_linear_gradient_parameter_t    grad_param,
    vg_lite_radial_gradient_spreadmode_t   spread_mode,
    vg_lite_uint8_t                        pre_mult
);
```

Parameters:

***grad**	Pointer to the vg_lite_ext_linear_gradient_t structure which is to be set.
count	Count of the colors in the gradient. The maxmum color stop count is defined by MAX_COLOR_RAMP_STOPS, which is currently 256.
***color_ramp**	This is the stop for the linear gradient. The number of parameters is 5 and gives the offset and color of the stop. Each stop is defined by a floating-point offset value and four floating-point values containing the sRGBA color and alpha value associated with each stop, in the form of a non-premultiplied (R, G, B, alpha) quad. And the range of all parameters in it is [0,1].
grad_param	Gradient parameters as specified in structure vg_lite_linear_gradient_parameter_t.
spread_mode	The tiling mode applied to the pixels out of the paint after transformation. Uses the same spread mode enumeration types as radial gradient. See vg_lite_radial_gradient_spreadmode_t enum.
pre_mult	This parameter controls whether color and alpha values are interpolated in premultiplied or non-premultiplied form.

Returns:

Returns VG_LITE_INVALID_ARGUMENTS to indicate the parameters are wrong.

vg_lite_get_linear_grad_matrix

Description:
This function returns a pointer to an extended linear gradient object's matrix. *(from March 2023)*

Syntax:

```
vg_lite_matrix_t* vg_lite_get_linear_grad_matrix (
    vg_lite_ext_linear_gradient_t   *grad,
);
```

Parameters:

***grad**	Pointer to the vg_lite_ext_linear_gradient_t structure

Returns:
Returns a pointer to a vg_lite_matrix_t for the specified extended linear gradient.

vg_lite_draw_linear_grad

Description:
This function is used to fill a path with a linear gradient according to specified fill rules. The specified path will be transformed according to the selected matrix and filled with the transformed linear gradient. *(from April 2022)*

Syntax:

```
vg_lite_error_t vg_lite_draw_linear_grad (
    vg_lite_buffer_t                *target,
    vg_lite_path_t                  *path,
    vg_lite_fill_t                  fill_rule,
    vg_lite_matrix_t                *path_matrix,
    vg_lite_ext_linear_gradient_t   *grad,
    vg_lite_color_t                 paint_color,
    vg_lite_blend_t                 blend,
    vg_lite_filter_t                filter
);
```

Parameters:

***target**	Pointer to the vg_lite_buffer_t structure containing data describing the target path.
***path**	Pointer to the vg_lite_path_t structure containing path data which describes the path to draw for the linear gradient. Refer to the section on Vector Path Data Opcodes in this document for opcode detail.
fill_rule	Specifies the vg_lite_fill_t enum value for the fill rule for the path.
***path_matrix**	Pointer to a vg_lite_matrix_t structure that defines the 3x3 transformation matrix of the path. If matrix is NULL, an identity matrix is assumed which is usually a bad idea since the path can be anything.
***grad**	Pointer to the vg_lite_ext_linear_gradient_t structure which contains the values to be used to fill the path. Note: grad->image.image_mode does not support VG_LITE_MULTIPLY_IMAGE_MODE.
paint_color	Specifies the paint color vg_lite_color_t RGBA value to be applied by VG_LITE_RADIAL_GRADIENT_SPREAD_FILL, which set by function vg_lite_set_linear_grad. When pixels are out of the image after transformation, this paint_color is applied to them. See also enum vg_lite_radial_gradient_spreadmode_t.
blend	Specifies the blend mode in the vg_lite_blend_t enum to be applied to each drawn pixel. If no blending is required, set this value to VG_LITE_BLEND_NONE (0).
filter	Specified the filter mode vg_lite_filter_t enum value to be applied to each drawn pixel. If no filtering is required, set this value to VG_LITE_BLEND_POINT (0).

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_update_linear_grad

Description:
This function is used to update or generate the corresponding image object to render. *(from April 2022)*

The vg_lite_ext_linear_gradient_t object has an image buffer which is used to render the linear gradient paint. The image buffer will be created/updated according to the specified grad parameters.

Syntax:

```
vg_lite_error_t vg_lite_update_linear_grad (
    vg_lite_ext_linear_gradient_t   *grad,
);
```

Parameters:

***grad**	Pointer to the vg_lite_ext_linear_gradient_t structure which is to be updated or created.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_clear_linear_grad

Description:
This function is used to clear the linear gradient object. This will reset the grad members and free the image buffer's memory. *(from April 2022)*

Syntax:

```
vg_lite_error_t vg_lite_clear_linear_grad (
    vg_lite_ext_linear_gradient_t   *grad,
);
```

Parameters:

***grad**	Pointer to the vg_lite_ext_linear_gradient_t structure which is to be cleared.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

9.6 Radial Gradient Functions

The following functions are available only with IP which supports radial gradients, such as GC355 and GC555. These functions are not available with GCNanoLiteV, or GCNanoUltraV or GCNanoV. Note: There is no init function required for radial gradients. Buffer initialization is done through the vg_lite_update_radial_grad function. *(from Nov 2020, requires GC355 or GC555 hardware)*

vg_lite_set_radial_grad

Description:
This function is used to set the values for the radial linear gradient definition. *(from November 2020, requires GC355 or GC555 hardware)*

Syntax:

```
vg_lite_error_t vg_lite_set_radial_grad (
    vg_lite_radial_gradient_t              *grad,
    vg_lite_uint32_t                       count,
    vg_lite_color_ramp_t                   *color_ramp,
    vg_lite_radial_gradient_parameter_t    grad_param,
    vg_lite_radial_gradient_spreadmode_t   spread_mode,
    vg_lite_uint8_t                        pre_mult
);
```

Parameters:

***grad**	Pointer to the vg_lite_radial_gradient_t structure for the radial gradient which will be set
count	This is the count of the color stops in the gradient. The maximum color stop count is defined by MAX_COLOR_RAMP_STOPS, which is currently 256.
***color_ramp**	Pointer to the vg_lite_color_ramp_t structure which defines the stops for the radial gradient. The five parameters provide the offset and color for the stop. Each stop is defined by a set of floating-point values which specify the offset and the sRGBA color and alpha values. Color channel values are in the form of a non-premultiplied (R, G, B, alpha) quad. All parameters are in the range of [0,1]. The red, green, blue, alpha value of [0, 1] is mapped to an 8-bit pixel value [0, 255].
grad_param	The radial gradient parameters are supplied as a vector of 5 floats in the order {cx,cy,fx,fy,r}. Parameters(cx,cy) specify the center point, (fx,fy) the focal point and r the radius. See struct vg_lite_radial_gradient_parameter_t.
spread_mode	The tiling mode that is applied to pixels out of the paint after transformation. See enum vg_lite_radial_gradient_spreadmode_t.
pre_mult	Controls whether color and alpha values are interpolated in premultiplied or non-premultiplied form. If this value is set to 1, the color value of vgColorRamp will be multipled by the alpha value of vgColorRamp.

Returns:
Returns VG_LITE_INVALID_ARGUMENTS to indicate the parameters are wrong.

vg_lite_update_radial_grad

Description:
This function is used to update or generate values for an image object that is going to be rendered. The vg_lite_radial_gradient_t object has an image buffer which is used to render the gradient pattern. The image buffer will be created or updated with the corresponding gradient parameters. *(from November 2020, requires GC355 or GC555 hardware)*

Syntax:

```
vg_lite_error_t vg_lite_update_radial_grad (
    vg_lite_radial_gradient_t    *grad,
);
```

Parameters:

***grad**	Pointer to the vg_lite_radial_gradient_t structure which contains the update values to be used for the object to be rendered.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_get_radial_grad_matrix

Description:
This function is used to get a pointer to the radial gradient object's transformation matrix. This allows an application to manipulate the matrix to facilitate correct rendering of the gradient path. *(from Nov 2020, requires GC355 or GC555 hardware).*

Syntax:

```
vg_lite_error_t vg_lite_get_radial_grad_matrix (
    vg_lite_radial_gradient_t    *grad,
);
```

Parameters:

***grad**	Pointer to the vg_lite_radial_gradient_t structure which contains the matrix to be retrieved.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_clear_radial_grad

Description:
This function is used to clear the values of a radial gradient object and free the image buffer's memory. *(from Nov 2020, requires GC355 or GC555 hardware)*

Syntax:.

```
vg_lite_error_t vg_lite_clear_radial_grad (
    vg_lite_radial_gradient_t   *grad,
);
```

Parameters:

***grad**	Pointer to the vg_lite_radial_gradient_t structure which is to be cleared.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

10 Stroke Operations

This part of the API performs stroke operations. *(from March 2022)*

10.1 Stroke Enumerations

10.1.1 vg_lite_cap_style_t Enumeration

Defines the style of cap at the end of a stroke. *(from March 2022)*

Used in structure: vg_lite_stroke_t.
Used in function: vg_lite_set_stroke.

vg_lite_cap_style_t String Values	Description
VG_LITE_CAP_BUTT	The Butt end cap style terminates each segment with a line perpendicular to the tangent at each endpoint.
VG_LITE_CAP_ROUND	The Round end cap style appends a semicircle with a diameter equal to the line width centered around each endpoint.
VG_LITE_CAP_SQUARE	The Square end cap style appends a rectangle with two sides of length equal to the line width perpendicular to the tangent, and two sides of length equal to half the line width parallel to the tangent, at each endpoint.

10.1.2 vg_lite_path_type_t Enumeration

Defines the type of draw path. *(from March 2022)*

Used in structure: vg_lite_path_t, vg_lite_stroke_t.
Used in function: vg_lite_set_path_type.

vg_lite_path_type_t String Values	Description
VG_LITE_DRAW_FILL_PATH	Draw path is fill.
VG_LITE_DRAW_STROKE_PATH	Draw path is stroke.
VG_LITE_DRAW_FILL_STROKE_PATH	Draw path is both fill and stroke.

10.1.3 vg_lite_join_style_t Enumeration

Defines the type of styles available for line joints. *(from March 2022)*

Used in structure: vg_lite_stroke_t.
Used in function: vg_lite_set_stroke.

vg_lite_join_style_t String Values	Description
VG_LITE_JOIN_MITER	The Miter join style appends a trapezoid with one vertex at the intersection point of the two original lines, two adjacent vertices at the outer endpoints of the two “fattened” lines and a fourth vertex at the extrapolated intersection point of the outer perimeters of the two “fattened” lines.
VG_LITE_JOIN_ROUND	The Round join style appends a wedge-shaped portion of a circle, centered at the intersection point of the two original lines, having a radius equal to half the line width.
VG_LITE_JOIN_BEVEL	The Bevel join style appends a triangle with two vertices at the outer endpoints of the two "fattened” lines and a third vertex at the intersection point of the two original lines.

10.2 Stroke Structures

10.2.1 vg_lite_path_t Structure

Defined under Vector Path Structures. LINK to vg_lite_path_t structure.
(additional members added for stroke from March 2022)

10.2.2 vg_lite_path_list_t Structure

The structure vg_lite_path_list_ptr points to a vg_lite_path_list structure which provides divided path data according to MOVE/MOVE_REL. *(from Aug 2023)*

Used (vg_lite_path_list_ptr) in structures: vg_lite_stroke_t.

vg_lite_path_list_t Members	Type	Description
path_points	vg_lite_path_point_ptr	Pointer to path point coordinates
path_end	vg_lite_path_point_ptr	Pointer to the last path point
point_count	vg_lite_uint32_t	Number of points in the path
next	vg_lite_path_list_ptr	Pointer to the next path list
closed	vg_lite_uint8_t	Flag to indicate if the path is closed

10.2.3 vg_lite_path_point_t Structure

The structure vg_lite_path_point_ptr points to a vg_lite_path_point structure which provides path detail. *(from March 2022)*

Used (vg_lite_path_point_ptr) in structures: vg_lite_path_point_t, vg_lite_stroke_conversion. vg_lite_sub_path_t.

vg_lite_path_point_t Members	Type	Description
x	vg_lite_float_t	X coordinate
y	vg_lite_float_t	Y coordinate
flatten_flag	vg_lite_uint8_t	Flatten flag for flattened path
curve_type	vg_lite_uint8_t	Curve type for the stroke path
tangentX	vg_lite_float_t	X tangent (Note: #define centerX tangent)
tangentY	vg_lite_float_t	Y tangent (Note: #define centerX tangent)
length	vg_lite_float_t	Line length
prev	vg_lite_path_point_ptr	Pointer to the previous point node

10.2.4 vg_lite_stroke_t Structure

The structure provides stroke parameters and pointers to temp storage for a stroke sub path. Refer to function vg_lite_set_stroke parameter descriptions for additional description for some members. *(from March 2022)*

Used in structure: vg_lite_path_t.

vg_lite_stroke_t Members	Type	Description
cap_style	vg_lite_cap_style_t	Stroke cap style
join_style	vg_lite_join_style_t	Stroke joint style
line_width	vg_lite_float_t	Stroke line width
miter_limit	vg_lite_float_t	Stroke miter limit
*dash_pattern	vg_lite_float_t	Pointer to stroke dash pattern
pattern_count	vg_lite_uint32_t	Number of dash pattern repetitions
dash_phase	vg_lite_float_t	Stroke dash phrase
dash_length	vg_lite_float_t	Stroke dash initial length
dash_index	vg_lite_uint32_t	Stroke dash initial index
half_width	vg_lite_float_t	Half line width
pattern_length	vg_lite_float_t	Total length of stroke dash patterns.
miter_square	vg_lite_float_t	For fast checking
path_points	vg_lite_path_point_ptr	Temp storage for stroke sub path
path_end	vg_lite_path_point_ptr	Temp storage for stroke sub path
point_count	unint32_t	Temp storage for stroke sub path
left_point	vg_lite_path_point_ptr	Temp storage for stroke sub path
right_pont	vg_lite_path_point_ptr	Temp storage for stroke sub path
stroke_points	vg_lite_path_point_ptr	Temp storage for stroke sub path
stroke_end	vg_lite_path_point_ptr	Temp storage for stroke sub path
stroke_count	vg_lite_uint32_t	Temp storage for stroke sub path
path_list_divide	vg_lite_path_list_ptr	Divide stroke path according to move or move_rel for avoiding implicit closure. *(from Aug 2023)*
cur_list	vg_lite_path_list_ptr	Pointer to current divided path data. *(from Aug 2023)*
add_end	vg_lite_uint8_t	Flag that adds end_path in driver *(from Aug 2023)*
dash_reset	vg_lite_uint8_t	Flag to indicate dash reset *(from Aug 2023)*
stroke_paths	vg_lite_sub_path_ptr	Pointer to stoke path linked list
last_stroke	vg_lite_sub_path_ptr	Pointer to the last stroke path in the list
swing_handling	vg_lite_uint32_t	Flag for swing handling
swing_deltax	vg_lite_float_t	Stroke swing delta x
swing_deltay	vg_lite_float_t	Stroke swing delta y
swing_start	vg_lite_path_point_ptr	Stroke swing start point

vg_lite_stroke_t Members	Type	Description
swing_stroke	vg_lite_path_point_ptr	Swing stroke path point
swing_length	vg_lite_float_t	Stroke swing length
swing_centlen	vg_lite_float_t	Stroke swing center point length
swing_count	vg_lite_uint32_t	Stroke swing point count
need_swing	vg_lite_uint8_t	Flag to indicate if swing is needed
swing_ccw	vg_lite_uint8_t	Flag to indicate swing winding direction CCW or not
stroke_length	vg_lite_float_t	Total length of all point lengths
stroke_size	vg_lite_uint32_t	Number of bytes in the stroke path data
fattened	vg_lite_uint8_t	the stroke line is fat line.
closed	vg_lite_uint8_t	Stoke path is closed or not

10.2.5 vg_lite_sub_path_t Structure

The structure vg_lite_sub_path_ptr points to a vg_lite_sub_path structure which provides sub path detail and a pointer to the next sub path. *(from March 2022)*

Used in structure: vg_lite_stroke_conversion.

vg_lite_path_point_t Members	Type	Description
next	vg_lite_sub_path_ptr	Pointer to the next sub path
point_count	vg_lite_uint32_t	Number of points in the sub path
point_list	vg_lite_path_point_ptr	Pointer to the point list.
end_point	vg_lite_path_point_ptr	Pointer to the last point.
closed	vg_lite_uint8_t	Indicates whether or not the path is closed.
length	vg_lite_float_t	Length of the sub path.

10.3 Stroke Functions

All return vg_lite_error_t status.

vg_lite_set_path_type

Description:
This function sets the path type. *(from March 2022)*

Syntax:

```
vg_lite_error_t vg_lite_set_path_type (
    vg_lite_path_t              *path,
    vg_lite_path_type_t         path_type
);
```

Parameters:

***path**	Pointer to the vg_lite_path_t structure that describes the path.
path_type	Pointer to a vg_lite_path_type_t structure that describes the path type.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_set_stroke

Description:
This function uses input parameters to set stroke attributes. *(from March 2022)*

Syntax:

```
vg_lite_error_t vg_lite_set_stroke (
    vg_lite_path_t              *path,
    vg_lite_cap_style_t         cap_style,
    vg_lite_join_style_t        join_style,
    vg_lite_float_t             line_width,
    vg_lite_float_t             miter_limit,
    vg_lite_float_t             *dash_pattern,
    vg_lite_uint32_t            pattern_count,
    vg_lite_float_t             dash_phase,
    vg_lite_color_t             color
);
```

Parameters:

***path**	Pointer to the vg_lite_path_t structure that describes the path.
cap_style	The end cap style defined by the vg_lite_cap_style_t enum.
join_style	The line join style defined by the vg_lite_join_style_t enum.
line_width	The line width of the stroke path. Note: A line width less than 0.5 prevents stroking from taking place.
miter_limit	When stroking using the Miter stroke vg_lite_join_style_t, the miter length (i.e., the length between the intersection points of the inner and outer perimeters of the two “fattened” lines) is compared to the product of the user-set miter limit and the line width. If the miter length exceeds this product, the Miter join is not drawn and a Bevel join is substituted. Note: Miter limit values less than 1 are silently clamped to 1.
***dash_pattern**	Pointer to a dash pattern which consists of a sequence of lengths of alternating "on" and "off" dash segments. The first value of the dash array defines the length, in user coordinates, of the first "on" dash segment. The second value defines the length of the following "off" segment. Each subsequent pair of values defines one "on" and one "off" segment. Note: If the dash pattern has an odd number of elements, the final element is ignored.
pattern_count	The count of dash on/off segments.
dash_phase	Defines the starting point in the dash pattern that is associated with the start of the first segment of the path. For example, if the dash pattern is [10 20 30 40] and the dash phase is 35, the path will be stroked with an "on" segment of length 25 (skipping the first "on" segment of length 10, the following "off" segment of length 20, and the first 5 units of the next "on" segment), followed by an "off" segment of length 40. The pattern will then repeat from the beginning, with an “on” segment of length 10, an "off" segment of length 20, an "on" segment of length 30.
color	The stroke color.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

vg_lite_update_stroke

Description:
This function uses the path and stroke attributes as specified with function vg_lite_set_stroke to update the stroke path's parameters and generate stroke path data. *(from March 2022)*

Syntax:

```
vg_lite_error_t vg_lite_update_stroke (
    vg_lite_path_t              *path,
);
```

Parameters:

***path**	Pointer to the vg_lite_path_t structure that describes the path.

Returns:
Returns VG_LITE_SUCCESS if the function is successful. See vg_lite_error_t enum for other return codes.

11 Deprecated and Renamed APIs

The following functions are deprecated and are either obsolete or replaced by a more efficient implementation. Their use is discouraged and will produce unpredictable behaviors.

The names of some functions, enums and structures were modified during code refinements in 2022Q3. If the parameters did not change, the deprecated syntax detail is not provided below. Changes to enums and structs are not mentioned here, instead refer to the item itself.

Deprecated or Renamed API	Recommended Replacement API	Source file	Date Deprecated
vg_lite_perspective	n/a	vg_lite.h	August 2022
vg_lite_set_dither	vg_lite_enable_dither vg_lite_disable_dither	vg_lite.h	August 2022
vg_lite_append_path	vg_lite_path_append	vg_lite.h	Sept 2022
vg_lite_path_calc_length	vg_lite_get_path_length	vg_lite.h	Sept 2022
vg_lite_set_image_global_alpha	vg_lite_set_source_global_alpha	vg_lite.h	Sept 2022
vg_lite_dest_global_alpha	vg_lite_set_dest_global_alpha	vg_lite.h	Sept 2022
vg_lite_mem_avail	vg_lite_get_mem_size	vg_lite.h	Sept 2022
vg_lite_enable_premultiply	n/a	vg_lite.h	Dec 2022
vg_lite_disable_premultiply	n/a	vg_lite.h	Dec 2022
vg_lite_set_premultiply	n/a	vg_lite.h	Aug 2023
vg_lite_radial_gradient_spreadmode_t enum	vg_lite_gradient_spreadmode_t enum	vg_lite.h	March 2023
API Name Refinement	***(no change to parameters)***		
vg_lite_buffer_upload	vg_lite_upload_buffer_	vg_lite.h	Sept 2022
vg_lite_*mask*	*most vg_lite_*mask_layer*	vg_lite.h	Sept 2022
vg_lite_*_grad	vg_lite_*_gradient *(parameters unchanged)*	vg_lite.h	Sept 2022
vg_lite_*_radial_grad*	vg_lite_*_rad_grad*	vg_lite.h	Sept 2022
vg_lite_buffer_image_mode_t	vg_lite_image_mode_t	vg_lite.h	Sept 2022
vg_lite_transparency_mode_t	vg_lite_transparency_t	vg_lite.h	Sept 2022
vg_lite_set_update_stroke	vg_lite_update_stroke	vg_lite.h	Sept 2022
vg_lite_set_draw_path_type	vg_lite_set_path_type	vg_lite.h	Sept 2022

11.1 Deprecated vg_lite Syntax

Syntax for deprecated functions is provided below for reference. Note: this list does not include items renamed during code refinement of Sept 2022.

vg_lite_perspective *(deprecated)*

Syntax:

```
void vg_lite_perspective (
    vg_lite_float_t             px,
    vg_lite_float_t             py,
    vg_lite_matrix_t            *matrix
);
```

vg_lite_set_dither *(deprecated)*

Syntax:

```
vg_lite_error_t vg_lite_set_dither (
    int                 enable
);
```

vg_lite_enable_premultiply *(deprecated)*

Syntax:

```
vg_lite_error_t vg_lite_enable_premultiply (
    void
);
```

vg_lite_disable_premultiply *(deprecated)*

Syntax:

```
vg_lite_error_t vg_lite_disable_premultiply (
    void
);
```

vg_lite_set_premultiply *(deprecated)*

Syntax:

```
vg_lite_error_t vg_lite_set_premultiply (
    vg_lite_uint8_t             src_premult,
    vg_lite_uint8_t             dst_premult,
);
```

12 VGLite API Programming Examples

12.1 vg_lite_clear Example

The **Conformance/samples/clear/clear.c** test program demonstrates the basic flow of a VGLite application program and the usage of the vg_lite_clear API. First, the program initializes the VGLite API with:

```
error = vg_lite_init(0, 0);
```

Note that as the tessellation buffer width and height are defined as (0, 0) in this vg_lite_init API call, this program cannot use the path rendering vg_lite_draw APIs. Only clear and blit APIs can be used in this program.

After initialization, the program allocates a 256x256 render buffer with a format of VG_LITE_RGB565.

```
buffer.width  = 256;
buffer.height = 256;
buffer.format = VG_LITE_RGB565;
error = vg_lite_allocate(&buffer);
fb = &buffer;
```

It clears the entire render buffer with blue color first with the vg_lite_clear API.

```
error = vg_lite_clear(fb, NULL, 0xFFFF0000);
```

Then it clears a 64x64 square at the position (64, 64) relative to the top-left origin of the render buffer.

```
vg_lite_rectangle_t rect = { 64, 64, 64, 64 };
error = vg_lite_clear(fb, &rect, 0xFF0000FF);
```

After that, it calls vg_lite_finish to flush the commands to Vivante Vector Graphics hardware and then frees up the allocated render buffer. Finally, it calls vg_lite_close to destroy the VGLite context which is initialized by vg_lite_init.

```
vg_lite_finish();
vg_lite_free(&buffer);
vg_lite_close();
```

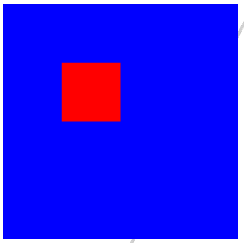


Figure 1. Example using vg_lite_clear

12.2 vg_lite_blit Example

The **Conformance/samples/ui/main.c** test program demonstrates the usage of the vg_lite_blit API. It clears a 320x480 render buffer with blue background color first, then it blits six 256x256 icon images to six different positions in the render buffer with a blit matrix for each icon. The blit matrix scales the original icon image to a proper size and translates the scaled icon to the right position in the render buffer. The vg_lite_blit API call is set as VG_LITE_BLEND_SRC_OVER so the icon image pixels with alpha value 0xFF cover the background blue color.

```
vg_lite_blit(fb, &icons[icon_id], &icon_matrix, VG_LITE_BLEND_SRC_OVER, 0,
                 VG_LITE_FILTER_POINT);
```

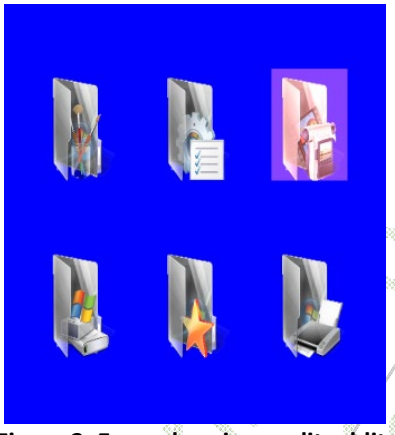


Figure 2. Example using vg_lite_blit.

12.3 vg_lite_draw Example

The **Conformance/samples/ui/main.c** test program also demonstrates the usage of the vg_lite_draw API with which draws a highlighted rectangle on the top-right icon in above image. The program defines a path (path_data[]) for a 10x10 square bounding box, and it sets up a proper "highlight_matrix" to translate/scale the 10x10 square to cover the top-right icon. The vg_lite_draw API call uses blend parameter VG_LITE_BLEND_SRC_OVER and blend color 0x22444488 (alpha value 0x22) to draw a semi-transparent rectangle on the top-right icon.

```
static char path_data[] = {
    2,  0,  0,          // moveto    0,  0
    4, 10,  0,          // lineto   10,  0
    4, 10, 10,          // lineto   10, 10
    4, 0,  10,          // lineto    0, 10
    0,                  // end
};
static vg_lite_path_t path = {
    {-10, -10, 10, 10}, // bounding box left, top, right, bottom
    VG_LITE_HIGH,       // quality
    VG_LITE_S8,         // -128 to 127 coordinate range
    {0},                // uploaded
    sizeof(path_data),  // path length
    path_data,          // path data
    1                   // path changed
};

error = vg_lite_draw(fb, &path, VG_LITE_FILL_EVEN_ODD, &highlight_matrix,
                VG_LITE_BLEND_SRC_OVER, 0x22444488);
```

After the vg_lite_draw call, vg_lite_clear_path(&path) is called to free and reset the path data.

12.4 vg_lite_draw_gradient Example

The **Conformance/samples/linearGrad/linearGrad.c** test program demonstrates the usage of the vg_lite_draw_grad API. It defines 5 colors (black, red, green, blue, white) in ramps[] and 5 stops in stops[] which are used for gradient color transition. It calls the following to setup the color gradient image.

```
vg_lite_uint32_t ramps[] = {0xff000000, 0xffff0000, 0xff00ff00, 0xff0000ff, 0xffffffff};
vg_lite_uint32_t stops[] = {0, 66, 122, 200, 255};
vg_lite_set_grad(&grad, 5, ramps, stops);
vg_lite_update_grad(&grad);
```

Note that the “colors” parameter (ramps[]) in vg_lite_set_grad API must be in ARGB8888 format with alpha at the higher byte.

It also sets up the gradient transformation matrix “matGrad” with a proper scale factor and 30 degree rotation.

```
matGrad = vg_lite_get_grad_matrix(&grad);
vg_lite_identity(matGrad);
vg_lite_rotate(30.0f, matGrad);
```

Then it calls:

```
vg_lite_draw_grad(fb, &path, VG_LITE_FILL_EVEN_ODD, &matPath, &grad, VG_LITE_BLEND_NONE);
```

with a ploygon path and color gradient image/matrix so that it generates the rendering effect as illustrated in the image below.

After the draw gradient API, it calls:

```
vg_lite_finish();
vg_lite_clear_grad(&grad);
```

to flush the VGLite commands and clean up the gradient image buffer.

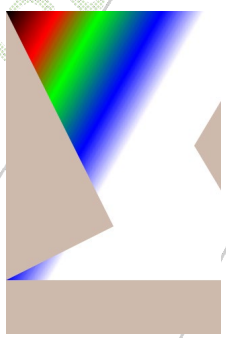


Figure 3. Example using vg_lite_draw_gradient

12.5 vg_lite_draw_pattern Example

The **Conformance/samples/patternFill/patternFill.c** test program demonstrates the usage of the vg_lite_draw_pattern API. It defines a vg_lite_path_t path for a convex polygon shape as shown below, and loads an image file "landscape.raw" with which to fill the polygon interior area.

It also defines two matrices, one named “matrix” for the image, another named “matPath” for the “path”. The image matrix rotates the image 33 degrees clockwise based on the image center.

```
vg_lite_identity(&matrix);
vg_lite_translate(fb_width / 2.0f, fb_height / 4.0f, &matrix);
vg_lite_rotate(33.0f, &matrix);
vg_lite_scale(0.4f, 0.4f, &matrix);
vg_lite_translate(fb_width / -2.0f, fb_height / -4.0f, &matrix);

vg_lite_identity(&matPath);
vg_lite_translate(fb_width / 2.0f, fb_height / 4.0f, &matPath);
vg_lite_scale(10, 10, &matPath);
```

Then it calls vg_lite_draw_pattern API two times with different parameters to draw the polygon twice.

```
error = vg_lite_draw_pattern(fb, &path, VG_LITE_FILL_EVEN_ODD, &matPath, &image, &matrix,
                  VG_LITE_BLEND_NONE, VG_LITE_PATTERN_COLOR, 0xffaabbcc, 0,
                  VG_LITE_FILTER_POINT);

error = vg_lite_draw_pattern(fb, &path, VG_LITE_FILL_EVEN_ODD, &matPath, &image, &matrix,
                  VG_LITE_BLEND_NONE, VG_LITE_PATTERN_PAD, 0xffaabbcc, 0,
                  VG_LITE_FILTER_POINT);
```

With the vg_lite_pattern_mode_t setting of VG_LITE_PATTERN_COLOR, the polygon area outside the pattern image of the upper polygon is filled with color 0xffaabbcc. With the vg_lite_pattern_mode_t setting of VG_LITE_PATTERN_PAD, the polygon area outside the pattern image of the lower polygon is filled with the border pixel color of the pattern image.

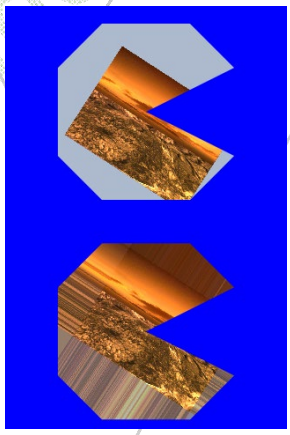


Figure 4. Example using vg_lite_draw_pattern

12.6 Vector-based Font Rendering Example

The **`Conformance/samples/glyphs2/glyphs2.c`** test program demonstrates vector-based font rendering with the `vg_lite_draw` API, which is capable of drawing quadratic curves and cubic curves based on end point and control point coordinates in the path data. The font path data can be generated by using a third-party font engine that can produce VGLite path data directly, or by using VeriSilicon's VGLite tools to convert other formats of font data, such as SVG, etc., to VGLite path data. Here is an example of path data for the character "~" (ASCII code 126):

```
float ascii_font_126[] =
{
    2,15.984375,20.273438,
    4,16.296875,20.476563,
    6,15.781250,21.351563,14.921875,21.992188,
    6,13.953125,22.710938,13.046875,22.710938,
    6,12.375000,22.710938,10.898438,22.203125,
    6,9.421875,21.695313,8.656250,21.695313,
    6,7.937500,21.695313,7.375000,22.117188,
    6,7.015625,22.382813,6.421875,23.117188,
    4,6.109375,22.914063,
    6,7.593750,20.664063,9.453125,20.664063,
    6,10.156250,20.664063,11.492188,21.140625,
    6,12.828125,21.617188,13.531250,21.617188,
    6,14.921875,21.617188,15.984375,20.273438,
    0
};
```

The first integer in each line is the path opcode, followed by the coordinates for each opcode. As listed in Section 8.4, opcode (2, x, y) moves the current position to (x, y); opcode (4, x, y) draws a line from the current position to (x, y); opcode (6, cx, cy, x, y) draws a quadratic curve from the current position to the given end point (x, y) using the specified control point (cx, cy).

The program calls:

```
error = vg_lite_init(256, 256);
```

to initialize VGLite with a 256x256 path tessellation buffer, then allocates a 320x320 render buffer with the format VG_LITE_RGBA8888. The size of the tessellation buffer is big enough to cover the font character bounding box.

The program renders the path for each character in the string "Hello,\nVerisilicon!" in a loop with calls to:

```
/* Draw the path using the matrix.*/
error = vg_lite_draw(fb, &path, VG_LITE_FILL_EVEN_ODD, &matrix, VG_LITE_BLEND_NONE,
                     0xFF0000FF);
```

The character's vector path is rendered without blending (VG_LITE_BLEND_NONE). The path interior is filled with the color red (0xFF0000FF).

Figure 5. Example using Vector Based Font Rendering

To demonstrate the smooth curve of vector-based path rendering with any scale factor, the program renders a single character “H” with a scaled size of 8X using following API calls.

```
vg_lite_identity(&matrix);
vg_lite_translate(startX, startY, &matrix);
vg_lite_scale(8.0, 8.0, &matrix);
error = vg_lite_draw(fb, &path, VG_LITE_FILL_EVEN_ODD, &matrix, VG_LITE_BLEND_NONE,
                     0xFF0000FF);
```

The following image example shows the resulting vector path rendering of character “H”.

Figure 6. Example with Vector Based Font Rendering Upscaled 8X.

13 VGLite API Version 2.0 to 3.0 Migration Guide

The VGLite API version 3.0 is not fully compatible with VGLite API version 2.0. VGLite API version 3.0 includes some new API functions for the new features in the latest VG GPU like GC555. Some VGLite API version 2.0 function interfaces are changed in API version 3.0. So the existing VGLite API version 2.0 applications need to be modified to compile and run properly with the VGLite API version 3.0 driver. This chapter provides guidance for migrating VGLite API version 2.0 applications to VGLite API version 3.0.

13.1 VGLite API Name Changes in API version 3.0

Some original VGLite API names are changed in API version 3.0 for API naming consistency. In the VGLite API version 3.0 header file vg_lite.h, a set of API name macros are defined for the equivalent API names between API version 3.0 and API version 2.0, so it is not necessary to modify the VGLite API function names in API version 2.0 applications for the application to compile and run with API version 3.0 driver.

The list of equivalent VGLite API functions between API version 3.0 and API version 2.0 is shown below. These API functions' parameters are the same between API version 3.0 and API version 2.0.

```
/* API name defines for backward compatibility to VGLite 2.0 APIs */
#define vg_lite_buffer_upload                   vg_lite_upload_buffer
#define vg_lite_path_append                     vg_lite_append_path
#define vg_lite_path_calc_length                vg_lite_get_path_length
#define vg_lite_set_ts_buffer                   vg_lite_set_tess_buffer
#define vg_lite_set_draw_path_type              vg_lite_set_path_type
#define vg_lite_create_mask_layer               vg_lite_create_masklayer
#define vg_lite_fill_mask_layer                 vg_lite_fill_masklayer
#define vg_lite_blend_mask_layer                vg_lite_blend_masklayer
#define vg_lite_generate_mask_layer_by_path     vg_lite_render_masklayer
#define vg_lite_set_mask_layer                  vg_lite_set_masklayer
#define vg_lite_destroy_mask_layer              vg_lite_destroy_masklayer
#define vg_lite_enable_mask                     vg_lite_enable_masklayer
#define vg_lite_enable_color_transformation     vg_lite_enable_color_transform
#define vg_lite_set_color_transformation        vg_lite_set_color_transform
#define vg_lite_set_image_global_alpha          vg_lite_source_global_alpha
#define vg_lite_set_dest_global_alpha           vg_lite_dest_global_alpha
#define vg_lite_clear_rad_grad                  vg_lite_clear_radial_grad
#define vg_lite_update_rad_grad                 vg_lite_update_radial_grad
#define vg_lite_get_rad_grad_matrix             vg_lite_get_radial_grad_matrix
#define vg_lite_set_rad_grad                    vg_lite_set_radial_grad
#define vg_lite_draw_linear_gradient            vg_lite_draw_linear_grad
#define vg_lite_draw_radial_gradient            vg_lite_draw_radial_grad
#define vg_lite_draw_gradient                   vg_lite_draw_grad
#define vg_lite_mem_avail                       vg_lite_get_mem_size
#define vg_lite_set_update_stroke               vg_lite_update_stroke
```

The list of equivalent VGLite API structures and enumerations is shown below:

```
#define vg_lite_buffer_image_mode_t             vg_lite_image_mode_t
#define vg_lite_draw_path_type_t                vg_lite_path_type_t
#define vg_lite_linear_gradient_ext_t           vg_lite_ext_linear_gradient_t
#define vg_lite_buffer_transparency_mode_t      vg_lite_transparency_t
```

13.2 vg_lite_set_scissor API Interface Change

The VGLite API **vg_lite_set_scissor**() function name is not changed in API version 3.0, but the API parameters are defined differently in API version 3.0.

In VGLite API version 3.0, **vg_lite_set_scissor**() function is defined as:

```
/* Set and enable a scissor rectangle for render target. */
vg_lite_error_t vg_lite_set_scissor(vg_lite_int32_t x, vg_lite_int32_t y,
                vg_lite_int32_t right, vg_lite_int32_t bottom);
```

In VGLite API version 2.0, **vg_lite_set_scissor**() function is defined as:

```
vg_lite_error_t vg_lite_set_scissor(int32_t x, int32_t y, int32_t width, int32_t
                height);
```

So **vg_lite_set_scissor**() API parameters “width” and “height” in VGLite API version 2.0 application need to be changed to “right” X coordinate value and “bottom” Y coordinate value.

13.3 vg_lite_map API Interface Change

The VGLite API **vg_lite_map**() function name is not changed in API version 3.0, but the API parameters are defined differently in API version 3.0.

In VGLite API version 3.0, **vg_lite_map**() function is defined as:

```
/* Map a buffer into hardware accessible address space. */
vg_lite_error_t vg_lite_map(vg_lite_buffer_t *buffer, vg_lite_map_flag_t flag, int32_t
                fd);
```

In VGLite API version 2.0, **vg_lite_map**() function is defined as:

```
vg_lite_error_t vg_lite_map(vg_lite_buffer_t *buffer);
```

So **vg_lite_map**() in VGLite API version 3.0 API requires two extra parameters “flag” and “fd”, which can simply be set as **vg_lite_map**(buffer, 0, 0) in applications.

13.4 vg_lite_enable_scissor / vg_lite_disable_scissor API

VGLite API **vg_lite_enable_scissor**() and **vg_lite_disable_scissor()** function are valid only for **vg_lite_scissor_rects()** API, they have no effect for **vg_lite_set_scissor()** in VGLite API version 3.0.

Although the behavior of **vg_lite_enable_scissor**() and **vg_lite_disable_scissor()** is changed in VGLite API version 3.0, there is no need to change these functions in VGLite API version 2.0 applications to work with VGLite API version 3.0 driver.

13.5 vg_lite_draw_pattern API Interface Change

The VGLite API **vg_lite_draw_pattern**() function name is not changed in API version 3.0, but the API parameters are defined differently in API version 3.0.

In VGLite API version 3.0, **vg_lite_draw_pattern**() function is defined as:

```
/* Draw a path that is filled by a transformed image pattern. */
vg_lite_error_t vg_lite_draw_pattern(vg_lite_buffer_t *target,
                                     vg_lite_path_t *path,
```

```
                                    vg_lite_fill_t fill_rule,
                                    vg_lite_matrix_t *path_matrix,
                                    vg_lite_buffer_t *pattern_image,
                                    vg_lite_matrix_t *pattern_matrix,
                                    vg_lite_blend_t blend,
                                    vg_lite_pattern_mode_t pattern_mode,
                                    vg_lite_color_t  pattern_color,
                                    vg_lite_color_t  color,
                                    vg_lite_filter_t filter);
```

Compared to the VGLite API version 2.0 **vg_lite_draw_pattern**() function, "color" is a new additional parameter. It specifies a 32bpp ARGB color (vg_lite_color_t) to be applied as a mix color. If non-zero, the mix color value gets multiplied with each source pixel before blending happens. If a mix color is not needed, set the color parameter to 0.

13.6 [New] vg_lite_copy_image in VGLite API Version 3.0

The new API **vg_lite_copy_image**() is added in VGLite API version 3.0 to support the OpenVG **vgCopyImage** API, which performs a pixel rectangle copy without pixel transformation, blending, filtering operations.

13.7 vg_lite_set_dither API is Deprecated in API Version 3.0

The original API version 2.0 function **vg_lite_set_dither(int enable)** API is removed from API version 3.0, it is replaced with two new APIs for dither enable/disable:

```
/* Enable dither function. Dither is OFF by default. */
vg_lite_error_t vg_lite_enable_dither();

/* Disable dither function. Dither is OFF by default. */
vg_lite_error_t vg_lite_disable_dither();
```

Thus, the **vg_lite_set_dither**(enable) function in VGLite API version 2.0 application needs to be replaced with **vg_lite_enable_dither**() or **vg_lite_disable_dither**() to work with VGLite API version 3.0 driver.

13.8 Deprecated VGLite API version 2.0 Functions

The VGLite API **vg_lite_perspective(), vg_lite_enable_premultiply(), vg_lite_disable_premultiply()** functions are removed from API version 3.0. These API functions need to be deleted from a VGLite API version 2.0 application to work with VGLite API version 3.0 driver.

In VGLite API version 3.0, the color premultiply setting is defined by the **vg_lite_blend_t** enumeration to replace the original **vg_lite_enable_premultiply()** and **vg_lite_disable_premultiply()** APIs.

- VG_LITE_BLEND_* enumeration values in **vg_lite_blend_t** define non-premultiplied blending modes.
- OPEVG_BLEND_* enumeration values in **vg_lite_blend_t** define premultiplied Porter-Duff blending modes.

So VGLite API version 3.0 application can set different blending modes to get the desired premultiplied/non-premultiplied blending result.

Appendix 1. GC265 0x420 Alignment Requirements

Table 5. GC265 0x420 Image Buffer Alignment Table

Image Format	Bits per pixel	Source Tile Mode	Start Address Alignment Requirement in Bytes	Stride Alignment Requirement in Bytes	Buffer Height Alignment Requirement	Supported for Destination
VG_LITE_INDEX1	1	linear	8B	1B	1	
	1	tile	8B	1B	4	
VG_LITE_INDEX2	2	linear	8B	1B	1	
	2	tile	8B	1B	4	
VG_LITE_INDEX4	4	linear	8B	1B	1	
	4	tile	8B	2B	4	
VG_LITE_INDEX8	8	linear	8B	1B	1	
	8	tile	8B	4B	4	
VG_LITE_A4	4	linear	8B	1B	1	
	4	tile	8B	2B	4	
VG_LITE_A8	8	linear	8B	1B	1	Yes
	8	tile	8B	4B	4	Yes
VG_LITE_L8	8	linear	8B	1B	1	Yes
	8	tile	8B	4B	4	Yes
VG_LITE_ARGB2222	8	linear	8B	1B	1	Yes
	8	tile	8B	4B	4	Yes
VG_LITE_RGB565	16	linear	8B	2B	1	Yes
	16	tile	8B	8B	4	Yes
VG_LITE_ARGB1555	16	linear	8B	2B	1	Yes
	16	tile	8B	8B	4	Yes
VG_LITE_ARGB4444	16	linear	8B	2B	1	Yes
	16	tile	8B	8B	4	Yes
VG_LITE_ARGB8888	32	linear	16B	4B	1	Yes
	32	tile	16B	16B	4	Yes
VG_LITE_XRGB8888	32	linear	16B	4B	1	Yes
	32	tile	16B	16B	4	Yes
VG_LITE_ARGB8565	24	linear	8B	3B*	1	Yes
	24	tile	8B	12B*	4	Yes
VG_LITE_RGB888	24	linear	8B	3B*	1	Yes
	24	tile	8B	12B*	4	Yes
VG_LITE_ARGB8565_PLANAR	24	linear	A: 8B RGB: 8B	A: 1B RGB: 2B	1	Yes
	24	tile	A: 8B RGB: 8B	A: 4B RGB: 8B	4	Yes
VG_LITE_YUY2/UYVY	16	linear	8B	4B	1	
	16	tile	8B	8B	4	
VG_LITE_NV12	12	linear	Y: 8B UV: 8B	Y: 2B UV: 2B	1	
	12	tile	Y: 8B UV: 8B	Y: 8B UV: 8B	4	
VG_LITE_YV12	12	linear	Y: 8B U: 8B V: 8B	Y: 2B U: 1B V: 1B	1	

Image Format	Bits per pixel	Source Tile Mode	Start Address Alignment Requirement in Bytes	Stride Alignment Requirement in Bytes	Buffer Height Alignment Requirement	Supported for Destination
	12	tile	Y: 8B U: 8B V: 8B	Y: 8B U: 4B V: 4B	4	
VG_LITE_NV16	16	linear	Y: 8B UV: 8B	Y: 2B UV: 2B	1	
VG_LITE_YV16	16	linear	Y: 8B U: 8B V: 8B	Y: 2B U: 1B V: 1B	1	
VG_LITE_YV24	24	linear	Y: 8B U: 8B V: 8B	Y: 1B U: 1B V: 1B	1	
	24	tile	Y: 8B U: 8B V: 8B	Y: 4B U: 4B V: 4B	4	
VG_LITE_ETC2	8	tile	8B	4B	4	

Table 6. GC265 0x420 Destination Buffer Alignment Table

Target Format	Bits per pixel	Target Tile Mode	VG Tile Mode	Start Address Alignment Requirement in Bytes	Stride Alignment Requirement in Bytes	Buffer Height Alignment Requirement
VG_LITE_A8	8	linear	linear	4B	1B	1
			tile	64B	64B	4
		tile	linear	64B	64B	4
			tile	64B	16B	4
VG_LITE_L8	8	linear	linear	4B	1B	1
			tile	64B	64B	4
		tile	linear	64B	64B	4
			tile	64B	16B	4
VG_LITE_ARGB2222	8	linear	linear	4B	1B	1
			tile	64B	64B	4
		tile	linear	64B	64B	4
			tile	64B	16B	4
VG_LITE_RGB565	16	linear	linear	4B	2B	1
			tile	64B	64B	4
		tile	linear	64B	64B	4
			tile	64B	16B	4
VG_LITE_ARGB1555	16	linear	linear	4B	2B	1
			tile	64B	64B	4
		tile	linear	64B	64B	4
			tile	64B	16B	4
VG_LITE_ARGB4444	16	linear	linear	4B	2B	1
			tile	64B	64B	4
		tile	linear	64B	64B	4
			tile	64B	16B	4
VG_LITE_ARGB8888	32	linear	linear	4B	4B	1
			tile	64B	64B	4
		tile	linear	64B	64B	4
			tile	64B	16B	4
VG_LITE_XRGB8888	32	linear	linear	4B	4B	1
			tile	64B	64B	4
		tile	linear	64B	64B	4
			tile	64B	16B	4
VG_LITE_ARGB8565	24	linear	linear	64B	3B*	1
			tile	64B	48B*	4
		tile	linear	64B	48B*	4
			tile	64B	12B*	4
VG_LITE_ARGB8565_PLANAR	24	linear	linear	64B	A: 1B RGB: 2B	1
			tile	64B	48B*	4
		tile	linear	64B	48B*	4
			tile	64B	A: 4B RGB: 8B	4
VG_LITE_RGB888	24	linear	linear	64B	3B*	1
			tile	64B	48B*	4
		tile	linear	64B	48B*	4
			tile	64B	12B*	4

Document Revision History

This section describes top level differences between document revisions:

Document Revision	Date	Compatible SW Release	Notes
2.19	2025-04-30	VGLite from March 2025 (1096789)	Section 5.6: Added description of multi-plane YUV buffer map support for vg_lite_map API. Section 7.1.8 vg_lite_param_type_t Enumeration: updated description; added VG_LITE_HARDWARE_RUNNING_TIME, VG_LITE_SRC_BUF_ALIGNED_CHECK, VG_LITE_DST_BUF_ALIGNED_CHECK string values. Section 7.4 Blit/Draw Extended Functions - vg_lite_get_parameter: updated type parameter description; vg_lite_enable_scissor: updated function description with supported APIs and supported IPs; vg_lite_disable_scissor/vg_lite_scissor_rects/vg_lite_set_scissor: removed GC355/GC555 hardware limitation.
2.18	2025-03-06	VGLite from March 2025 (1096789)	Section 5.4.8 vg_lite_image_mode_t Enumeration: added missing descriptions for members. Section 5.6 Pixel Buffer Functions vg_lite_allocate: added note for DEC compressed buffer parameters requirement. Section 8.3 Vector Path Functions vg_lite_init_path: added note on “data” parameter storage. Section 10.2.2 vg_lite_path_list_t Structure: added missing descriptions for members. Section 10.2.4 vg_lite_stroke_t Structure: added missing descriptions for members.
2.17	2024-11-25	VGLite from November 2024	Added Appendix 1 for GC265 0x420 Alignment Requirements. Section 10.3: Added a note for stroke line width less than 0.5 in vg_lite_set_stroke line_width parameter. Section 8.4: Updated Table 4: Vector Path Data Opcodes.
2.16	2024-08-13	VGLite from August 2024	Added Section 5.4.2 Image Buffer Alignment Requirement. Added Section 5.4.3 Destination Buffer Alignment Requirement. Section 9.3: Updated vg_lite_draw_pattern API description. The pattern_image parameter can now support VG_LITE_MULTIPLY_IMAGE_MODE.
2.15	2024-07-30	VGLite from June 2024	Section 7.4: Updated vg_lite_set_scissor API to include information on enable/disable scissor function. Section 4.1: Added new API vg_lite_frame_delimiter.
2.14	2024-06-12	VGLite from March 2024 (860817)	Section 9.1.5 vg_lite_gradient_spreadmode_t: Added more description details and descriptions for string values.. Section 9.1.7 vg_lite_radial_gradient_spreadmode_t: Added descriptions for string values.
2.13	2024-03-19	VGLite from March 2024 (860817)	Section 7.2.8 vg_lite_point4_t Structure: removed “Used in blit function: vg_lite_get_transform_matrix.” Added Section 7.2.9 vg_lite_float_point Structure. Added Section 7.2.10 vg_lite_float_point4_t Structure. Section 7.3 BLIT Functions: vg_lite_get_transform_matrix: changed src and dst parameter type to vg_lite_float_point4_t; updated descriptions for src and dst parameter types and function descriptions to float points.
2.12	2024-02-22	VGLite from February 2024 (844751)	Section 7.1.1: Update vg_lite_blend_t enum *_BLEND_DARKEN, *_BLEND_LIGHTEN with correct equations. Add a note to indicate LVGL blend modes are supported by all VG cores.

Document Revision	Date	Compatible SW Release	Notes
2.11	2023-12-18	VGLite from December 2023 (*750376*)	Section 7.1.1: Update vg_lite_blend_t enum table with correct equations. Section 7.4: Change vg_lite_get_parameter API parameters.
2.10	2023-12-07	VGLite from December 2023 (*750376*)	Section 4.1: Add new API vg_lite_set_memory_pool. Section 7. vg_lite_scissor_rects: add new parameter “target.” Added missing Returns: to functions.
2.09	2023-11-13	VGLite from November 2023 (749346)	Added Section 13.6: [New] vg_lite_copy_image in VGLite API Version 3.0. Added Section 13.8 Deprecated VGLite API version 2.0 Functions.
2.08	2023-10-17	VGLite from August 2023 *(718431)*	Section 4.1: Expand text for vg_lite_init tess buffer size. Section 5.5.1: correct hyperlink for image_mode. Sections 7.3 and 9.3: vg_lite_blit(), vg_lite_blit_rect()and vg_lite_draw_pattern(), add note for vg_buffer_t setting requirement for image_mode. Section 13: added for migration notes.
2.07	2023-09-05	VGLite from August 2023 *(718431)*	Section 5.4.1: add/refine OpenVG Image format detail; refine column for start and stride alignment. Section 5.5.1: struct vg_lite_buffer_t, add scissor_layer and premultiplied. Section 7.2: add vg_lite_param_type enum. Section 7.4 and 11: deprecate vg_lite_set_premultiply. Section 7.4: add vg_lite_get_parameter function. Refine vg_lite_*_scissor functions. Section 10.2.3: add struct vg_lite_path_list_t. Section 10.2.4: update order of vg_lite_stroke_t struct members; add dash_reset, path_list_divide, cur_list, add_end, change type and location for dash_reset.
2.06	2023-06-29	VGLite from June 2023 *(690729)*	Section 9.2.5: update vg_lite_linear_gradient_parameter structure text to indicate perpendicular instead of parallel lines.
2.05	2023-05-22	VGLite from May 2023 *(677579)*	Section 5.4.8 and 5.5.1: Add enum vg_lite_paint_type. Section 9.5: Add color parameter to vg_lite_draw pattern.
2.04	2023-04-13	VGLite from March 2023 *(640414)*	Section 9.5: Correct typo in function name vg_lite_get_linear_grad_matrix and parameter description.
2.03	2023-03-17	VGLite from March 2023 *(640414)*	Section 1.1: Refine text in second paragraph. Section 7.1.1: Add more blend modes, correct formula for SUBTRACT. Sections 7.4, 7.5, 7.6, 9.5, 9.6: Heading text refined. Section 8.2.2: add description for member add_end. Various: remove numerical enum values. Miscellaneous refinements.

Document Revision	Date	Compatible SW Release	Notes
2.02	2023-03-15	VGLite from March 2023 *(639839)*	Various: inline history removed. Section 1.1: Add note re V4 driver. Section 1.4: revised. Section 3.3.1: feature enum values added. Section 5.1: paragraph 1 revised. Section 5.4.1: column added for alignment notes. Section 5.4.1.1: revised Section 5.4.3: add enum vg_lite_map_flag. Section 5.4: add enums compress_mode, index_endian Section 5.5: add struct vg_lite_fc_buffer_t Section 5.5.1: add members to structure, update transparency_mode member name. Section 5.6: add function vg_lite_flush_mapped_buffer; add parameters to function vg_lite_map flag, fd. Section 6.3: return now vg_lite_error_t, was void Section 7.1.3: Add gaussian value for vg_lite_filter_t enum Section 7.3: correct syntax typo for blend in vg_lite_blit_rect. Sections 7.4, 7.5, 7.6: arrangement modified Section 7.6: change global_alpha parameter alpha_value to vg_lite_unit8_t, was 32. Section 8.2.2: add member add_end Section 8.3: vg_lite_get_path, vg_lite_append_path *cmd now *opcode, *data now data. Section 9.5: add function vg_lite_get_linear_grad_matrix. Miscellaneous refinements. Section 9.1.5: Add 2 values for enum vg_lite_pattern_mode.
2.01	2022-12-09	VGLite from December 2022 *(603862)*	Section 7.4.1: Change parameters for VGLite API vg_lite_error_t vg_lite_set_premultiply (vg_lite_uint8_t ~~src_enable~~ src_premult, vg_lite_uint8_t ~~dst_enable~~ dst_premult~~, vg_lite_uint8_t lerp_enable~~) Section 9.3, 9.4: add note to disable VG_LITE_MULTIPLY_IMAGE_MODE support in the following three APIs. −vg_lite_draw_pattern −vg_lite_draw_linear_gradient −vg_lite_draw_radial_gradient Section 11: deprecate vg_lite_enable_premultiply, vg_lite_disable_premultiply. Some wording refinements. Update document template and related format refinements.
2.00	2022-10-03	VGLite from September 2022 *(560435)*	Update for API version 3.0 to add support for GC555 functionality. Various: add vg_lite prefix to more common parameter types Section 5.6: Add function vg_lite_set_gamma. Section 7.4.1: Add function vg_lite_set_premulitply. Section 7.4.3: Added vg_lite_enable_color_transform, vg_lite_disable_color_transform, vg_lite_set_color_transform. Section 8.2.2, 9.2.4 and 9.2.9: undate variable names for structures. Section 11: List renamed functions.

Document Revision	Date	Compatible SW Release	Notes
1.20	2022-08-15	VGLite from August 2022	Section 3.1.1: add enum values gcFEATURE_BIT_VG_MASK, gcFEATURE_BIT_VG_QUALITY_8X, gcFEATURE_BIT_VG_MIRROR. Section 5.6: add vg_lite_enable_dither, vg_lite_disable_dither; deprecate vg_lite_set_dither. Section 7.1.5: add enum vg_lite_mask_operation_t. Section 7.1.6: add enum vg_lite_orientation_t. Section 7.3: revise reference to color_key compatible cores. Section 7.4.2: add vg_lite_scissor_rects; deprecate vg_lite_set_scissor. Section 7.4.3: add vg_lite_enable_mask, vg_lite_create_mask_layer, vg_lite_fill_mask_layer, vg_lite_blend_mask_layer, vg_lite_set_mask_layer, vg_lite_generate_mask_layer_by_path, vg_lite_destroy_mask_layer. Section 7.4.5: add vg_lite_set_mirror., Section 8.1.2: update description for VG_LITE_UPPER value. Section 11 added. Template updated. Miscellaneous Refinements.
1.19	2022-04-18	VGLite from April 2022	Format refinements. Template changed, no change to text content.
1.19	2022-04-15	VGLite from April 2022	Various: adjustments for GC555. Section 2.2, Table 1: add VG_LITE_FP32 enum value for vg_lite_format_t. Section 3.1: Add enum values gcFFEATURE_BIT_LINEAR_GRADIENT_EXT, gcFEATURE_BIT_VG_COLOR_KEY. Section 6.1: add vg_lite_pixel_matrix_t[20] Section 6.2.2 : add struct vg_lite_pixel_channel_enable_t. Section 6.3: add function vg_lite_set_pixel_matrix_t. Section 7.2: Add struct vg_lite_color_key_t, vg_lite_color_key4_t Section 7.3: Add function vg_lite_set_color_key. Section 7.4: Add vg_lite_disable_scissor, vg_lite_set_scissor. Section 9.2: Add struct vg_lite_linear_gradient_parameter, vg_lite_linear_gradient_ext. Section 9.4: vg_lite_set_linear_grad, vg_lite_update_linear_grad, vg_lite_clear_linear_grad, vg_lite_draw_linear_gradient. Miscellaneous refinements
1.18	2022-04-01	VGLite from March 2022	Section 8.2: Added members to vg_lite_path_t. Section 10: Added for stroke. Section 10.1: Add enums vg_lite_cap_style, vg_lite_draw_path_type, vg_lite_join_style. Section 10.2: Add struct vg_lite_path_point, vg_lite_sub_path, vg_lite_stroke_conversion, vg_lite_path. Section 10.3: Add function vg_lit_set_stroke, vg_lite_update_stroke, vg_lite_set_draw_path_type.
1.17	2022-03-03	VGLite from March 2022	Legal Notices and throughout: update branding and layout to include VeriSilicon. Sections 1.1 and 1.4: update references for newer hardware Section 3.1: Add vg_lite_feature enum values update gcFEATURE_BIT_VG_YUV_OUTPUT, gcFEATURE_BIT_VG_DOUBLE_IMAGE. Section 5.1: Remove last paragraph Section 5.4 Alignment notes expand text, Table 2 Rename table, refine column heading text, for dest remove last sentence of first bullet. Section 7.1: Refine vg_lite_blend_t Section 7.3: Add vg_lite_blit2.
1.16	2022-01-20	VGLite from January 2022	Section 8.3: vg_lite_path_append now returns vg_lite_error_t.

Document Revision	Date	Compatible SW Release	Notes
1.15	2021-12-01	VGLite from December 2021	Section 4.1: Add functions vg_lite_set_command_buffer and vg_lite_set_ts_buffer.
1.14	2021-10-25	VGLite from October 2021	Section 3.1: Add vg_lite_feature enum values gcFEATURE_BIT_VG_DITHER Section 5.4: Add 24bit color formats. Section 5.6: Add vg_lite_set_dither.
1.13	2021-10-19	VGLite from August 2021	Section 5.4: Add three values for enum type vg_lite_buffer_format_t: NV12_TILED, ANV12_TILED AYUY2_TILED.
1.12	2021-08-17	VGLite from August 2021	Section 2.2: Add two values for enum type vg_lite_error_t. Section 5.6: Function vg_lite_set_CLUT description modified. Section 9.2: Adjust vg_lite_radial_gradient_parameter_t the member order of structure.
1.11	2021-05-25	VGLite from June 2021	Section 3.1: Add vg_lite_feature enum values gcFEATURE_BIT_VG_GLOBAL_ALPHA and gcFEATURE_BIT_VG_IM_FASTCLEAR Section 7.1: Add enum vg_lite_global_alpha_t. Section 7.3: Add APIs vg_lite_set_image_global_alpha, vg_lite_set_dest_global_alpha
1.10	2021-04-07	VGLite from April 2021	Section 9.2: Add images for members of vg_lite_radial_gradient_parameter_t. Miscellaneous refinements.
1.09	2021-03-10	VGLite from March 2021	Section 7.2: Add structs vg_lite_point, vg_lite_point4_t. Section 7.3: Add API vg_lite_get_transform_matrix.
1.08	2021-03-03	VGLite from February 2021	Section 8.3: Add vg_lite_init_arc_path API. Section 8.4: Add Opcodes for arc paths.
1.07	2021-02-05	VGLite as previous (November 2020)	Legal Notices: Distribution Level changed B to A. Section 1, 1.1 and 1.4 and various: Add GCNanoUltraV
1.06	2020-12-15	VGLite as previous (November 2020)	Section 5.1 and 5.4, Alignment Notes: Refine alignment discussion.
1.04	2020-08-11	VGLite as of pre-release (August 2020) and GCNanoLiteV HW from V1311p	Section 7: –Add note re premulitply support in hardware Section 7.4: section created - Add vg_lite_enable/disable_premulitply - Modify vg_lite_enable/disable_scissor Change parameters for vg_lite_set_scissor function to use width, height instead of right, bottom.
1.03	2020-05-21	VGLite as of pre-release 2.0.17 (June 2020) and GCNanoLiteV HW from V1311p	Correct sw version. Legal Notices: Distribution Level changed A to B. Section 3.3: added vg_lite_mem_avail Section 8.1: deprecate vg_lite_quality_t.VG_LITE_UPPER value
1.02	2020-04-20	VGLite as of pre-release sw (April 2020) and GCNanoLiteV HW from V1311p	Section 7.3: vg_lite_blit and vg_lite_blit_rect blend parameter, note expanded.

Document Revision	Date	Compatible SW Release	Notes
1.01	2020-03-27	VGLite as of pre-release sw (March 2020) and GCNanoLiteV HW from V1311p	Section 3.1: Add vg_lite_feature_t enum values for scissor, ARGB2 and border culling. Section 4.1: Add vg_lite_set_command_buffer_size. Section 5.1 and 5.4: add notes re alignment for tiled data. Section 7.3: Expand notes for vg_lite_blit* description and blend parameter; add 3 scissor functions. Add more hyperlinks to enums and structures. Miscellaneous refinements. Distribution Level now A.
1.00	2019-12-24	VGLite as of sw (December 2019) and GCNanoLiteV HW from V1311p	Initial version for sw with GCNanoLiteV 1311p